UiO : Faculty of Mathematics and Natural Sciences
University of Oslo

Kurs i Programimit në Python

Nga bazat tek projektet reale

Alion Emini

Tekst zyrtar i kursit

2025

Përmbledhje

Ky libër shoqëron kursin "Kurs i Programimit në Python"dhe synon të ndërtojë një bazë solide në gjuhën Python për fillestarë dhe studentë që ndjekin programe praktike në IT dhe zhvillim software.

Çfarë ofron ky libër

- Shpjegime të qarta në shqip.
- Shembuj konkretë, të testueshëm në kompjuter.
- Ushtrime të strukturuara për çdo kapitull.
- Kapituj të plotë për strukturat bazë, funksionet, strukturat e të dhënave, punën me skedarë dhe projektet praktike.

Falënderime

Ky tekst është rezultat i shumë viteve mësimdhënieje, diskutimesh, gabimesh të bukura në laborator dhe qindra pyetjesh të sinqerta nga studentë që donin ta kuptonin *pse* funksionon diçka, jo vetëm *si* shkruhet. Çdo kapitull është formësuar nga situata reale të zhvillimit të software-it, ku zgjidhjet e thjeshta, të pastra dhe të kuptueshme janë po aq të rëndësishme sa edhe teknikat më të avancuara. Falënderoj në mënyrë të veçantë:

- studentët e *kurse.informatika*, të cilët me pyetjet, sugjerimet dhe kritikat e tyre të drejtpërdrejta ndihmuan të sqarohej çdo shembull dhe ushtrim;
- kolegët që ndanë përvojën e tyre nga industria dhe akademia, testuan projektet praktike dhe propozuan mënyra më të mira për të shpjeguar konceptet;
- komunitetin open source për mjetet, bibliotekat dhe dokumentimin që e bëjnë programimin më të aksesueshëm për të gjithë;
- kursantët online dhe lexuesit që u morën mundimin të raportojnë gabime, pasaktësi ose pjesë të paqarta — çdo email dhe mesazh ka ndihmuar që ky material të bëhet më i saktë dhe më i dobishëm;
- familjen time, për durimin dhe mbështetjen në të gjitha netët kur vetëm edhe një shembull u kthye në shumë orë pune shtesë.

Shpresoj që ky libër të të kursejë kohë, të të japë ide të reja dhe të të shtyjë të ndërtosh gjëra reale me Python. Nëse ndonjë shembull, figurë ose ushtrim të bën ta shohësh kodin me më shumë besim, atëherë puna jonë e përbashkët ia ka vlejtur.

Përmbajtja

Falënderime **iii**

Hyrje **1**

1 Files në Python **3**
1.1 Përse na duhen skedarët? . . . 3
1.2 Llojet dhe metodat e aksesit e skedarëve . . . 3
1.3 Emrat e skedarëve dhe objektet e skedarëve . . . 4
1.4 Hapja e skedarit . . . 5
1.5 Specifikimi i vendndodhjes së skedarit (path) . . . 5
1.6 Shkrimi i të dhënave në skedar . . . 6
1.7 Mbyllja e skedarit dhe buffer-i i shkrimit . . . 7
1.8 Leximi i të dhënave nga skedari . . . 8
1.9 Pozicioni i leximit (read position) . . . 9
1.10 Bashkimi i `\n` me një *string* . . . 10
1.11 Leximi i një *string*-u dhe heqja e `\n` . . . 11
1.12 Shtimi i të dhënave në një skedar ekzistues . . . 12
1.13 Shkrimi i shumë vlerave me ciklin `for` . . . 15
1.14 Leximi me ciklin `while` dhe zbulimi i fundit të skedarit (EOF) . . . 16
1.15 Leximi më i thjeshtë: `for` direkt mbi objektin e skedarit . . . 17
1.16 Përpunimi i *registreve* (records) . . . 18
1.17 Përdorimi i `with` me skedarët (context manager) . . . 21
1.18 Encoding i tekstit . . . 24
1.19 Shkrimi në skedar me `print` . . . 24
1.20 Skedarët binarë . . . 24

2 Fjalorët dhe Set **25**
2.1 Fjalorët (Dictionaries) . . . 25
2.2 Set . . . 38
2.3 Fjalorë të folezuar (*Nested Dictionaries*) . . . 46
2.4 Serializimi i objekteve (*Pickling*) . . . 48
2.5 Formati JSON dhe përdorimi në Python . . . 50

3 Programimi OOP **55**
3.1 Programimi procedural dhe i orientuar nga objektet . . . 55
3.2 Klasat . . . 57
3.3 Puna me instancat . . . 69
3.4 Teknika për projektimin e klasave . . . 73

4 dhe Polimorfizmi **81**
4.1 Hyrje në trashëgiminë . . . 81
4.2 Shembull praktikë: `Automobile` . . . 84
4.3 Trashëgimia në diagramet UML . . . 90
4.4 Polimorfizmi . . . 91
4.5 Përdorimi i `super()` dhe zgjerimi i metodave të superklasës . . . 95

4.6 Atributet e klasës dhe të instancës 96
4.7 Marrëdhëniet “is-a” dhe “has-a” 101
4.8 Duck typing dhe `isinstance` 102

Appendices **105**

A Zgjidhje të Ushtrimeve të Zgjedhura **107**

Figurat

1.1 Shkrimi i të dhënave në skedar. 4
1.2 Leximi i të dhënave në skedar. 4
1.3 Një variabël referon një objekt skedari që është i lidhur me një skedar. 5
1.4 Pozicioni i leximit fillon në fillim të skedarit dhe përparon pas çdo leximi. 9
1.5 Pozicioni i leximit pas leximit të rreshtit të parë (te `David Hume`). 10
1.6 Pozicioni i leximit pas leximit të dy rreshtave (te `Edmund Burke`). 10
1.7 Pozicioni i leximit pas leximit të të gjithë skedarit. 10
1.8 Logjika e përgjithshme për të përcaktuar fundin e një skedari. 17
1.9 Fushat në një rekord. 19
1.10 Rekordet në një skedar. 19

3.1 Një objekt përmban atribute të të dhënave dhe metoda 56
3.2 Kodi jashtë objektit ndërvepron me metodat e objektit 56
3.3 Një planimetri dhe shtëpitë e ndërtuara nga planimetria. 58
3.4 Objektet `hoursefly` and `mosquito` janë istanca të klasës `Insect` 58
3.5 Veprimet që kryhen nga deklarata `Coint()`. 62
3.6 Struktura e përgjithshme e një diagrami UML për një klasë. 73
3.7 Diagram UML për klasën `Coin`. 73
3.8 Diagram UML për klasën `CellPhone`. 73
3.9 Diagram UML për klasën `Customer`. 76
3.10 Diagram UML për klasën `Car`. 77
3.11 Diagram UML për klasën `ServiceQuote`. 78

4.1 Bletët dhe karkalecat janë versione të specializuara të një insekti. 81
4.2 Qentë si objekte të klasës `Dog`. 83
4.3 Shumë lloje të ndryshme kafshësh janë të ngjashme me qentë. 83
4.4 Trashëgimia mes klasave `Automobile`, `Car`, `Truck` dhe `SUV` në UML. 91

Tabelat

1.1 Disa nga modet më të zakonshme të skedarëve në Python 5

2.1 Disa nga metodat e fjalorit 33

Hyrje

Mirë se vjen në këtë tekst hyrës për programimin me Python. Qëllimi i këtij libri është të të mësojë të mendosh si programues: të analizosh një problem, të gjesh një zgjidhje të qartë dhe ta kthesh atë zgjidhje në kod funksional. Libri nuk supozon asnjë përvojë të mëparshme në programim dhe është i përshtatshëm si për studentë universitarë, ashtu edhe për kursantë që vijnë nga fusha të tjera. Fokusi kryesor është kombinimi i **koncepteve bazë të programimit** me **shembuj konkretë në Python**, me stil shpjegues të qartë dhe pa ngarkesë të panevojshme teorike. Në çdo kapitull do të gjesh:

- shpjegime hap-pas-hapi me gjuhë të thjeshtë;
- shembuj të vegjël që ilustrojnë një ide të vetme;
- shembuj më të mëdhenj që tregojnë gjithë procesin e zgjidhjes së një problemi;
- skema, diagrame rrjedhjeje dhe tabele që vizualizojnë logjikën;
- ushtrime të strukturuara dhe detyra praktike për projekte reale;
- theks të veçantë te stili i saktë, emërtimi i qartë dhe praktikat profesionale.

Qasja është: *fillimisht kontrolli i rrjedhës dhe mendimi algoritmik*, *më pas* ndërtimi i strukturave më të avancuara si klasat, objektet, GUI dhe puna me të dhëna.

Si ta përdorësh këtë libër

Libri është menduar të përdoret si material kryesor për kurse hyrëse në programim me Python ose kurse për logjikë programimi. Mund ta ndjekësh nga fillimi në fund, ose të kapërcish në kapitullin që të nevojitet, falë shembujve të pavarur dhe përmbledhjeve në fund të secilit kapitull.

- **Lexo shembujt duke i shtypur vetë.** Mos u mjafto me lexim vizual – shkruaji, ekzekutoji, ndryshoji.
- **Puno ushtrimet.** Aty ndodh mësimi i vërtetë: kur diçka nuk punon dhe detyrohesh të kuptosh pse.
- **Eksperimento.** Ndrysho vlera, shto raste, prish kodin me qëllim, shiko mesazhet e gabimeve.

Shembujt janë ndërtuar për t'u ekzekutuar me Python 3 (versionet moderne 3.x). Sintaksa, si p.sh. *f-strings*, përdoret në mënyrë konsistente për të reflektuar praktikat aktuale në Python.

Organizimi i kapitujve

Në vijim është një panoramë e shkurtër e temave kryesore. (Emrat dhe ndarja mund të variojnë lehtë sipas edicionit ose kursit, por logjika mbetet e njëjtë.)

Kapitulli 1: Hyrje në kompjuterët dhe programimin Çfarë është një kompjuter, si përfaqësohen të dhënat, çfarë është programi, pse përdorim gjuhë të nivelit të lartë dhe ku hyn Python. Ambienti i punës, ekzekutimi i skripteve dhe mënyra interaktive.

Kapitulli 2: Input, përpunim dhe output Variablat, tipet bazë, leximi nga tastiera, llogaritjet, shfaqja e rezultateve, formatimi i tekstit dhe hapat logjikë të zhvillimit të një programi të thjeshtë. Pseudokodi dhe diagramet e rrjedhës si mjete për të menduar para se të kodosh.

Kapitulli 3: Strukturat vendimmarrëse dhe logjika booleane Operatorët krahasues, shprehjet booleane, strukturat `if`, `if-else`, `if-elif-else`, kushtet e përbëra, krahasimet korrekte dhe gabimet tipike.

Kapitulli 4: Ciklet (repetition structures) Ciklet `while` dhe `for`, numërues, akumulatorë, *running total*, sentinel values, validimi i input-it dhe përdorimi i cikleve për automatizim.

Kapitulli 5: Funksionet Pse ndajmë kodin në funksione, si shkruhen dhe thirren, parametra dhe vlera kthimi, variabla lokale, modularizim dhe organizim i projekteve. Praktika standarde me `if __name__ == '__main__':`.

Kapitulli 6: Skedarët dhe përjashtimet (Hyrja ku futet kapitulli i këtij teksti.) Skedarë teksti, leximi/shkrimi sekuencial, record-e dhe field-e, menaxhimi i gabimeve dhe *exception handling*. Përdorimi i `with` për menaxhim të sigurt të burimeve.

Kapitulli 7: Listat dhe tuple Struktura sekuenciale, indeksimi, prerjet, iterimi, kërkimi, agregimet, list comprehensions dhe listat 2D. Si të përdorim lista për të ruajtur dhe përpunuar të dhëna.

Kapitulli 8: Stringjet Përpunimi i tekstit, metodat e dobishme, ndarja e stringjeve, analiza karakter pas karakteri, hyrje në CSV dhe parsing.

Kapitulli 9: Fjalorët dhe set-et `dict` dhe `set` si struktura të fuqishme për kërkim dhe anëtarësi, operacionet kryesore, përdorime praktike dhe serializimi i të dhënave.

Kapitulli 10–11: Klasat, OOP dhe trashëgimia Kalimi nga qasja procedurale te ajo e orientuar nga objektet. Klasat, atributet, metodat, enkapsulimi, `__init__`, aksesues/mutatorë, trashëgimia, mbishkrimi i metodave dhe polimorfizmi.

Kapitulli 12: Rekursion Si funksionet mund të thërrasin vetveten, shembuj klasikë (faktoriali, GCD, Towers of Hanoi), kur ka kuptim të përdoret rekursioni dhe kur jo.

Kapitulli 13: GUI dhe aplikacione të thjeshta Ndërtimi i ndërfaqeve grafike me `tkinter`, widget-et bazë, eventet dhe funksionet reaguse (callback), dizajn praktik GUI.

Kapitulli 14: Baza të të dhënave dhe aplikacione praktike Hyrje në SQLite, konceptet kryesore të databazës, përdorimi i SQL nga Python, operacionet CRUD dhe lidhja e programeve me të dhënat e persistuara.

Synimi përfundimtar i këtij libri është që, pasi ta kesh përfunduar, të ndihesh i aftë të lexosh dhe kuptosh kod të huaj, të shkruash skripte të dobishme vetë, të zgjidhësh probleme reale dhe të kesh themelet e duhura për të vazhduar më tej me Python, web development, data science apo fusha të tjera të inxhinierisë së software-it.

KAPITULLI 1

Files në Python

1.1 Përse na duhen skedarët?

Kur një program duhet të ruajë të dhëna për t'i përdorur më vonë, ai i *shkruan* ato në një skedar në disk. Më vonë, programi mund t'i *lexojë* të dhënat nga skedari dhe t'i rikthejë në memorie. Programet që kemi shkruar deri tani kërkojnë që përdoruesi t'i fusë të dhënat sa herë që programi ekzekutohet, sepse të dhënat që mbahen në RAM (referencuar nga variablat) *zhduken* sapo programi ndalon. Nëse duam të ruajmë të dhënat *midis* ekzekutimeve, duhet një mënyrë persistente ruajtjeje: **skedari** në disk. Pasi të dhënat të jenë shkruar në një skedar, ato *mbeten* atje edhe pasi programi përfundon dhe mund të lexohen përsëri kur të nevojiten. Shumica e paketave të zakonshme të softuerit përdorin skedarë për të ruajtur të dhëna:

- **Përpunues teksti** (p.sh. *Word*): shkruan dokumente (shkresa, raporte) dhe i ruan në skedarë për t'u redaktuar dhe printuar më vonë.
- **Redaktues imazhesh**: vizatime dhe foto (p.sh. nga kamera dixhitale) ruhen në skedarë.
- **Spreadsheet**: të dhënat numerike dhe formulat vendosen në rreshta/kolona dhe ruhen në skedarë për përdorim të mëvonshëm.
- **Lojëra**: ruajnë tabela me emra lojtarësh e pikë, madje dhe *gjendjen* e lojës për ta vazhduar më vonë.
- **Shfletues web**: ruajnë skedarë të vegjël *cookie* me informacion të sesionit (p.sh. shporta e blerjeve).

Përkufizim — Output file vs. Input file

"Shkrim" në skedar do të thotë kopjimi i të dhënave nga RAM-i (nga një variabël) *në skedar*. Skedari ku *shkruajmë* quhet **skedar daljeje** (*output file*). **"Lexim" nga skedari** do të thotë kopjimi i të dhënave nga skedari *në RAM* dhe referencimi i tyre nga një variabël. Skedari prej të cilit *lexojmë* quhet **skedar hyrjeje** (*input file*).

Sa herë që një program përdor një skedar, ndiqen gjithmonë këta tre hapa bazë:

1. **Hap skedarin.** Krijon një lidhje midis programit dhe skedarit. Hapja e një skedari *daljeje* zakonisht krijon skedarin në disk dhe lejon shkrimin. Hapja e një skedari *hyrjeje* lejon leximin.
2. **Proceso skedarin.** Shkruaj të dhëna nëse është skedar daljeje; lexo të dhëna nëse është skedar hyrjeje.
3. **Mbyll skedarin.** Kur puna përfundon, mbyllja e skedarit *shkëput* lidhjen me programin dhe siguron që të dhënat të jenë shkruar siç duhet.

1.2 Llojet dhe metodat e aksesit e skedarëve

Në përgjithësi ka dy lloje kryesore skedarësh:

- **Skedarë teksti** — përmbajtja ruhet si *tekst* (ASCII/Unicode). Edhe numrat shkruhen si karaktere, ndaj mund të hapen me redaktues teksti (p.sh. *Notepad*).
- **Skedarë binarë** — përmbajtja *nuk* është konvertuar në tekst; është e destinuar për t'u lexuar nga programe. Përmbajtja nuk është e lexueshme me redaktues teksti.

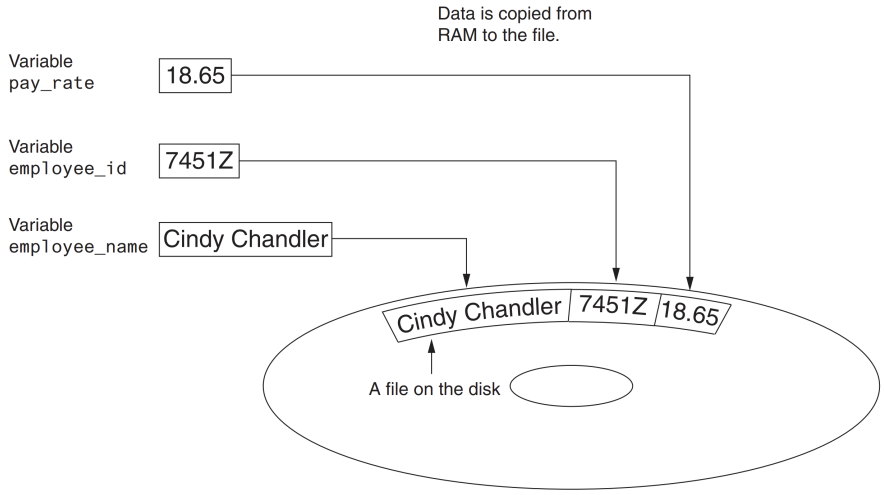

Figura 1.1: Shkrimi i të dhënave në skedar.

fig: writedata

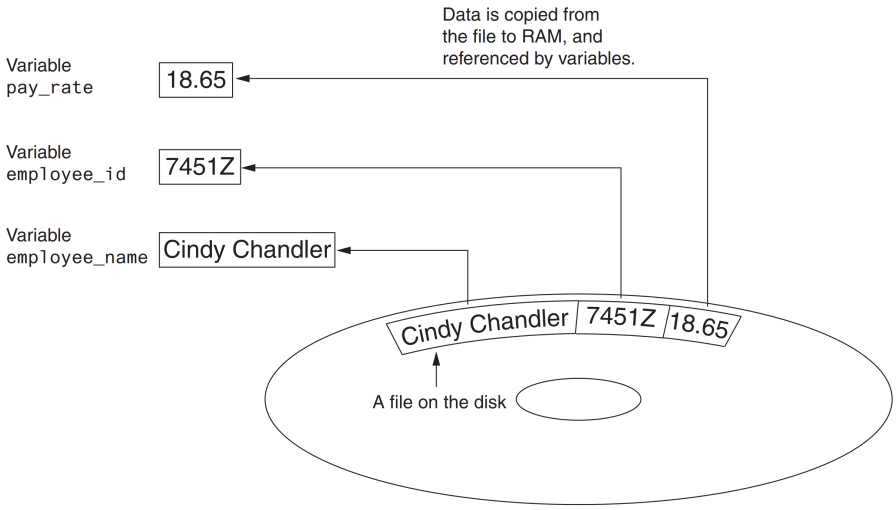

Figura 1.2: Leximi i të dhënave në skedar.

fig: readdata

Në këtë kapitull do të punojmë me **skedarë teksti**, që t'i mund të inspektohen lehtësisht me një editor dhe për të punuar më skedarët, gjuhët e programimit ofrojnë dy mënyra tipike për të aksesuar të dhënat në skedar:

- **Akses *sekuencial*** — leximi bëhet nga fillimi deri në fund. Për të arritur një pjesë në fund, duhet lexuar gjithçka që vjen përpara (si një kasetë muzikore).
- **Akses *direkt* (random)** — mund të "kërceni" direkt te cilado pjesë e skedarit (si te një CD/MP3 player).

Në këtë kapitull do të përdorim skedarë **me akses sekuencial**, të thjeshtë për t'u trajtuar dhe mjaftueshëm për të kuptuar operacionet bazë me skedarë.

1.3 Emrat e skedarëve dhe objektet e skedarëve

Çdo skedar identifikohet nga një **emër skedari** (*filename*). Shumë sisteme përdorin *zgjatime* (p.sh. `.jpg`, `.txt`, `.docx`), që tregojnë llojin e përmbajtjes (imazh JPEG, tekst, dokument Word, etj.). Që një program të punojë me një skedar në disk, ai krijon në memorie një **objekt skedari** (*file object*), i cili lidhet me skedarin përkatës dhe ofron metoda për lexim/shkrim. Në kod, një *variabël* referon këtë objekt dhe përdoret për të kryer operacionet mbi skedarin.

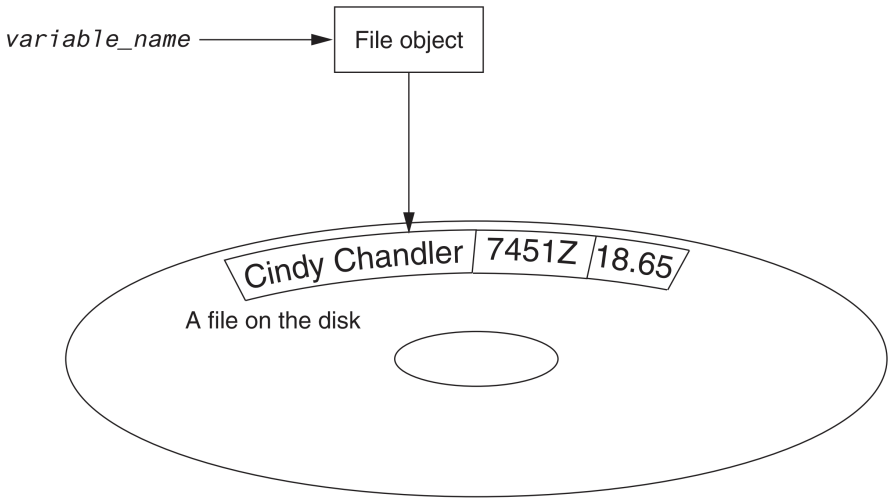

Figura 1.3: Një variabël referon një objekt skedari që është i lidhur me një skedar.

fig: fileObjec

1.4 Hapja e skedarit

Për të punuar me një skedar, përdorim funksionin `open`. Ky funksion krijon një *file object* në memorie dhe e lidh atë me një skedar fizik në disk.

```
file_variable = open(filename, mode)
```

Në këtë format: **`file_variable`** është emri i variablës që do të referojë *file object*-in, **`filename`** është një *string* me emrin e skedarit dhe **`mode`** është një *string* që tregon mënyrën e hapjes (lexim, shkrim, shtim, etj.).

Modaliteti	Përshkrimi
'r'	Hap skedarin vetëm për lexim. Skedari nuk mund të modifikohet ose të shkruhet.
'w'	Hap skedarin për shkrim. Nëse ekziston, fshihet përmbajtja e tij. Nëse nuk ekziston, krijohet.
'a'	Hap skedarin për shtim në fund. Të gjitha të dhënat do t'i shtohen fundit të skedarit. Nëse nuk ekziston, krijohet.

Tabela 1.1: Disa nga modet më të zakonshme të skedarëve në Python

tab: modesFile

Shembulli i kodit më poshtë tregon se si të hapësh një skedar për lexim dhe një tjetër për shkrim:

```
customer_file = open('customers.txt', 'r')
sales_file = open('sales.txt', 'w')
```

1.5 Specifikimi i vendndodhjes së skedarit (path)

Kur i kalon funksionit `open` vetëm emrin e skedarit (pa rrugë), Python supozon që skedari ndodhet në të njëjtin folder me programin që po ekzekutohet. Për shembull, nëse programi yt ndodhet te `C:\Users\Blake\Documents\Python` dhe ekzekuton këtë rresht:

```
test_file = open('test.txt', 'w')
```

atëherë skedari `test.txt` do të krijohet në të njëjtin folder me programin. Nëse dëshiron të hapësh ose krijosh një skedar në një vendndodhje tjetër, duhet të specifikosh edhe rrugën (path) përveç emrit të skedarit. Në

Windows, kur përdor rrugë absolute me `\`, është e rëndësishme të përdorësh prefiksin `r` për të treguar që string-u është *raw string*. Kjo bën që Python të lexojë `\`-t si karaktere të zakonshme, jo si escape sequences. Për shembull:

```
test_file = open(r'C:\Users\Blake\temp\test.txt', 'w')
```

Ky rresht krijon skedarin `test.txt` në folderin `C:\Users\Blake\temp`. Nëse nuk përdor `r`, Python mund të ngatërrojë `\`-t me escape sequences dhe të japë gabim.

```
test_file = open('test.txt', 'w')
test_file = open(r'C:\Users\Blake\temp\test.txt', 'w')
```

Në Windows, përdor `r'...'` (raw string) për rrugë me `\` që të mos interpretohen si *escape sequences*. Në Linux/macOS, përdor `'/home/user/temp/test.txt'` ose `'/Users/blake/temp/test.txt'`. Në projekte, prefero `pathlib.Path` për rrugë portabël (p.sh. `Path('temp') / 'test.txt'`).

1.6 Shkrimi i të dhënave në skedar

Deri tani, kemi përdorur funksione të ndryshme të bibliotekës së Python-it dhe kemi shkruar gjithashtu funksionet tona. Tani prezantojmë një koncept të ri: **metoda**. Një *metodë* është një funksion që i përket një objekti dhe vepron mbi të. Për shembull, pasi hapim një skedar, përdorim metodat e objektit të skedarit për të kryer operacione mbi të.

- **Funksion:** `len('abc')` — funksioni `len` përdoret mbi një objekt (string).
- **Metodë:** `'abc'.upper()` — metoda `upper()` thirret nga vetë objekti string.

Pasi një skedar është hapur për shkrim (`'w'` ose `'a'`), përdorim metodat e *file object*-it. Metoda `write(string)` shkruan *string* në skedar. Skedari duhet të jetë hapur për shkrim përndryshe ndodh gabim.

```
customer_file.write('Charles Pace')
```

```
name = 'Charles Pace'
customer_file.write(name)
```

Në shembullin e dytë, vlera e variablës `name` shkruhet në skedarin e lidhur me `customer_file`. Mund të shkruani edhe numra, por duhet t'i ktheni në string me `str()` ose f-strings. Metoda `write()` gjithashtu kthen një **numër të plotë** që tregon sa karaktere janë shkruar në skedar. Zakonisht, ky vlerë injorohet, por mund të përdoret për të kontrolluar nëse shkrimi u krye me sukses.

```
n = customer_file.write('Charles Pace')
print(n)
```

Output-i i programit të mësipërm do të ishte:

```
12
```

1.7 Mbyllja e skedarit dhe buffer-i i shkrimit

Kur programi përfundon punën me një skedar, duhet ta mbyllë atë. Mbyllja e skedarit shkëput lidhjen me skedarin dhe siguron që të gjitha të dhënat e pambetura në *buffer* të shkruhen në disk. Në disa sisteme, nëse nuk mbyllet skedari, mund të humben të dhëna, sepse të dhënat fillimisht ruhen në një *buffer* në memorie për performancë më të mirë. Vetëm kur buffer-i mbushet ose skedari mbyllet, të dhënat ruhen fizikisht në disk.

```
customer_file.close()
```

Programi 1 (`file_write.py`)

```
def main():
    # Hape skedarin "philosophers.txt" për shkrim.
    outfile = open('philosophers.txt', 'w')

    # Shkruaj tre emra filozofësh (secili me \n në fund).
    row1 = outfile.write('John Locke\n')
    row2 = outfile.write('David Hume\n')
    row3 = outfile.write('Edmund Burke\n')

    # Mbyll skedarin.
    outfile.close()

    # Printimi i numrit të karaktereve të shkruara në skedar.
    print(f'Numri i karaktereve të shkruara në rreshtin e parë: {row1}')
    print(f'Numri i karaktereve të shkruara në rreshtin e dytë: {row2}')
    print(f'Numri i karaktereve të shkruara në rreshtin e tretë: {row3}')

if __name__ == '__main__':
    main()
```

Output-i i programit të mësipërm do të ishte:

```
Numri i karaktereve të shkruara në rreshtin e parë: 12
Numri i karaktereve të shkruara në rreshtin e dytë: 11
Numri i karaktereve të shkruara në rreshtin e tretë: 13
```

Përmbajtja e skedarit `philosophers.txt` do të ishte:

```
John Locke
David Hume
Edmund Burke
```

Në programin e mësipërm, hapet skedari `philosophers.txt` me modalitetin `'w'`. Kjo bën që skedari të krijohet (nëse nuk ekziston) dhe të hapet për shkrim. Po ashtu, krijohet një objekt skedari në memorie dhe i caktohet variablës `outfile`. Në këtë program shkruhen tre stringje në skedar. Çdo rresht përdor metodën `write()` për të shtuar një emër filozofi, ku secili string përfundon me `\n` (escape sequence për rresht të ri). Më pas mbyllet skedari. Pas ekzekutimit të këtij programi, përmbajtja e skedarit `philosophers.txt` në të vërtet ka formatin e mëposhtëm:

```
John Locke\textbackslash nDavid Hume\textbackslash nEdmund Burke\textbackslash n
```

Vini re se çdo string i shkruar përfundon me `\n`, që është escape sequence për rresht të ri. Kjo jo vetëm ndan elementët në skedar, por bën që secili të shfaqet në rresht të veçantë kur hapet me një editor teksti (p.sh. Notepad).

1.8 Leximi i të dhënave nga skedari

Kur një skedar është hapur për lexim (`'r'`), mund të përdorim metodat e objektit të skedarit për të marrë të dhënat nga ai. Dy metodat më të përdorura janë:

- `read()` — lexon **të gjithë** përmbajtjen e skedarit si një *string* të vetëm.
- `readline()` — lexon **një rresht** nga skedari (deri në `\n`), përfshirë edhe `\n`-në në fund të rreshtit.

Programi 2 (`file_read.py`)

Nëse dëshiron të lexosh të gjithë përmbajtjen e një skedari në memorie, përdor metodën `read()` . Kjo është e përshtatshme kur skedari është i vogël dhe mund të mbahet i tëri në RAM.

```
def main():
    # Hap skedarin për lexim.
    infile = open('philosophers.txt', 'r')

    # Lexo të gjithë përmbajtjen si një string.
    file_contents = infile.read()

    infile.close()

    # Shtyp përmbajtjen e lexuar.
    print(file_contents)

if __name__ == '__main__':
    main()
```

Output-i i programit të mësipërm do të ishte:

```
John Locke
David Hume
Edmund Burke
```

Kur thërret `infile.read()` , përmbajtja e plotë e skedarit lexohet dhe ruhet si një string në variablën `file_contents` . Në këtë rast, variabili `file_contents` do të jetë një string që përmban të gjithë këtë tekst:

```
John Locke\nDavid Hume\nEdmund Burke\n
```

duke përfshirë karakteret `\n` që ndajnë rreshtat. Kur thërret `print(file_contents)` , Python shfaq tekstin në ekran, duke respektuar karakteret e rreshtit të ri (`\n`), kështu që secili emër shfaqet në rresht të veçantë.

Programi 3 (`line_read.py`)

Shpesh, programet kanë nevojë të lexojnë dhe të përpunojnë të dhënat nga skedari një nga një, p.sh. për të llogaritur totalin e disa vlerave ose për të analizuar çdo rresht veçmas. Për këtë përdoret metoda `readline()`, e cila lexon një rresht të vetëm nga skedari dhe e kthen si string (përfshirë `\n` në fund, nëse ekziston).

```
def main():
    infile = open('philosophers.txt', 'r')

    # Lexo tre rreshta nga skedari.
    line1 = infile.readline()
    line2 = infile.readline()
    line3 = infile.readline()

    infile.close()

    # Kujdes: çdo line përmban edhe '\n'.
    print(line1)
    print(line2)
    print(line3)

if __name__ == '__main__':
    main()
```

Output-i i programit të mësipërm do të ishte:

```
John Locke

David Hume

Edmund Burke
```

Çdo thirrje e `readline()` lexon rreshtin pasues nga skedari, duke përfshirë karakterin `\n` në fund të stringut (nëse ekziston). Kjo bën që kur përdor `print(line1)`, të shfaqet një rresht bosh midis emrave, sepse `print()` shton një `\n` tjetër. Për të shmangur këtë, mund të përdorësh `print(line1, end="")`.

Nëse rreshti i fundit në skedar **nuk** përfundon me `\n`, metoda `readline()` do të kthejë rreshtin pa karakterin e rreshtit të ri.

1.9 Pozicioni i leximit (read position)

Kur hap një skedar për lexim, Python mban një *pozicion leximi* (*read position*), që tregon se ku ndodhet leximi aktual në skedar. Fillimisht, ky pozicion është në fillim të skedarit. Sa herë që thërret `read()` ose `readline()`, pozicioni përparon përkatësisht në fund të skedarit ose në fillim të rreshtit tjetër.

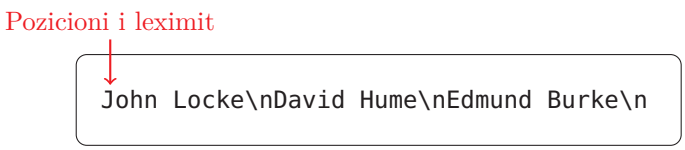

Figura 1.4: Pozicioni i leximit fillon në fillim të skedarit dhe përparon pas çdo leximi.

Pas çdo thirrjeje të `readline()`, pozicioni i leximit zhvendoset në fillim të rreshtit tjetër. Kjo lejon që të lexosh skedarin rresht pas rreshti, derisa të arrish fundin.

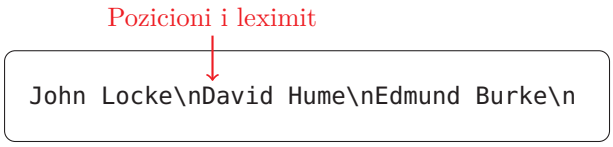

Figura 1.5: Pozicioni i leximit pas leximit të rreshtit të parë (te `David Hume`).

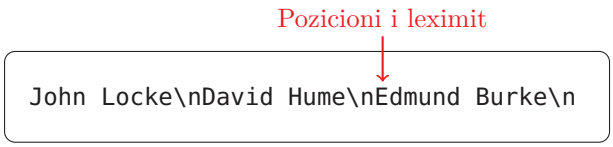

Figura 1.6: Pozicioni i leximit pas leximit të dy rreshtave (te `Edmund Burke`).

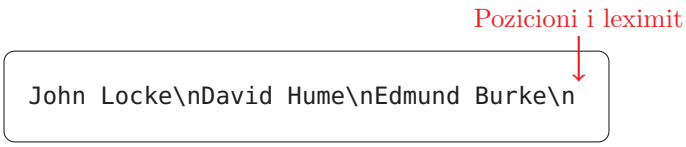

Figura 1.7: Pozicioni i leximit pas leximit të të gjithë skedarit.

1.10 Bashkimi i `\n` me një *string*

Në *Programin 1* shkruam tre *string literals* dhe secili përfundonte me `\n`. Në praktikë, të dhënat që shkruhen në skedar vijnë zakonisht nga **variablat** (p.sh. të futura nga përdoruesi), jo nga literalet. Në këto raste, para se t'i shkruajmë në skedar, zakonisht **bashkojmë** (*concatenate*) `\n` në fund, që çdo pjesë të dhënash të jetë në **rresht më vete**.

Programi 4 (`write_names.py`)

```
def main():
    print('Shkruani emrat e tre shokëve.')
    name1 = input('Shoku #1: ')
    name2 = input('Shoku #2: ')

    # Hape skedarin friends.txt për shkrim.
    myfile = open('friends.txt', 'w')

    # Shkruaji emrat në skedar (secilin në një rresht).
    myfile.write(name1 + '\n')
    myfile.write(name2 + '\n')

    myfile.close()
    print('Emrat u shkruan në friends.txt.')

if __name__ == '__main__':
    main()
```

Output-i i programit të mësipërm do të ishte:

```
Shkruani emrat e tre shokëve.
Shoku #1: Joe
Shoku #2: Rose
Emrat u shkruan në friends.txt.
```

Pas këtij ekzekutimi, përmbajtja e `friends.txt` do të jetë (me `\n` që ndan rreshtat):

```
Joe\textbackslash nRose\textbackslash nGeri\textbackslash n
```

ose, kur hapet me një editor teksti:

```
Joe
Rose
Geri
```

Rreshtat e shkrimit mund të shkruhen edhe si:

- `myfile.write(f'{name1}\n')`
- `myfile.write(f'{name2}\n')`
- `myfile.write(f'{name3}\n')`

Kjo rrit qartësinë dhe shmang `+` për bashkim *string*-ësh.

1.11 Leximi i një *string*-u dhe heqja e \n

Kur përdorim `readline()`, *string*-u i kthyer zakonisht mbaron me `\n`. Kjo është e dobishme brenda skedarit (separon elementët), por gjatë shfaqjes mund të krijojë **rreshta bosh** (sepse edhe `print()` shton një `\n`). Zgjidhja tipike është **heqja** e `\n`-së në fund me metodën `rstrip()`.

```
name = 'Joanne Manchester\n'
name = name.rstrip('\n') # -> 'Joanne Manchester'
```

Programi 5 (`strip_newline.py`)

```
def main():
   infile = open('philosophers.txt', 'r')

   line1 = infile.readline()
   line2 = infile.readline()
   line3 = infile.readline()

   # Hiq '\n' nga fundi i secilit string (nëse ekziston).
   line1 = line1.rstrip('\n')
   line2 = line2.rstrip('\n')
   line3 = line3.rstrip('\n')

   infile.close()

   print(line1)
   print(line2)
   print(line3)

if __name__ == '__main__':
   main()
```

Output-i i programit të mësipërm do të ishte:

```
John Locke
David Hume
Edmund Burke
```

1.12 Shtimi i të dhënave në një skedar ekzistues

Derisa modaliteti `'w'` hap një skedar për **shkrim nga fillimi** dhe **fshin** çdo përmbajtje ekzistuese (ose krijon një skedar të ri nëse mungon), shpesh kemi situata kur duam të *ruajmë* të dhënat ekzistuese dhe të **shtojmë** informacione të reja në fund të skedarit. **Shtimi** (angl. *appending*) i të dhënave në një skedar do të thotë të shkruash rreshta të rinj *pas* përmbajtjes që tashmë ndodhet në skedar, pa e prekur atë. Në Python, për këtë përdorim modalitetin `'a'` (*append*) në funksionin `open()` :

- Nëse skedari ekziston, **nuk** fshihet; pozicioni i shkrimit vendoset në **fund** të përmbajtjes aktuale.
- Nëse skedari nuk ekziston, ai **krijohet** dhe më pas të dhënat shkruhen në të.
- Çdo `write()` do të shtojë tekst **në fund** të skedarit ekzistues.

Supozojmë se skedari `friends.txt` përmban fillimisht:

```
Joe
Rose
Geri
```

Për të shtuar emra të tjerë në fund, pa fshirë këta që kemi, përdorim modalitetin `'a'` :

```
myfile = open('friends.txt', 'a')
myfile.write('Matt\n')
myfile.write('Chris\n')
myfile.write('Suze\n')
myfile.close()
```

Pas ekzekutimit, `friends.txt` do të përmbajë:

```
Joe
Rose
Geri
Matt
Chris
Suze
```

Vini re që **asnjë** nga rreshtat ekzistues nuk është fshirë; emrat e rinj janë thjesht shtuar në fund të skedarit.

Programi 6 (`file_names.py`)

```
def main():
   outfile = open('philosophers.txt', 'a')

   outfile.write('John Locke\n')
   outfile.write('David Hume\n')
   outfile.write('Edmund Burke\n')
   outfile.close()

   print('The names were appended to philosophers.txt.')

if __name__ == '__main__':
   main()
```

Output-i i programit të mësipërm do të ishte:

```
The names were appended to philosophers.txt.
```

Programi 7 (`write_numbers.py`)

Metoda `write()` mund të shkruajë **drejtpërdrejt** vetëm *string*-ë në një skedar teksti. Numrat *duhet* të kthehen më parë në *string*. Për këtë përdorim funksionin e brendshëm `str()`. Nëse `num = 99`, shprehja `str(num)` kthen `'99'`.

```
def main():
   outfile = open('numbers.txt', 'w')

   num1 = int(input('Shkruaj një numër: '))
   num2 = int(input('Shkruaj një numër tjetër: '))

   # Shkruaji numrat në skedar (secili në rresht më vete).
   outfile.write(str(num1) + '\n')
   outfile.write(str(num2) + '\n')

   outfile.close()
   print('Të dhënat u shkruan në numbers.txt')

if __name__ == '__main__':
   main()
```

Output-i i programit të mësipërm do të ishte:

```
Shkruaj një numër: 22
Shkruaj një numër tjetër: 14
Të dhënat u shkruan në numbers.txt
```

Shprehja `str(num1) + '\n'` konverton vlerën e `num1` në *string* dhe i bashkon `\n` në fund (p.sh. prodhon `'22\n'`). E njëjta gjë ndodh për `num2` dhe `num3`. Pas ekzekutimit, `numbers.txt` përmban:

```
22
14
```

Gabime tipike

- **Shkrim pa konvertim:** `outfile.write(num1)` *hedh gabim* sepse `write()` pret *string*.
- **Lexim pa konvertim:** vlerat e lexuara janë *string*-ë (p.sh. `'22\n'`); përpara llogaritjeve përdor `int()` ose `float()`. Përmbajtje jo-numerike do të shkaktojë `ValueError`.

Programi 8 (`read_numbers.py`)

Kur lexon nga skedari me `readline()`, përfiton një *string*. P.sh.:

```
infile = open('numbers.txt', 'r')
value = infile.readline() # -> p.sh. '22\n'
infile.close()
```

Për të bërë llogaritje, ktheje në tip numerik:

```
infile = open('numbers.txt', 'r')
string_input = infile.readline() # '22\n'
value = int(string_input) # -> 22 (int ignoron '\n')
infile.close()
```

Shpesh e kombinojmë në një rresht:

```
infile = open('numbers.txt', 'r')
value = int(infile.readline()) # readline() -> '22\n' -> int(...) -> 22
infile.close()
```

```
def main():
    infile = open('numbers.txt', 'r')

    num1 = int(infile.readline())
    num2 = int(infile.readline())
    num3 = int(infile.readline())

    infile.close()

    # Llogarit totalin.
    total = num1 + num2 + num3

    # Shfaq numrat dhe totalin.
    print(f'The numbers are: {num1}, {num2}, {num3}')
    print(f'Their total is: {total}')

if __name__ == '__main__':
    main()
```

Output-i i programit të mësipërm do të ishte:

```
The numbers are: 22, 14, -99
Their total is: -63
```

Rreth `int()` dhe `float()`

`int()` dhe `float()` **injorojnë** `\n` në fund të *string*-ut; por do të hedhin `ValueError` nëse *string*-u nuk paraqet një numër të vlefshëm.

1.13 Shkrimi i shumë vlerave me ciklin `for`

Shumë programe ruajnë *shuma* të mëdha të dhënash në skedarë. Në këto raste, shkrimi dhe leximi bëhen zakonisht me **cikle** (*loops*). Ciklet janë një nga konceptet më të rëndësishme në programim, sepse lejojnë që një bllok kodi të ekzekutohet shumë herë, duke përpunuar të dhëna të shumta pa përsëritur kodin manualisht. Kur punojmë me skedarë, ciklet na lejojnë të shkruajmë ose të lexojmë të gjitha të dhënat, pavarësisht sa të mëdha janë ato, në mënyrë të automatizuar dhe efikase. Pse përdorim cikle me skedarë?

- **Automatizim:** Në vend që të shkruajmë ose lexojmë çdo vlerë me dorë, cikli e bën këtë automatikisht për çdo element.
- **Shkallëzim:** Nuk ka rëndësi nëse kemi 3 apo 3000 rreshta; kodi mbetet i njëjtë.
- **Menaxhim i lehtë i të dhënave:** Mund të përpunojmë të gjitha të dhënat në skedar, të llogarisim statistika, të kërkojmë vlera, etj.

Kur duam të ruajmë shumë vlera në një skedar, përdorim një cikël `for` për të përsëritur procesin e marrjes së të dhënave dhe shkrimit të tyre. Kjo është shumë më efikase sesa të përsërisim kodin për çdo vlerë.

Programi 9 (`write_sales.py`)

```
def main():
    num_days = int(input('Për sa ditë ke shitje? '))

    sales_file = open('sales.txt', 'w')

    # Për çdo ditë, merre shumën dhe shkruaje në skedar.
    for count in range(1, num_days + 1):
        sales = float(input(f'Fut shitjet për ditën #{count}: '))
        sales_file.write(f'{sales}\n')

    sales_file.close()
    print('Të dhënat u shkruan në sales.txt.')

if __name__ == '__main__':
    main()
```

Në këtë shembull, cikli `for` përsëritet aq herë sa ditë ka futur përdoruesi. Çdo vlerë futet nga përdoruesi dhe shkruhet në skedar në një rresht të veçantë. Kjo mënyrë është shumë më e thjeshtë dhe më e sigurt sesa të shkruash manualisht për çdo ditë.

```
Për sa ditë ke shitje? 5
Fut shitjet për ditën #1: 1000.0
Fut shitjet për ditën #2: 2000.0
Fut shitjet për ditën #3: 3000.0
Fut shitjet për ditën #4: 4000.0
Fut shitjet për ditën #5: 5000.0
Të dhënat u shkruan në sales.txt.
```

Përmbajtja e skedarit `sales.txt` do të ishte:

```
1000.0
2000.0
3000.0
4000.0
5000.0
```

1.14 Leximi me ciklin `while` dhe zbulimi i fundit të skedarit (EOF)

Shpesh nuk e dimë *paraprakisht* sa rreshta ka një skedar. P.sh., `sales.txt` mund të ketë 5, 100 apo 1000 rreshta, varësisht çfarë futi përdoruesi. Si ta dimë kur kemi arritur **fundin e skedarit**?

Përkufizim — Çfarë është EOF? Priming read dhe cikli sentinel.

1. **EOF** (*End Of File*) është një shenjë që tregon se nuk ka më të dhëna për t'u lexuar nga skedari. Në Python, kur thërret `readline()` dhe ke arritur fundin e skedarit, ajo kthen një **string bosh** (`''`). Kjo është mënyra standarde për të zbuluar fundin e skedarit në shumë gjuhë programimi.
2. **Priming read** është një teknikë ku lexon një vlerë para se të hysh në ciklin `while`. Kjo vlerë përdoret për të kontrolluar nëse duhet të hysh në cikël. Brenda ciklit, pas përpunimit të vlerës, lexon vlerën tjetër. Ky model quhet edhe *cikli sentinel*, sepse ndalon kur has një vlerë të veçantë (në këtë rast, stringun bosh që tregon EOF). Ky model është shumë i përdorur në programim, jo vetëm për skedarë, por edhe për hyrje të tjera të të dhënave.

Programi 10 (`read_sales.py`)

```python
def main():
    sales_file = open('sales.txt', 'r')
    line = sales_file.readline()

    while line != '':
        amount = float(line)
        print(f'{amount:.2f}')
        line = sales_file.readline()

    sales_file.close()

if __name__ == '__main__':
    main()
```

Output-i i programit të mësipërm do të ishte:

```
1000.00
2000.00
3000.00
4000.00
5000.00
```

Në këtë shembull, cikli `while` vazhdon të lexojë dhe të përpunojë rreshta derisa të arrihet fundi i skedarit (EOF). Kjo është mënyra më klasike për të lexuar të gjitha të dhënat nga një skedar kur nuk dihet sa rreshta ka.

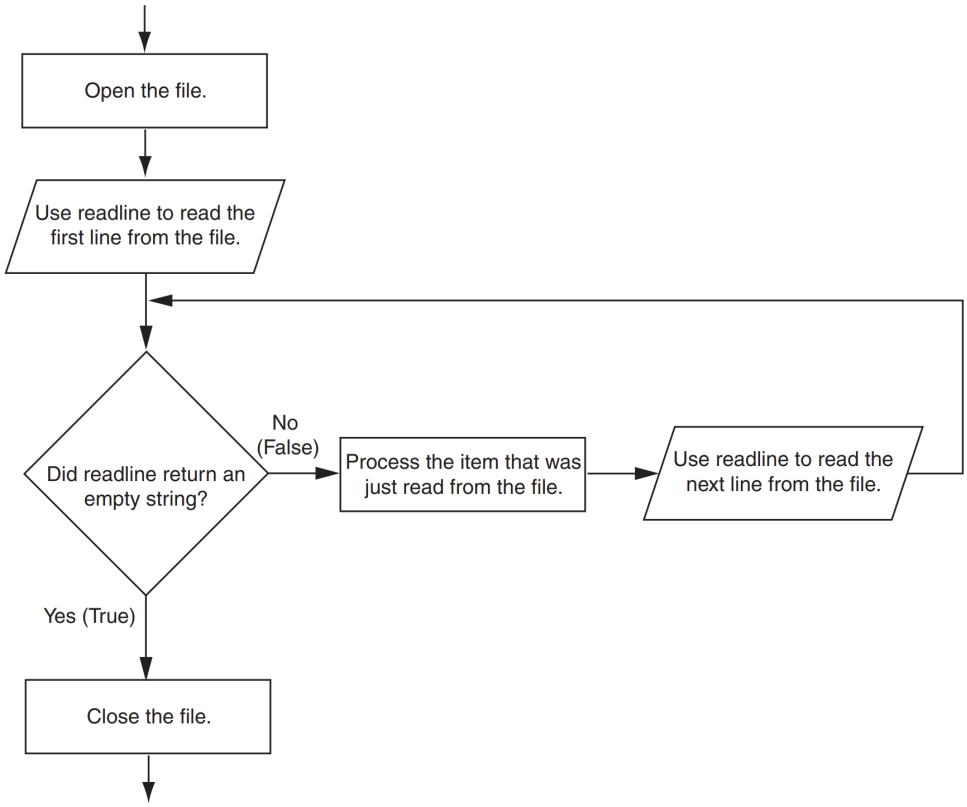

Figura 1.8: Logjika e përgjithshme për të përcaktuar fundin e një skedari.

fig: EOFLogic

Mos harro konvertimin

`readline()` kthen *string*; duhet `float(...)` ose `int(...)` përpara llogaritjeve ose formatimit numerik. Tekst i pavlefshëm numerik do të shkaktojë `ValueError`.

1.15 Leximi më i thjeshtë: `for` direkt mbi objektin e skedarit

Në Python, mund të iterosh **drejtpërdrejt** mbi rreshtat e një skedari duke përdorur një cikël `for`. Kjo është një nga veçoritë më të fuqishme të Python-it, sepse e bën kodin më të shkurtër, më të lexueshëm dhe më të sigurt. Objekti i skedarit në Python është një *iterable*, që do të thotë se mund të përdoret në një cikël `for` për të marrë rresht pas rreshti automatikisht. Python kujdeset vetë për leximin dhe ndaljen në fund të skedarit (EOF), pa pasur nevojë për *priming read* ose kontroll manual të stringut bosh.

```
for line in file_object:
    print(line)
```

Ky cikël `for` lexon rresht pas rreshti dhe **ndalon vetë** në fund të skedarit, pa pasur nevojë për *priming read* apo testim explicit për `''`.

Programi 11 (`read_sales2.py`)

```python
def main():
   sales_file = open('sales.txt', 'r')

   # Lexo të gjitha rreshtat një nga një.
   for line in sales_file:
      amount = float(line)
      print(f'{amount:.2f}')

   sales_file.close()

if __name__ == '__main__':
   main()
```

Output-i i programit të mësipërm do të ishte:

```
1000.00
2000.00
3000.00
4000.00
5000.00
```

Ky stil është më i thjeshtë, më i sigurt dhe më i lehtë për t'u mirëmbajtur. Python kujdeset automatikisht për ndaljen e ciklit kur arrihet fundi i skedarit.

Kur të përdorësh `for` dhe kur `while`

- Përdor `for line in file` kur do të lexosh të gjitha rreshtat një nga një, pa kushtëzime të veçanta.
- Përdor `while` me *priming read* kur ke nevojë për kontroll më të hollësishëm të rrjedhës së leximit, ose kur duhet të ndalosh në një kusht të caktuar (jo vetëm në fund të skedarit).

1.16 Përpunimi i *regjistreve* (records)

Përkufizim — Record dhe field

Record (*regjistër*) është një *set i plotë* të dhënash që përshkruan një njësi (p.sh. një punonjës). **Field** (*fushë*) është një copë e vetme informacioni brenda një record-i (p.sh. emri, ID-ja, departamenti). Kur shkruajmë në një skedar me akses sekuencial, fusha e një record-i shkruhen *njëra pas tjetrës* në rreshta të ndarë me `\n`.

Në programim, një **record** është një strukturë që përmbledh disa të dhëna të lidhura logjikisht, që i përkasin një entiteti të vetëm. Çdo pjesë e të dhënave quhet *field*. Për shembull, një punonjës mund të përshkruhet nga emri, ID-ja dhe departamenti. Këto janë fushat e record-it të punonjësit. Në skedarë tekst, çdo fushë zakonisht ruhet në një rresht të veçantë, dhe të gjitha fushat njëra pas tjetrës përbëjnë një record të plotë. Kjo mënyrë është shumë e përdorur për të ruajtur të dhëna të strukturuara në mënyrë të thjeshtë dhe të lexueshme.

Record-et na lejojnë të ruajmë të dhëna të lidhura së bashku në mënyrë të organizuar. Kjo është thelbësore kur kemi shumë njësi të të njëjtit lloj (p.sh. shumë punonjës, studentë, produkte), sepse na lejon të lexojmë, përpunojmë dhe modifikojmë të dhënat për secilën njësi në mënyrë të qëndrueshme.

Kur ruajmë shumë rekord-e në një skedar, është shumë e rëndësishme që të gjitha rekord-et të kenë të njëjtën strukturë (të njëjtat fusha, në të njëjtin rend). Kjo siguron që gjatë leximit të dimë saktësisht se çfarë përfaqëson çdo rresht. Nëse struktura prishet (p.sh. mungon një fushë), të gjitha leximet pasuese do të jenë të gabuara.

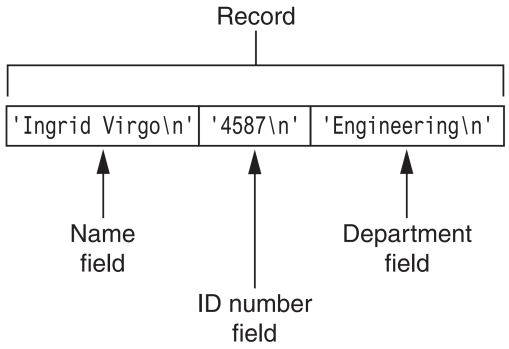

Figura 1.9: Fushat në një rekord.

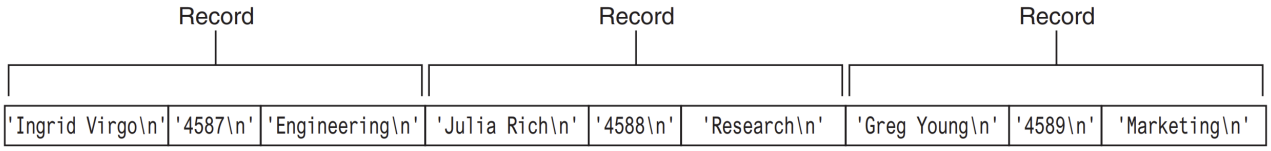

Figura 1.10: Rekordet në një skedar.

Programi 12 (`save_emp_records.py`)

Kur duam të ruajmë një listë të të dhënave të strukturuara (si një listë punonjësish), përdorim një cikël për të marrë të dhënat për çdo record dhe i shkruajmë në skedar, një fushë për rresht. Kjo mënyrë është shumë e përdorur në programim, sidomos për të ruajtur të dhëna që më vonë do të lexohen dhe përpunohen nga programe të tjera.

```
def main():
   # Merre numrin e record-eve.
   num_emps = int(input('Sa record-e punonjësish do të krijosh? '))

   emp_file = open('employees.txt', 'w')

   for count in range(1, num_emps + 1):
      print(f'Fut të dhënat për punonjësin #{count}')
      name = input('Name: ')
      id_num = input('ID number: ')
      dept = input('Department: ')

      # Shkruaji fushat (secila në rresht më vete).
      emp_file.write(f'{name}\n')
      emp_file.write(f'{id_num}\n')
      emp_file.write(f'{dept}\n')

      print()

   emp_file.close()
   print('Record-et u ruajtën në employees.txt.')

if __name__ == '__main__':
   main()
```

Output-i i programit të mësipërm do të ishte:

```
Sa record-e punonjësish do të krijosh? 3
Fut të dhënat për punonjësin #1
Name: Ingrid Virgo
ID number: 4587
Department: Engineering

Fut të dhënat për punonjësin #2
Name: Julia Rich
ID number: 4588
Department: Research

Fut të dhënat për punonjësin #3
Name: Greg Young
ID number: 4589
Department: Marketing

Record-et u ruajtën në employees.txt.
```

Rëndësi e renditjes së fushave

Renditja e fushave në **shkrim** duhet të përputhet saktësisht me renditjen e **leximit**. Në shembullin tonë: *name* → *id* → *dept*.

Ciklet na lejojnë të përpunojmë një numër të madh record-esh pa përsëritur kodin manualisht për secilin. Kjo është thelbësore për shkallëzim dhe mirëmbajtje të kodit. Nëse struktura e record-it ndryshon (p.sh. shtohet një fushë e re), mjafton të ndryshojmë vetëm pjesën e ciklit, jo të gjithë kodin.

Programi 13 (`read_emp_records.py`)

Për të lexuar record-et nga një skedar sekuencial, lexojmë *fushë pas fushe* derisa të kemi të plota të tre fushat. Përsërisim derisa të arrijmë fundin e skedarit.

```python
def main():
    emp_file = open('employees.txt', 'r')

    name = emp_file.readline()

    while name != '':
        # Lexo fushat e tjera të record-it aktual.
        id_num = emp_file.readline()
        dept = emp_file.readline()

        name = name.rstrip('\n')
        id_num = id_num.rstrip('\n')
        dept = dept.rstrip('\n')

        # Shfaq record-in.
        print(f'Name: {name}')
        print(f'ID: {id_num}')
        print(f'Dept: {dept}')
        print()

        name = emp_file.readline()

    emp_file.close()

if __name__ == '__main__':
    main()
```

Output-i i programit të mësipërm do të ishte:

```
Name: Ingrid Virgo
ID: 4587
Dept: Engineering

Name: Julia Rich
ID: 4588
Dept: Research

Name: Greg Young
ID: 4589
Dept: Marketing
```

Gabime të zakonshme me record-e

- **Fusha të munguar/a tepruara:** Nëse harron të shkruash një fushë, *sinkronizimi* prishet dhe leximet pasuese do të jenë të gabuara.
- **Rreshta të zbrazët:** Kujdes me rreshtat bosh të padëshiruar; përdor `rstrip('\n')` gjatë leximit dhe mos shkruaj `\n` shtesë.
- **Tipet:** Të gjitha fushat lexohen si *string*. Ktheji me `int(...)` / `float(...)` vetëm kur fusha është numerike.

Leximi i record-eve nga një skedar kërkon që të dimë saktësisht sa fusha ka çdo record dhe në çfarë rendi janë shkruar. Nëse struktura është e qëndrueshme, mund të lexojmë të dhënat në mënyrë të sigurt dhe të përpunojmë çdo record veçmas. Kjo teknikë është bazë për shumë aplikacione që ruajnë të dhëna të strukturuara në skedarë tekst.

1.17 Përdorimi i `with` me skedarët (context manager)

Në Python, është thelbësore që skedarët të mbyllen gjithmonë pasi të përfundojë puna me ta. Nëse harron të thërrasësh `close()`, mund të humbasësh të dhëna, të bllokosh skedarin për përdorime të tjera, ose të shkaktosh gabime të paparashikuara në program. Për ta bërë këtë proces më të sigurt dhe më të thjeshtë, Python ofron fjalën kyçe `with`, e cila përdor një mekanizëm të quajtur *context manager*. **Context manager** është një objekt që përcakton se si të menaxhohet një burim (si një skedar, një lidhje me bazë të dhënash, etj.) gjatë një blloku të caktuar të kodit. Ai siguron që burimi të hapet në fillim të bllokut dhe të mbyllet automatikisht në fund, edhe nëse ndodh një gabim (*exception*) gjatë ekzekutimit të kodit brenda bllokut. Pse është i rëndësishëm context manager-i?

- **Menaxhim automatik i burimeve:** Skedarët, lidhjet me bazë të dhënash, rrjetet, etj., janë burime që duhet të mbyllen kur nuk nevojiten më. Context manager-i siguron që ato të mbyllen gjithmonë, pa pasur nevojë për kod shtesë.
- **Mbrojtje ndaj gabimeve:** Edhe nëse ndodh një *exception* brenda bllokut, context manager-i kujdeset që burimi të mbyllet siç duhet.
- **Kod më i pastër dhe më i lexueshëm:** Sintaksa e `with` bën që kodet për hapje/mbyllje të burimeve të jenë më të qarta dhe më të lehta për t'u mirëmbajtur.

Sintaksa bazë është:

```python
with open('emri_skedarit.txt', 'w') as file:
    # Këtu shkruan ose lexon nga skedari
    file.write('Përshëndetje!')

# Kur del nga blloku, skedari mbyllet automatikisht
```

Kur përdor `with`, Python thërret automatikisht `file.close()` sapo del nga blloku, edhe nëse ndodh një gabim brenda tij. Kjo e bën kodin më të sigurt dhe shmang rrezikun e lënies së skedarëve të hapur.

Programi 14 (`with_write.py`)

Në këtë shembull, skedari `philosophers.txt` hapet për shkrim dhe mbyllet automatikisht sapo del nga blloku `with`. Nuk ka nevojë të thërrasësh `f.close()`.

```
def main():
 with open('philosophers.txt', 'w', encoding='utf-8') as f:
    f.write('John Locke\n')
    f.write('David Hume\n')
    f.write('Edmund Burke\n')

if __name__ == '__main__':
   main()
```

Përmbajtja e skedarit `philosophers.txt` do të ishte:

```
John Locke
David Hume
Edmund Burke
```

Programi 15 (`with_read.py`)

Këtu, çdo rresht i skedarit lexohet dhe konvertohet në numër. Edhe nëse ndodh një gabim gjatë leximit, skedari do të mbyllet automatikisht.

```
def main():
   with open('sales.txt', 'r', encoding='utf-8') as f:
      for line in f:
         amount = float(line)
         print(f'{amount:.2f}')

if __name__ == '__main__':
   main()
```

Output-i i programit të mësipërm do të ishte:

```
1000.00
2000.00
3000.00
4000.00
5000.00
```

Programi 16 (`with_multiple.py`)

Ky shembull tregon si mund të lexosh nga një skedar dhe të shkruash në një tjetër në të njëjtën kohë, duke përdorur dy `with` të bashkuara. Të dy skedarët do të mbyllen automatikisht, edhe nëse ndodh një gabim gjatë përpunimit.

```
def replace_department(emp_id, new_dept):
   with open('employees.txt', 'r', encoding='utf-8') as src, \
       open('employees.tmp', 'w', encoding='utf-8') as dst:

      while True:
         name = src.readline()
         if name == '':
            break
         id_num = src.readline()
         dept = src.readline()

         name = name.rstrip('\n')
         id_num = id_num.rstrip('\n')
         dept = dept.rstrip('\n')

         if id_num == emp_id:
            dept = new_dept

         dst.write(f'{name}\n{id_num}\n{dept}\n')

def main():
   replace_department('4587', 'HR')
   replace_department('4588', 'Finance')
   replace_department('4789', 'IT')

if __name__ == '__main__':
   main()
```

Përmbajtja e skedarit `employees.txt` pas ekzekutimit të `with_multiple.py` do të ishte:

```
Ingrid Virgo
4587
HR
Julia Rich
4588
Finance
Greg Young
4589
Marketing
```

Pasi del nga blloku `with`, objekti i skedarit është mbyllur dhe nuk mund të përdoret më. Nëse tenton të lexosh ose shkruash pas kësaj pike, do të marrësh gabim. Kjo sjellje është e qëllimshme për të siguruar që burimet të mos përdoren gabimisht pasi janë mbyllur. Kujdes përdorimin e objekteve të skedarëve jashtë bllokut `with` spese pas çdo përpjekje për të përdorur objektin e skedarit pasi ka përfunduar blloku `with` do të shkaktojë gabim të tipit `ValueError: I/O operation on closed file`. `with` nuk përdoret vetëm për skedarë. Mund të përdoret për menaxhimin e lidhjeve me bazë të dhënash, bllokimin e thread-eve, menaxhimin e burimeve të rrjetit, etj. Çdo objekt që implementon metodat speciale `__enter__()` dhe `__exit__()` mund të përdoret si context manager.

- Siguron mbyllje automatike të skedarëve, edhe në rast gabimi.
- E bën kodin më të pastër, më të shkurtër dhe më të lehtë për t'u mirëmbajtur.
- Është praktikë e rekomanduar në Python për çdo operacion me skedarë ose burime të tjera të jashtme.

1.18 Encoding i tekstit

Derisa deri tani kemi lënë Python-in të zgjedhë vetë encoding-un, praktikë e mirë është të jemi **eksplicit**, sidomos kur punojmë me shkronja shqipe. UTF-8 është encoding standard modern që mbështetet nga shumica e sistemeve operative dhe editorëve.

```
# Shkrim me encoding të specifikuar
with open('mesazh.txt', 'w', encoding='utf-8') as f:
    f.write('Përshëndetje, botë!\n')

# Lexim me të njëjtin encoding
with open('mesazh.txt', 'r', encoding='utf-8') as f:
    text = f.read()
    print(text)
```

Nëse nuk specifikohet encoding, Python përdor konfigurimin e sistemit. Kjo mund të funksionojë në kompjuterin tënd, por të japë gabime ose simbole të çuditshme në kompjuterë të tjerë.

1.19 Shkrimi në skedar me `print`

Për tekst të thjeshtë, mund të përdorim funksionin e zakonshëm `print`, duke i treguar në cilin skedar të shkruajë:

```
with open('log.txt', 'w', encoding='utf-8') as f:
    print('Start i programit', file=f)
    print('Ky është një mesazh tjetër.', file=f)
```

`print` shton automatikisht `\n` në fund (si në ekran), ndaj nuk kemi nevojë ta shkruajmë vetë.

1.20 Skedarët binarë

Deri tani kemi punuar me **skedarë teksti**. Por imazhet, audio, dokumentet PDF, etj. ruhen si **skedarë binarë**. Për t'i trajtuar, përdorim modalitete si `'rb'` (lexim) dhe `'wb'` (shkrim):

```
# Kopjo një skedar binar (p.sh. imazh)
with open('foto.png', 'rb') as src, open('kopje.png', 'wb') as dst:
    data = src.read()
    dst.write(data)
```

Në skedarët binarë punojmë me *bytes*, jo me *string*-ë. Ky është një hap më i avancuar që do ta prekim më shumë në kapitujt vijues.

KAPITULLI 2

Fjalorët dhe Set

2.1 Fjalorët (Dictionaries)

Përkufizim

Fjalor (*dictionary*) quhet një objekt që ruan një koleksion të dhënash. Çdo element në një fjalor ka dy pjesë: një **çelës** (*key*) dhe një **vlerë** (*value*). Përdor një çelës për të gjetur një vlerë specifike.

Kur dëgjon fjalën 'fjalor', me shumë gjasa të shkon mendja te një libër i madh si fjalori Merriam–Webster, që përmban fjalë dhe përkufizimet e tyre. Nëse do kuptimin e një fjale të caktuar, e gjen atë në fjalor për të parë përkufizimin. Në Python, një **fjalor** është një objekt që ruan një koleksion të dhënash. Çdo element i ruajtur në një fjalor ka dy pjesë: një *çelës* dhe një *vlerë*. Në fakt, elementët e fjalorit referohen shpesh si **çifte çelës–vlerë** (*key–value pairs*). Kur dëshiron të marrësh një vlerë specifike nga një fjalor, përdor çelësin që është i lidhur me atë vlerë. Kjo i ngjan procesit të kërkimit të një fjale në fjalorin Merriam–Webster, ku fjalët janë çelësa dhe përkufizimet janë vlera. Për shembull, supozo që çdo punonjës në një kompani ka një numër ID, dhe duam të shkruajmë një program që na lejon të kërkojmë emrin e një punonjësi duke futur ID-në e tij. Mund të krijojmë një fjalor ku çdo element përmban numrin e ID-së së punonjësit si *çelës*, dhe emrin e atij punonjësi si *vlerë*. Nëse dimë numrin e ID-së, mund të marrim emrin e punonjësit. Një shembull tjetër do të ishte një program që na lejon të fusim emrin e një personi dhe na jep numrin e tij të telefonit. Programi mund të përdorë një fjalor ku çdo element përmban emrin e personit si *çelës*, dhe numrin e telefonit të atij personi si *vlerë*. Nëse dimë emrin e një personi, mund të marrim numrin e tij të telefonit.

Shënim

Çiftet *çelës–vlerë* shpesh quhen edhe **mappings**, sepse çdo çelës *mapohet* te një vlerë.

Krijimi i një fjalori

Mund të krijosh një fjalor duke i futur elementët brenda një palë kllapash kaçurrelë `{ }`. Një element përbëhet nga një *çelës*, i pasuar nga dy pika `:`, i pasuar nga *vlera*. Elementët ndahen me presje. Deklarata në vijim tregon një shembull:

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'}
```

Kjo deklaratë krijon një fjalor dhe ia cakton atë variablës `phonebook`. Fjalori përmban këto tre elementë:

- Elementi i parë është `'Chris':'555-1111'`. Këtu çelësi është `'Chris'` dhe vlera është `'555-1111'`.
- Elementi i dytë është `'Katie':'555-2222'`. Këtu çelësi është `'Katie'` dhe vlera është `'555-2222'`.
- Elementi i tretë është `'Joanne':'555-3333'`. Këtu çelësi është `'Joanne'` dhe vlera është `'555-3333'`.

Në këtë shembull, çelësat dhe vlerat janë *string*-je. Vlerat në një fjalor mund të jenë objekte të çfarëdoshme tipi, por **çelësat duhet të jenë objekte të pandryshueshme** (*immutable*). Për shembull, çelësat mund të jenë *string*-je, *integers*, numra me presje lëvizëse, ose *tuple*. **Çelësat nuk mund të jenë lista** ose ndonjë tjetër tip *i ndryshueshëm*.

Sqarim i domosdoshëm

Në Python, *çelësat duhet të jenë të pandryshueshëm (immutable)* dhe *nuk* mund të jenë tipe *të ndryshueshme (mutable)* si listat.

Marrja e një vlere nga fjalori

Elementët në një fjalor **nuk** ruhen sipas një rendi të caktuar të indeksuar si në sekuenca. Për shembull, shiko sesionin e mëposhtëm interaktiv ku krijohet një fjalor dhe shfaqen elementët e tij:

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'}
print(phonebook)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{'Joanne': '555-3333', 'Chris': '555-1111', 'Katie': '555-2222'}
```

Vër re që rendi në të cilin shfaqen elementët është ndryshe nga rendi në të cilin u krijuan. Kjo tregon se fjalorët *nuk janë sekuenca* si listat, tuplet dhe stringjet. Si rezultat, **nuk** mund të përdorësh një indeks numerik për të marrë një vlerë sipas pozicionit nga një fjalor. Në vend të kësaj, përdor një *çelës* për të marrë një vlerë.

Shënim modern i dobishëm

Që nga Python 3.7, fjalorët ruajnë *rendin e futjes* gjatë iterimit. Megjithatë, fjalori nuk është strukturë e indeksuar si listat dhe *nuk* duhet përdorur për aksesim me indeks numerik.

Për të marrë një vlerë nga një fjalor, thjesht shkruan një shprehje me formatin e përgjithshëm:

```
dictionary_name[key]
```

Në këtë format, `dictionary_name` është variabla që referon fjalorin dhe `key` është një çelës. Nëse çelësi ekziston në fjalor, shprehja kthen vlerën që është e lidhur me të. Nëse çelësi nuk ekziston, ngrihet përjashtimi `KeyError`. Sesioni i mëposhtëm e demonstron këtë:

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'}
print(phonebook['Chris'])
print(phonebook['Joanne'])
print(phonebook['Katie'])
print(phonebook['Kathryn'])
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
555-1111
555-3333
555-2222
Traceback (most recent call last):
  File "script.py", line 4, in <module>
    print(phonebook['Kathryn'])
KeyError: 'Kathryn'
```

Le ta shohim më nga afër sesionin:

- **Rreshti 1** krijon një fjalor që përmban emra (si çelësa) dhe numra telefoni (si vlera).

- **Në rreshtin 2**, shprehja `phonebook['Chris']` kthen vlerën nga fjalori `phonebook` që është e lidhur me çelësin `'Chris'`. Vlera shfaqet në rreshtin 3.
- **Në rreshtin 4**, shprehja `phonebook['Joanne']` kthen vlerën e lidhur me çelësin `'Joanne'`. Vlera shfaqet në rreshtin 5.
- **Në rreshtin 6**, shprehja `phonebook['Katie']` kthen vlerën e lidhur me çelësin `'Katie'`. Vlera shfaqet në rreshtin 7.
- **Në rreshtin 8**, shkruhet `phonebook['Kathryn']`. Nuk ka një çelës `'Kathryn'` në fjalorin `phonebook`, ndaj ngrihet përjashtimi `KeyError`.

Shënim

Mbaj mend që krahasimet e *string*-jeve janë **sensibile ndaj madhësisë së shkronjave** (*case sensitive*). Shprehja `phonebook['katie']` nuk do të gjejë çelësin `'Katie'` në fjalor.

Përdorimi i operatorëve `in` dhe `not in`

Siç u demonstrua më sipër, ngrihet një `KeyError` nëse tenton të marrësh një vlerë nga një fjalor duke përdorur një çelës që nuk ekziston. Për të parandaluar një përjashtim të tillë, mund të përdorësh operatorin `in` për të përcaktuar nëse një çelës ekziston përpara se ta përdorësh atë për të marrë një vlerë. Sesioni i mëposhtëm e demonstron:

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'}
if 'Chris' in phonebook:
   print(phonebook['Chris'])
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
555-1111
```

Deklarata `if` në rreshtin 2 përcakton nëse çelësi `'Chris'` gjendet në fjalorin `phonebook`. Nëse po, deklarata në rreshtin 3 shfaq vlerën që është e lidhur me atë çelës. Mund të përdorësh gjithashtu operatorin `not in` për të përcaktuar nëse një çelës *nuk* ekziston, siç tregohet në sesionin vijues:

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222'}
if 'Joanne' not in phonebook:
   print('Joanne is not found')
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
Joanne is not found
```

Shënim

Kujto se krahasimet e *string*-jeve me operatorët `in` dhe `not in` janë **sensibile ndaj madhësisë së shkronjave** (*case sensitive*).

Shtimi i elementëve në një fjalor ekzistues

Fjalorët janë objekte *mutable*. Mund të shtosh çifte të reja *çelës–vlerë* me një deklaratë caktimi me formatin e përgjithshëm:

```
dictionary_name[key] = value
```

Në këtë format, `dictionary_name` është variabla që referon fjalorin, dhe `key` është një çelës. Nëse `key` ekziston tashmë në fjalor, vlera e tij do të zëvendësohet me `value`. Nëse çelësi nuk ekziston, do të *shtohet* në fjalor bashkë me vlerën e tij. Shembulli i mëposhtëm e demonstron:

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'}
phonebook['Joe'] = '555-0123' # shto çelës të ri
phonebook['Chris'] = '555-4444' # ndrysho vlerën e një çelësi ekzistues
print(phonebook)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{'Chris': '555-4444', 'Joanne': '555-3333', 'Joe': '555-0123', 'Katie': '555-2222'}
```

Shënim

Nuk mund të kesh **çelësa të dublikuar** në një fjalor. Nëse i cakton një vlerë një çelësi ekzistues, vlera e re *zëvendëson* vlerën e vjetër.

Fshirja e elementëve

Shpesh nuk duam vetëm të shtojmë gjëra në një fjalor, por edhe të heqim ato që nuk na duhen më (p.sh. një përdorues i fshirë, një produkt i mbaruar, një student që ka përfunduar kursin). Për të fshirë një çift ekzistues *çelës–vlerë* përdorim fjalën kyçe `del`.

```
del dictionary_name[key]
```

Pas ekzekutimit të kësaj deklarate:

- çelësi `key` hiqet nga fjalori,
- edhe vlera e lidhur me të zhduket — nuk ka më as çelës, as vlerë për atë hyrje.

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'}
print(phonebook)

del phonebook['Chris']
print(phonebook)

del phonebook['Chris'] # provë për ta fshirë sërish
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{'Chris': '555-1111', 'Joanne': '555-3333', 'Katie': '555-2222'}
{'Joanne': '555-3333', 'Katie': '555-2222'}
Traceback (most recent call last):
File "script.py", line 7, in <module>
del phonebook['Chris']
KeyError: 'Chris'
```

Çfarë ndodh këtu?

- Fillimisht, `'Chris'` është pjesë e fjalorit, ndaj `del phonebook['Chris']` funksionon normalisht.

- Pas fshirjes, çifti `'Chris': '555-1111'` nuk ekziston më.
- Thirrja e dytë `del phonebook['Chris']` tenton të fshijë një çelës që nuk ekziston më — prandaj ngrihet `KeyError`.

Kur del bëhet problem

`del` supozon që çelësi ekziston. Nëse nuk ekziston, ngrihet `KeyError`. Prandaj:

- nëse je i sigurt që çelësi ekziston (p.sh. e ke shtuar pak më parë), `del` është në rregull;
- nëse nuk je i sigurt, kontrollo paraprakisht ose përdor metoda më të sigurta.

Një mënyrë e thjeshtë për të shmangur gabimin është të testosh më parë ekzistencën e çelësit me `in`:

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'}

if 'Chris' in phonebook:
del phonebook['Chris']

print(phonebook)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{'Joanne': '555-3333', 'Katie': '555-2222'}
```

Alternativë praktike

Për raste ku nuk të intereson nëse çelësi ekziston apo jo, mund të përdorësh edhe:

- `dictionary.pop(key, default)` — heq çelësin dhe kthen vlerën; nëse mungon, kthen `default` pa ngritur gabim.

Kjo është më e "butë" se `del` dhe shpesh më praktike në programe reale.

Marrja e numrit të elementëve në një fjalor

Shpesh duam të dimë: "Sa hyrje ka ky fjalor?". Nuk numërojmë çelësat "me dorë"; përdorim funksionin e brendshëm `len`, njësoj si për lista dhe *strings*.

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222'}
num_items = len(phonebook)
print(num_items)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
2
```

`len(phonebook)` kthen **numrin e çifteve** *çelës–vlerë* në fjalor. Nëse fshin ose shton hyrje:

- shtimi i një çifti të ri e rrit rezultatin e `len`;
- fshirja e një çifti e ul rezultatin.

Për t'u mbajtur mend

`len` s'ka të bëjë me madhësinë e vlerave. P.sh. nëse vlera është një listë me 100 elementë, për fjalorin ajo prapë llogaritet si *një* hyrje.

Përzierja e tipeve të të dhënave në një fjalor

Fjalorët janë shumë fleksibël. Dy rregulla kryesore:

- **Çelësat** duhet të jenë *immutable* (p.sh. `str`, `int`, `float`, `tuple`, `bool`).
- **Vlerat** mund të jenë *çfarëdo tipi*: numra, *strings*, lista, fjalorë të tjerë, objekte të personalizuara, etj.

Shembull ku vlerat janë lista notash:

```
test_scores = {
'Kayla': [88, 92, 100],
'Luis': [95, 74, 81],
'Sophie': [72, 88, 91],
'Ethan': [70, 75, 78]
}

print(test_scores)
print(test_scores['Sophie'])

kayla_scores = test_scores['Kayla']
print(kayla_scores)
```

Këtu:

- Çelësat janë emrat e studentëve (`str`).
- Vlerat janë lista me nota (`list` me `int`).
- `test_scores['Sophie']` na jep direkt listën e notave të Sophies.

Pse është kaq i dobishëm një fjalor i tillë?

Në vend që të kemi disa lista paralele (`names`, `scores1`, `scores2` ...), i kemi të gjitha të lidhura natyrshëm: emri → lista e notave. Kjo e bën kodin më të qartë dhe më të lehtë për t'u mirëmbajtur.

Vlerat brenda *të njëjtit* fjalor mund të jenë edhe të tipeve të ndryshme; gjithashtu mund të kemi çelësa të tipeve të ndryshme (sa kohë që janë *immutable*). Megjithatë, në praktikë shpesh zgjedhim një stil të qëndrueshëm (p.sh. të gjithë çelësat `str`) që kodi të jetë më i lexueshëm.

Shembull — Fjalor me tipe të përziera të të dhënave

Shembull demonstrues (pak i ekzagjeruar) me çelësa dhe vlera të tipeve të ndryshme:

```
mixed_up = {'abc': 1, 999: 'yada yada', (3, 6, 9): [3, 6, 9]}
print(mixed_up)
```

Output i mundshëm:

```
{(3, 6, 9): [3, 6, 9], 'abc': 1, 999: 'yada yada'}
```

Këtu kemi:

- `'abc': 1` — çelës *string*, vlerë *integer*.

- `999: 'yada yada'` — çelës *integer*, vlerë *string*.
- `(3, 6, 9): [3, 6, 9]` — çelës *tuple* (i pandryshueshëm), vlerë *listë*.

Përdorim praktik i tuple si çelës

`(row, col)` si çelës për një tabelë 2D; `(lat, lon)` për koordinata gjeografike; `(year, month)` për statistika mujore, etj. *Tuple*-t janë ideale kur çelësi përbëhet nga disa pjesë.

Shembull — Të dhëna për një punonjës

Ky është një shembull shumë tipik i përdorimit të fjalorit si "mini-database" për një rekord:

```
employee = {'name': 'Kevin Smith', 'id': 12345, 'payrate': 25.75}
print(employee)
```

```
{'payrate': 25.75, 'name': 'Kevin Smith', 'id': 12345}
```

Çdo çift përfaqëson një informacion të qartë:

- `'name': 'Kevin Smith'` — emri i punonjësit.
- `'id': 12345` — identifikues unik (mund ta përdorim si referencë në sisteme të tjera).
- `'payrate': 25.75` — pagesa për orë.

Avantazhi i fjalorit:

- është **vetë-dokumentues**: `employee['name']` është shumë më i qartë se `employee[0]` ;
- lejon shtim fushe të re pa prishur strukturën ekzistuese, p.sh. `employee['active'] = True` , `employee['department'] = 'IT'` ;
- përshtatet natyrshëm për lista fjalorësh: një listë `employees` ku çdo element është fjalor `employee` .

Krijimi i një fjalori bosh

Ndonjëherë ke nevojë të nisësh me një fjalor bosh dhe ta mbushësh gjatë ekzekutimit:

```
phonebook = {}
phonebook['Chris'] = '555-1111'
phonebook['Katie'] = '555-2222'
phonebook['Joanne'] = '555-3333'
print(phonebook)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{'Chris': '555-1111', 'Joanne': '555-3333', 'Katie': '555-2222'}
```

Mund të përdorësh edhe funksionin e brendshëm `dict()` për të krijuar një fjalor bosh:

```
phonebook = dict()
```

Përdorimi i ciklit `for`

Mund të përdorësh ciklin `for` në formatin e përgjithshëm më poshtë për të iteruar mbi të *gjithë çelësat* e një fjalori:

```
for var in dictionary:
   statement
   statement
   # etj.
```

Në këtë format, `var` është emri i një variable dhe `dictionary` është emri i fjalorit. Cikli iteron një herë për çdo element të fjalorit. Çdo iterim i cakton `var`-it një *çelës*. Sesioni i mëposhtëm e demonstron:

```
1  phonebook = {
2     'Chris': '555-1111',
3     'Katie': '555-2222',
4     'Joanne': '555-3333'
5  }
6
7  for key in phonebook:
8     print(key)
9
10 for key in phonebook:
11    print(key, phonebook[key])
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
Chris
Joanne
Katie
Chris 555-1111
Joanne 555-3333
Katie 555-2222
```

Përmbledhje e deklaratave:

- **Rreshtat 7–8** përmbajnë një `for` që iteron një herë për çdo element të `phonebook`. Çdo herë `key` merr një çelës dhe `print` shfaq vlerën e `key`. Rreshtat e daljes shfaqen më sipër.
- **Rreshtat 10–11** përmbajnë një `for` tjetër që, për çdo element, shfaq çelësin dhe më pas vlerën e lidhur me të, me `phonebook[key]`.

Disa metoda të fjalorit

Objektet *dictionary* kanë disa metoda. Në këtë seksion shohim disa prej më të dobishmeve, të përmbledhura në Tabelën 2.1.

Shënim modern

Në Python 3, `items()`, `keys()` dhe `values()` kthejnë *dictionary views* (jo lista të zakonshme). `items()` është një view me `(key, value)` *tuples*, `keys()` një view me çelësa, dhe `values()` një view me vlera.

Tabela 2.1: Disa nga metodat e fjalorit

tab: dict-methods

Metoda	Përshkrimi
`clear`	Zbraz përmbajtjen e fjalorit (e lë të zbrazët).
`get`	Merr vlerën e lidhur me një çelës të caktuar. Nëse çelësi nuk gjendet, metoda *nuk* ngre përjashtim; në vend të kësaj kthen një vlerë të parazgjedhur.
`items`	Kthen të gjithë çelësat e një fjalori *dhe* vlerat e tyre të lidhura si një renditje *tuplesh*.
`keys`	Kthen të gjithë çelësat e një fjalori si një renditje *tuplesh*.
`pop`	Kthen vlerën e lidhur me një çelës të caktuar *dhe* e heq atë çift nga fjalori. Nëse çelësi nuk gjendet, metoda kthen një vlerë të parazgjedhur.
`popitem`	Kthen, si *tuple*, çiftin çelës–vlerë që u shtua së fundi në fjalor. Metoda gjithashtu e heq atë çift nga fjalori.
`values`	Kthen të gjitha vlerat në fjalor si një renditje *tuplesh*.

Metoda `clear`

`clear` fshin të gjithë elementët e një fjalori, duke e lënë bosh. Formati i përgjithshëm:

```
dictionary.clear()
```

Shembull:

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222'}
print(phonebook)
phonebook.clear()
print(phonebook)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{'Chris': '555-1111', 'Katie': '555-2222'}
{}
```

Vër re se pas thirrjes së `clear()`, fjalori `phonebook` nuk përmban elementë.

Metoda `get`

`get` mund të përdoret si alternativë e operatorit `[]` për marrjen e një vlere nga fjalori. `get` *nuk* ngre përjashtim nëse çelësi nuk gjendet. Formati i përgjithshëm:

```
dictionary.get(key, default)
```

Ku `dictionary` është emri i fjalorit, `key` është çelësi që kërkohet dhe `default` është vlera që kthehet nëse çelësi nuk gjendet.

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222'}

value = phonebook.get('Katie', 'Entry not found')
print(value)

value = phonebook.get('Andy', 'Entry not found')
print(value)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
555-2222
Entry not found
```

Përmbledhje:

- Rreshti 3 kërkon çelësin `'Katie'` — gjehet dhe vlera i caktohet `value`.
- Rreshti 6 kërkon çelësin `'Andy'` — nuk gjendet, ndaj `'Entry not found'` i caktohet `value`.

Metoda `items`

`items` kthen të gjithë çelësat dhe vlerat e fjalorit si një *dictionary view*, ku çdo element është një *tuple* `(key, value)`.

```
phonebook = {
   'Chris': '555-1111',
   'Katie': '555-2222',
   'Joanne': '555-3333'
}

for key, value in phonebook.items():
   print(key, value)
```

```
Chris 555-1111
Joanne 555-3333
Katie 555-2222
```

Përmbledhje:

- Cikli `for` thërret `phonebook.items()`, merr një renditje tuple-sh dhe iteron një herë për çdo tuple.
- Çdo herë, vlerat e tuple-it u caktohen variablave `key` dhe `value`, të cilat shtypen me `print`.

Metoda `keys`

`keys` kthen të gjithë çelësat e fjalorit si një *dictionary view* (renditje çelësash).

```
phonebook = {
   'Chris': '555-1111',
   'Katie': '555-2222',
   'Joanne': '555-3333'
}

for key in phonebook.keys():
   print(key)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
Chris
Joanne
Katie
```

Metoda `pop`

`pop` kthen vlerën e lidhur me një çelës të caktuar *dhe* e heq çiftin nga fjalori. Nëse çelësi nuk gjendet, kthen një vlerë të parazgjedhur.

Formati i përgjithshëm:

```
dictionary.pop(key, default)
```

```
phonebook = {
   'Chris': '555-1111',
   'Katie': '555-2222',
   'Joanne': '555-3333'
}

phone_num = phonebook.pop('Chris', 'Entry not found')
print(phone_num)
print(phonebook)

phone_num = phonebook.pop('Andy', 'Element not found')
print(phone_num)
print(phonebook)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
555-1111
{'Joanne': '555-3333', 'Katie': '555-2222'}
Element not found
{'Joanne': '555-3333', 'Katie': '555-2222'}
```

Përmbledhje:

- Thirrja e parë e `pop('Chris', ...)` kthen vlerën e `'Chris'` dhe e *heq* çiftin përkatës nga fjalori.
- Thirrja e dytë kërkon `'Andy'` — nuk gjendet, ndaj kthehet mesazhi i paracaktuar.

Metoda `popitem`

`popitem` kryen dy veprime: (1) heq nga fjalori çiftin *çelës–vlerë* që u shtua *së fundi*, dhe (2) e kthen atë çift si një *tuple*. Formati i përgjithshëm:

```
dictionary.popitem()
```

Mund të përdorësh një caktim të shumëfishtë për të marrë çelësin dhe vlerën në variabla të veçantë:

```
k, v = dictionary.popitem()
```

Pas ekzekutimit, `k` merr çelësin dhe `v` merr vlerën e atij çelësi. Çifti hiqet nga fjalori.

```
phonebook = {
   'Chris': '555-1111',
   'Katie': '555-2222',
   'Joanne': '555-3333'
}

print(phonebook)
key, value = phonebook.popitem()
print(key, value)
print(phonebook)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{'Chris': '555-1111', 'Joanne': '555-3333', 'Katie': '555-2222'}
Chris 555-1111
{'Joanne': '555-3333', 'Katie': '555-2222'}
```

Kujdes

Në Python 3.7+, `popitem()` është *LIFO* (heq çiftin e *fundit* të futur). Thirrja e `popitem()` mbi një fjalor *bosh* ngre `KeyError`.

Metoda `values`

`values` kthen të gjitha vlerat (pa çelësit) si një *dictionary view*. Çdo element i view-t është një *vlerë* nga fjalori.

```
phonebook = {
   'Chris': '555-1111',
   'Katie': '555-2222',
   'Joanne': '555-3333'
}

for val in phonebook.values():
   print(val)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
555-1111
555-3333
555-2222
```

Shprehje me fjalor (*Dictionary Comprehensions*)

Dictionary comprehension është një shprehje që lexon një renditje elementësh hyrës dhe, duke i përpunuar, prodhon një *fjalor*. Ajo është e ngjashme me *list comprehensions*, por rezultat prodhon çifte *çelës–vlerë*.

Shembull — Dictionary comprehension: shembull

Shembull duke marrë listën e mëposhtme të numrave

```
numbers = [1, 2, 3, 4]
```

duam të krijojmë një fjalor ku *çelësat* janë vetë elementët e `numbers`, dhe *vlerat* janë katrorët e tyre. Pra synojmë:

```
{1: 1, 2: 4, 3: 9, 4: 16}
```

Pa *comprehension*, ka shumë gjasa të shkruanim:

```
squares = {}
for item in numbers:
   squares[item] = item**2
```

Me një **dictionary comprehension** e arrijmë *në një rresht*:

```
squares = {item: item**2 for item in numbers}
```

Pjesët e një *dictionary comprehension*

Forma e përgjithshme është `{result_expr for target in iterable}`, ku për fjalorë `result_expr` është `key_expr : value_expr`.

Në shembullin `squares = {item: item**2 for item in numbers}`:

- **Shprehja e rezultatit** (`result expression`): `item: item**2`
- **Shprehja e iterimit** (`iteration expression`): `for item in numbers`

Çdo iterim ndërton një element të fjalorit me `key = item` dhe `value = item**2`.

```
numbers = [1, 2, 3, 4]
squares = {item: item**2 for item in numbers}

print(numbers)
print(squares)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
[1, 2, 3, 4]
{1: 1, 2: 4, 3: 9, 4: 16}
```

Përdorimi i një fjalori ekzistues si hyrje.

Mund të përdorim edhe një fjalor ekzistues si burim hyrjeje. Supozojmë fjalorin:

```
phonebook = {'Chris': '555-1111', 'Katie': '555-2222', 'Joanne': '555-3333'}
```

Për të krijuar një kopje me *dictionary comprehension*:

```
phonebook_copy = {k: v for (k, v) in phonebook.items()}
```

Këtu:

- **Shprehja e rezultatit**: `k: v`
- **Shprehja e iterimit**: `for k, v in phonebook.items()`

Vër re se `phonebook.items()` kthen një renditje *tuplesh* `(key, value)`, p.sh.:

```
[('Chris', '555-1111'), ('Katie', '555-2222'), ('Joanne', '555-3333')]
```

Shembull — Kopjimi i fjalorit me dictionary comprehension

```
phonebook = {'Chris': '555-1111', 'Katie': '555-2222', 'Joanne': '555-3333'}
phonebook_copy = {k: v for (k, v) in phonebook.items()}

print(phonebook)
print(phonebook_copy)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{'Chris': '555-1111', 'Katie': '555-2222', 'Joanne': '555-3333'}
{'Chris': '555-1111', 'Katie': '555-2222', 'Joanne': '555-3333'}
```

Përdorimi i klauzolave `if` në *dictionary comprehensions*

Shpesh duam të përzgjedhim vetëm disa elementë gjatë përpunimit. Supozo fjalorin:

```
populations = {
   'New York': 8398748,
   'Los Angeles': 3990456,
   'Chicago': 2705994,
   'Houston': 2325502,
   'Phoenix': 1660272,
   'Philadelphia': 1584138
}
```

Duam një fjalor të ri që kopjon vetëm qytetet me popullsi $> 2\,000\,000$. Me një `for` të zakonshëm:

```
largest = {}
for k, v in populations.items():
   if v > 2_000_000:
      largest[k] = v
```

E njëjta gjë me një **dictionary comprehension** duke shtuar një `if` filtrues në fund:

```
largest = {k: v for k, v in populations.items() if v > 2000000}
```

Pas ekzekutimit, `largest` përmban:

```
{'New York': 8398748, 'Los Angeles': 3990456, 'Chicago': 2705994, 'Houston': 2325502}
```

2.2 Set

Përkufizim

Një **set** përmban një koleksion *vlerash unike* dhe funksionon si një *set matematikor*.
Ja disa gjëra të rëndësishme për *set*-et:

- Të gjithë elementët në një *set* duhet të jenë unikë. As dy elementë nuk mund të kenë të njëjtën vlerë.

- *Set*-et janë *të parenditura* — elementët nuk ruhen sipas ndonjë rendi të caktuar.
- Elementët e ruajtur në një *set* mund të jenë të tipeve të ndryshme të të dhënave.

Krijimi i një *set*-i

Për të krijuar një *set*, thërret funksionin e brendshëm `set`. Shembull për të krijuar një *set* bosh:

```
myset = set()
```

Pas kësaj deklarate, variabla `myset` referon një *set* bosh. Mund të kalosh edhe një *argument* te funksioni `set`. Argumenti duhet të jetë një objekt *iterable* (p.sh. listë, *tuple*, ose *string*). Elementët individualë të atij objekti bëhen elementë të *set*-it.

```
myset = set(['a', 'b', 'c'])
print(myset) # {'a', 'b', 'c'} (rendi mund të ndryshojë)
```

Nëse kalon një *string* si argument, çdo karakter individual në *string* bëhet anëtar i *set*-it:

```
myset = set('abc')
print(myset) # {'a', 'b', 'c'} (rendi mund të ndryshojë)
```

Set-et nuk mund të përmbajnë duplikime. Nëse i kalon `set`-it një argument që përmban elementë të dublikuar, vetëm *një* kopje e secilit element do të ruhet:

```
myset = set('aaabc')
print(myset) # {'a', 'b', 'c'} (rendi mund të ndryshojë)
```

Po sikur të duam të krijojmë një *set* ku *çdo element* është një *string* me më shumë se një karakter? P.sh. një *set* që përmban `'one'`, `'two'` dhe `'three'`.

```
# This is error: set accepts only one argument
myset = set('one', 'two', 'three')
```

```
# This does not make what we want
myset = set('one two three')
print(myset) # {'o', 'n', 'e', ' ', 't', 'w', 'h', 'r'} (karaktere individuale)
```

Për të marrë një *set* me `'one'`, `'two'`, `'three'`, duhet t'i kalojmë `set`-it një *listë* me këto *string*-je:

```
# This is correct
myset = set(['one', 'two', 'three'])
print(myset) # {'one', 'two', 'three'} (rendi mund të ndryshojë)
```

Marrja e numrit të elementëve në një *set*

Ashtu si te listat, *tuple*-t dhe fjalorët, mund të përdorësh `len` për të marrë numrin e elementëve të një *set*-i:

```
myset = set([1, 2, 3, 4, 5])
print(len(myset)) # 5
```

Shtimi dhe heqja e elementëve

Set-et janë objekte *mutable*, ndaj mund t'u shtosh ose t'u heqësh elementë. Përdor `add` për të shtuar një element:

```
myset = set()
myset.add(1)
myset.add(2)
myset.add(3)
print(myset) # {1, 2, 3}

myset.add(2) # 2 ekziston; s'ngre përjashtim, thjesht s'e shton
print(myset) # {1, 2, 3}
```

Përdor `update` për të shtuar disa elementë njëherësh nga një objekt *iterable* (listë, *tuple*, *string*, ose *set* tjetër):

```
myset = set([1, 2, 3])
myset.update([4, 5, 6])
print(myset) # {1, 2, 3, 4, 5, 6}
```

```
set1 = set([1, 2, 3])
set2 = set([8, 9, 10])
set1.update(set2)
print(set1) # {1, 2, 3, 8, 9, 10}
print(set2) # {8, 9, 10} (mbetet i pandryshuar)
```

```
myset = set([1, 2, 3])
myset.update('abc') # shton çdo karakter si element më vete
print(myset) # {1, 2, 3, 'a', 'b', 'c'} (rendi mund të ndryshojë)
```

Metodat `remove` dhe `discard` marrin si argument elementin për t'u hequr. Dallimi është te sjellja kur elementi *nuk* gjendet: `remove` ngre `KeyError`, ndërsa `discard` *jo*.

```
myset = set([1, 2, 3, 4, 5])
print(myset) # {1, 2, 3, 4, 5}

myset.remove(1)
print(myset) # {2, 3, 4, 5}

myset.discard(5)
print(myset) # {2, 3, 4}

myset.discard(99) # s'ngre përjashtim edhe nëse s'ekziston

myset.remove(99) # ngre KeyError
```

```
Traceback (most recent call last):
  File "script.py", line 12, in <module>
    myset.remove(99)
KeyError: 99
```

Zbrazja me `clear`.

`clear` heq të gjithë elementët e *set*-it:

```
myset = set([1, 2, 3, 4, 5])
print(myset) # {1, 2, 3, 4, 5}
myset.clear()
print(myset) # set()
```

Vër re: kur shfaqet një *set* bosh, interpretuese tregon `set()`.

Përdorimi i ciklit `for` për të iteruar mbi një *set*

Mund të përdorësh `for` në formatin e përgjithshëm:

```
for var in myset:
   statement
   statement
   # etj.
```

Cikli iteron një herë për çdo element të *set*-it, duke i caktuar `var`-it nga një element në çdo hap.

```
myset = set(['a', 'b', 'c'])
for val in myset:
   print(val)
```

Output i mundshëm (rendi s'garantohet):

```
a
c
b
```

Përdorimi i operatorëve `in` dhe `not in` në *set*

Përdor `in` për të përcaktuar nëse një vlerë ekziston në një *set*:

```
myset = set([1, 2, 3])
if 1 in myset:
   print('The value 1 is in the set.')
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
The value 1 is in the set.
```

Përdor `not in` për të përcaktuar nëse një vlerë *nuk* ekziston:

```
myset = set([1, 2, 3])
if 99 not in myset:
   print('The value 99 is not in the set.')
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
The value 99 is not in the set.
```

Gjetja e bashkimit të *set*-eve (*Union*)

Bashkimi i dy *set*-eve është një *set* që përmban të gjithë elementët e të dyjave. Në Python mund të thërrasësh metodën `union` për të marrë bashkimin e dy *set*-eve. Formati i përgjithshëm:

```
set1.union(set2)
```

Këtu `set1` dhe `set2` janë *set*-e. Metoda kthen një *set* që përmban elementët e të dy `set1` dhe `set2` (pa duplikime). Sesioni vijues e demonstron:

```
set1 = set([1, 2, 3, 4])
set2 = set([3, 4, 5, 6])
set3 = set1.union(set2)
print(set3)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{1, 2, 3, 4, 5, 6}
```

Mund të përdorësh gjithashtu operatorin `|` për të gjetur bashkimin e dy *set*-eve. Formati i përgjithshëm:

```
set1 | set2
```

Shprehja kthen një *set* me elementët e të dy `set1` dhe `set2`. Demonstrim:

```
set1 = set([1, 2, 3, 4])
set2 = set([3, 4, 5, 6])
set3 = set1 | set2
print(set3)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{1, 2, 3, 4, 5, 6}
```

Gjetja e prerjes (*Intersection*)

Prerja e dy *set*-eve është një *set* që përmban vetëm elementët e përbashkët të të dyjave. Përdor metodën `intersection` për të marrë prerjen. Formati i përgjithshëm:

```
set1.intersection(set2)
```

Metoda kthen një *set* që përmban elementët që gjenden si në `set1`, ashtu edhe në `set2`. Shembull:

```
set1 = set([1, 2, 3, 4])
set2 = set([3, 4, 5, 6])
```

```
set3 = set1.intersection(set2)
print(set3)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{3, 4}
```

Mund të përdorësh edhe operatorin `&` për të gjetur prerjen. Formati i përgjithshëm:

```
set1 & set2
```

Shprehja kthen *set*-in e elementëve të përbashkët. Demonstrim:

```
set1 = set([1, 2, 3, 4])
set2 = set([3, 4, 5, 6])
set3 = set1 & set2
print(set3)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{3, 4}
```

Gjetja e diferencës (*Difference*)

Diferenca `set1 - set2` janë elementët që shfaqen në `set1`, por *jo* në `set2`. Përdor metodën `difference` për ta marrë. Formati i përgjithshëm:

```
set1.difference(set2)
```

Metoda kthen *set*-in e elementëve që gjenden në `set1`, por jo në `set2`. Shembull:

```
set1 = set([1, 2, 3, 4])
set2 = set([3, 4, 5, 6])
set3 = set1.difference(set2)
print(set3)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{1, 2}
```

Mund të përdorësh edhe operatorin `-` për të gjetur diferencën:

```
set1 - set2
```

Shembull:

```
set1 = set([1, 2, 3, 4])
set2 = set([3, 4, 5, 6])
set3 = set1 - set2
print(set3)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{1, 2}
```

Gjetja e diferencës simetrike (*Symmetric Difference*)

Diferenca simetrike e dy *set*-eve është *set*-i i elementëve që *nuk* janë të përbashkët — pra elementët që janë në njërin *set* por jo në të dy. Përdor metodën `symmetric_difference`. Formati i përgjithshëm:

```
set1.symmetric_difference(set2)
```

Metoda kthen elementët që gjenden në `set1` *ose* `set2`, por jo në të dy. Shembull:

```
set1 = set([1, 2, 3, 4])
set2 = set([3, 4, 5, 6])
set3 = set1.symmetric_difference(set2)
print(set3)
```

```
{1, 2, 5, 6}
```

Mund të përdorësh edhe operatorin `^` për të gjetur diferencën simetrike:

```
set1 ^ set2
```

Shembull:

```
set1 = set([1, 2, 3, 4])
set2 = set([3, 4, 5, 6])
set3 = set1 ^ set2
print(set3)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{1, 2, 5, 6}
```

Nënsete (*Subsets*) dhe mbisete (*Supersets*)

Supozo se ke dy *set*-e, dhe njëri prej tyre përmban të gjithë elementët e tjetrit:

```
set1 = set([1, 2, 3, 4])
set2 = set([2, 3])
```

Në këtë shembull, `set1` përmban të gjithë elementët e `set2`, që do të thotë se `set2` është një **nënset** i `set1`. Po ashtu, `set1` është një **mbiset** i `set2`. Në Python, mund të thërrasësh metodën `issubset` për të përcaktuar nëse një *set* është nënset i tjetrit. Formati i përgjithshëm:

```
set2.issubset(set1)
```

Metoda kthen `True` nëse `set2` është nënset i `set1`, përndryshe `False`. Mund të thërrasësh metodën `issuperset` për të përcaktuar nëse një *set* është mbiset i tjetrit. Formati i përgjithshëm:

```
set1.issuperset(set2)
```

Metoda kthen `True` nëse `set1` është mbiset i `set2`, përndryshe `False`. Demonstrim:

```
set1 = set([1, 2, 3, 4])
set2 = set([2, 3])
print(set2.issubset(set1)) # True
print(set1.issuperset(set2)) # True
```

Mund të përdorësh gjithashtu operatorin `<=` për të testuar nëse një *set* është nënset i një tjetri, dhe operatorin `>=` për të testuar nëse një *set* është mbiset i një tjetri. Formati i përgjithshëm me `<=`:

```
set2 <= set1
```

Shprehja kthen `True` nëse `set2` është nënset i `set1`, përndryshe `False`. Ndërsa me `>=`:

```
set1 >= set2
```

Shprehja kthen `True` nëse `set1` është mbiset i `set2`, përndryshe `False`. Demonstrim:

```
set1 = set([1, 2, 3, 4])
set2 = set([2, 3])

print(set2 <= set1) # True
print(set1 >= set2) # True
print(set1 <= set2) # False
```

Shprehje me *set* (*Set Comprehensions*)

Set comprehension është një shprehje që lexon një renditje elementësh hyrës dhe prodhon një *set*. Ato funksionojnë si *list comprehensions*, por *set*-et shkruhen me kllapa kaçurrelë `{}` (ndërsa listat me kllapa katrore `[]`).

Shembull — Kopjimi i një *set*-i

```
set1 = set([1, 2, 3, 4, 5])
set2 = {item for item in set1}
print(set1)
print(set2) # një kopje (rendi mund të ndryshojë)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

Shembull — Katrorët e elementëve

```
set1 = set([1, 2, 3, 4, 5])
set2 = {item**2 for item in set1}
print(set2) # {1, 4, 9, 16, 25} (rendi mund të ndryshojë)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{1, 4, 9, 16, 25}
```

Shembull — Filtrim me `if`

Supozojmë se duam vetëm vlerat më të vogla se 10:

```
set1 = set([1, 20, 2, 40, 3, 50])
set2 = {item for item in set1 if item < 10}
print(set2) # {1, 2, 3} (rendi mund të ndryshojë)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{1, 2, 3}
```

Shënime të dobishme

- **Set**-et ruajnë vetëm *vlera unike*, ndaj duplikimet hiqen automatikisht gjatë ndërtimit.
- Rendi i shfaqjes së elementëve në një *set nuk garantohet*.
- `iterable` mund të jetë listë, *tuple*, *string*, *set* etj.

2.3 Fjalorë të folezuar (*Nested Dictionaries*)

Përkufizim

Fjalor i folezuar quhet një fjalor që si *vlera* përmban një ose më shumë *fjalorë* të tjerë. Kjo lejon modelimin e të dhënave në nivele (*hierarki*), p.sh. *departament* → *punonjës* → *atribute*.

```
students = {
    'A101': {'name': 'Arta', 'scores': [90, 85, 92], 'active': True},
    'B202': {'name': 'Bledi', 'scores': [78, 92], 'active': False},
}
print(students['A101']['name']) # Arta
print(students['A101']['scores'][0]) # 90
```

Krijimi dhe përditësimi

Mund të shtosh një element të ri në nivelin e jashtëm dhe pastaj të plotësosh *fjalorin e brendshëm*:

```
students['C303'] = {}
students['C303']['name'] = 'Clea'
students['C303']['scores'] = [88, 91]
students['C303']['active'] = True
```

Këshillë praktike: `setdefault`

Kur do të shtosh në mënyrë të sigurt një nivel të brendshëm, përdor `setdefault`:

```
catalog = {}
catalog.setdefault('books', {}).setdefault('python', []).append('Fluent Python')
```

Kjo krijon automatikisht fjalorët/listat që mungojnë, pa ngritur `KeyError`.

Aksesim i sigurt me `get`

`dict.get` ndihmon të shmangësh `KeyError` kur një çelës mund të mungojë:

```
dept = {
   'IT': {'manager': 'Elira', 'headcount': 12},
   'HR': {'headcount': 4}
}

manager_it = dept.get('IT', {}).get('manager', 'N/A') # 'Elira'
manager_hr = dept.get('HR', {}).get('manager', 'N/A') # 'N/A'
manager_fin = dept.get('Finance', {}).get('manager', 'N/A') # 'N/A'
print(manager_it, manager_hr, manager_fin)
```

Kujdes me kopjimin (*shallow* vs *deep*)

`dict.copy()` bën *shallow copy*: fjalori i jashtëm kopjohet, por fjalorët/listat e brendshme *ndahen* (referenca të njëjta). Përdor `copy.deepcopy` për kopjim të thellë.

```
import copy
orig = {'A101': {'scores': [90, 95]}}
shallow = orig.copy()
deep = copy.deepcopy(orig)

shallow['A101']['scores'].append(100)
print(orig) # Ndryshohet edhe orig (shallow copy)
print(deep) # I pandryshuar
```

Iterimi në nivele

Mund të *çpaketosh* çifte çelës–vlerë dhe pastaj të hysh brenda:

```
for sid, info in students.items():
   name = info.get('name', 'Unknown')
   active = info.get('active', False)
   first = info.get('scores', [None])[0]
   print(sid, name, active, first)
```

Fshirja në nivele

Fshij një fushë të brendshme ose të gjithë hyrjen:

```
del students['B202']['active'] # heq vetëm fushën 'active'
if not students['B202']: # nëse bosh, hiqe gjithë hyrjen
   del students['B202']
```

Nested dictionary comprehensions

Mund të ndërtosh struktura të folezuara me *comprehensions*:

```
names = ['Ana', 'Besa']
levels = [1, 2, 3]
squares_by_name = {
   name: {n: n**2 for n in levels}
   for name in names
}
print(squares_by_name)
# {'Ana': {1: 1, 2: 4, 3: 9}, 'Besa': {1: 1, 2: 4, 3: 9}}
```

2.4 Serializimi i objekteve (*Pickling*)

Përkufizim

Serializim quajmë procesin e shndërrimit të një objekti në një rrjedhë bajtësh (*byte stream*) që mund të ruhet në një skedar dhe të rikthehet më vonë. Në Python, serializimi i objekteve quhet **pickling**, ndërsa rikthimi i tyre quhet **unpickling**. Moduli standard `pickle` ofron funksionet përkatëse.

- Ruajtje në disk të strukturave komplekse (`dict` , `set` , lista, `tuple` , etj.).
- Dërgim midis proceseve ose seancave të ndryshme të aplikacionit.
- *Cache* e përllogaritjeve të kushtueshme.

Siguria në radhë të parë

Mos bëj `pickle.load` me të dhëna nga burime të pabesueshme. `pickle` mund të ekzekutojë kod gjatë *unpickling*. Për shkëmbim të dhënash me palë të treta përdor formate të sigurta si `json` .

Hapat bazë për *pickling*

Hapat për të ruajtur një objekt në një skedar me `pickle` janë si më poshtë:

1. `import pickle`
2. Hap një skedar për **shkrim binar** me `'wb'` .
3. Thirr `pickle.dump(obj, file, protocol=pickle.HIGHEST_PROTOCOL)` .
4. Mbyll skedarin (ose përdor `with`).

```
import pickle

```

```
phonebook = {'Chris': '555-1111', 'Katie': '555-2222', 'Joanne': '555-3333'}

with open('phonebook.dat', 'wb') as f:
   pickle.dump(phonebook, f, protocol=pickle.HIGHEST_PROTOCOL)
```

Hapat bazë për *unpickling*

Hapat për të rikthyer një objekt nga një skedar me `pickle` janë si më poshtë:

1. `import pickle`
2. Hap skedarin për **lexim binar** me `'rb'`.
3. Thirr `obj = pickle.load(file)` për të rikthyer objektin.

```
import pickle

with open('phonebook.dat', 'rb') as f:
   pb = pickle.load(f)

print(pb)
# {'Chris': '555-1111', 'Joanne': '555-3333', 'Katie': '555-2222'}
```

Shënim

Mund të **dump**-osh disa objekte rresht në të njëjtin skedar. Gjatë leximit, thirr `pickle.load` në të njëjtën renditje derisa të kapësh `EOFError` (fund i skedarit).

Programi 1 - Pickling multiple objects (`pickle_objects.py`)

```
import pickle

def main():
   again = 'y'
   output_file = open('info.dat', 'wb') # hap për shkrim binar një herë
   try:
      while again.lower() == 'y':
         person = get_person()
         pickle.dump(person, output_file, protocol=pickle.HIGHEST_PROTOCOL)
         again = input('Enter more data? (y/n): ')
   finally:
      output_file.close()

def get_person():
   person = {}
   person['name'] = input('Name: ')
   person['age'] = int(input('Age: '))
   person['weight'] = float(input('Weight: '))
   return person
```

Programi 2 - Unpickling multiple objects (`unpickle_objects.py`)

```
import pickle

def main():
   end_of_file = False
   input_file = open('info.dat', 'rb')
   try:
      while not end_of_file:
         try:
            person = pickle.load(input_file)
```

```
            display(person)
        except EOFError:
            end_of_file = True
    finally:
        input_file.close()

def display(person):
    print('Name:', person['name'])
    print('Age:', person['age'])
    print('Weight:', person['weight'])
    print()
```

Këshilla praktike

- Përdor `with open(...)` për mbyllje automatike të skedarit.
- Përdor `protocol=pickle.HIGHEST_PROTOCOL` për skedarë më kompaktë dhe më të shpejtë.
- `pickle` është **specifik për Python** dhe jo i lexueshëm nga njerëzit. Për shkëmbim ndërgjuhësh përdor `json`.

2.5 Formati JSON dhe përdorimi në Python

JSON (JavaScript Object Notation) është një *format tekstual, i lehtë dhe i ndërveprueshëm* për shkëmbim të dhënash. Është i pavarur nga gjuha, bazohet në Unicode/UTF-8 dhe përbëhet nga *objekte* (`{...}` me çifte *çelës–vlerë*) dhe *array* (`[...]`, renditje vlerash). Çelësat **duhet** të jenë *string* me thonjëza të dyfishta. Vlerat mund të jenë *string*, *numra*, *boolean* (`true` / `false`), `null`, *array* ose *objekte* të tjera.

Pse JSON?

Ideale për: API, konfigurime, log-e, skedarë të lexueshëm nga njerëzit dhe të shumëgjuhshëm. Krahaso me `pickle`: JSON është më i sigurt për shkëmbim publik dhe ndërplatformë, ndërsa `pickle` ruan objekte Python (por s'është i portueshëm dhe ka rreziqe sigurie).

Më poshtë është një shembull kompleks i një dokumenti JSON:

```
{
  "app": "KursePython",
  "version": "1.0.0",
  "last_updated": "2025-11-05T15:00:00Z",
  "users": [
    {
      "id": 1,
      "name": "Arta K.",
      "roles": ["student"],
      "active": true,
      "scores": [90, 85, 92]
    },
    {
      "id": 2,
      "name": "Bledi H.",
      "roles": ["instructor", "admin"],
      "active": false,
      "scores": null
    }
  ],
  "events": [
    {
      "event_id": "evt-100",
      "type": "login",
      "user_id": 1,
```

```
      "timestamp": "2025-11-05T14:12:00Z",
      "meta": { "ip": "192.0.2.1" }
    },
    {
      "event_id": "evt-101",
      "type": "submission",
      "user_id": 1,
      "timestamp": "2025-11-05T14:45:30Z",
      "meta": { "assignment": "hw1", "score": 88 }
    }
  ],
  "config": {
    "language": "sq",
    "features": { "json_export": true, "pickling_allowed": false }
  },
  "pricing": {
    "basic": 0,
    "pro": 29.99,
    "enterprise": null
  }
}
```

Rregulla të rëndësishme

- **Pa komente, pa presje fundore** (trailing commas).
- Çelësat **me thonjëza të dyfishta** “ `key` ” (jo `'key'`).
- Vlerat speciale si `NaN` , `Infinity` *nuk* janë pjesë e specifikimit standard.

Serializim/deserializim bazë

Serializimi dhe deserializimi i objekteve Python me JSON bëhet me funksionet `json.dumps` / `json.loads` (për string) dhe `json.dump` / `json.load` (për skedarë). Për të përdorur JSON, importo modulin `json` .

```python
import json

data = {
   "name": "Arta",
   "age": 20,
   "scores": [90, 85, 92],
   "active": True
}

# -> String JSON
s = json.dumps(data, ensure_ascii=False) # ruaj shkronjat shqipe pa \u-escape
print(s)

# <- Nga string JSON në objekte Python
obj = json.loads(s)
print(obj["name"], obj["scores"][0])
```

Për skedarë: `json.dump` / `json.load`

```python
# Writting in JSON file (pretty-printed with indentation)
with open("students.json", "w", encoding="utf-8") as f:
   json.dump(data, f, ensure_ascii=False, indent=2, sort_keys=True)

# Reading from JSON file
with open("students.json", "r", encoding="utf-8") as f:
   students = json.load(f)
```

Opsione të dobishme

`indent=2` për formatim të lexueshëm; `sort_keys=True` për rend të qëndrueshëm të çelësave; `ensure_ascii=False` për Unicode natyral (shqip).

JSON me fjalorë të folezuar

Përshtatet natyrshëm me strukturat e seksionit "Fjalorë të folezuar":

```
import json
students = {
   "A101": {"name": "Arta", "scores": [90, 85, 92], "active": True},
   "B202": {"name": "Bledi", "scores": [78, 92], "active": False}
}

# -> JSON
s = json.dumps(students, ensure_ascii=False, indent=2)
print(s)

# <- JSON
back = json.loads(s)
print(back["A101"]["scores"][0]) # 90
```

Lexim i shpejtë i rrjedhave "një-JSON-për-rresht" (NDJSON)

```
import json

# skedar ku çdo rresht është një objekt JSON i veçantë
with open("events.ndjson", "r", encoding="utf-8") as f:
   for line in f:
      if line.strip():
         evt = json.loads(line)
         # përpuno ngjarjen 'evt'
```

Trajtimi i gabimeve

```
import json
bad = '{"a": 1,}' # presja fundore e paligjshme

try:
   json.loads(bad)
except json.JSONDecodeError as e:
   print("JSON i pavlefshëm:", e)
```

Kujdes me vlefshmërinë

- *Pa* komente `// ...` ose `/* ... */`.
- *Pa* presje fundore në objekte/array.
- Çelësat *duhet* të jenë `string` (jo numra, jo `None`).

Kur të përdorësh JSON vs `pickle`

JSON: kur do format të lexueshëm, të portueshëm dhe të sigurt për API/konfigurime/skedarë publikë.
`pickle` : cache lokale ose ruajtje/transferim objektesh specifike për Python (jo për input të pabesueshëm).

	pickle	**json**
Formati	binar, jo i lexueshëm	tekst, i lexueshëm
Portabiliteti	vetëm Python	shumëgjuhësh
Siguria	rrezik me burime të pabesueshme	më i sigurt për input publik
Shpejtësia/Tipet	i shpejtë, ruan objekte komplekse	i kufizuar te tipet bazë

KAPITULLI 3

Programimi OOP

3.1 Programimi procedural dhe i orientuar nga objektet

Përkufizim

Programimi procedural është një mënyrë për të shkruar software. Është një praktikë programimi e përqendruar te procedurat, pra te veprimet që ndodhin në një program. **Programimi i orientuar nga objektet** (*object-oriented programming*) është i përqendruar te *objektet*. Objektet krijohen nga tipe abstrakte të të dhënave që *inkapsulojnë* bashkë si të dhënat, ashtu edhe funksionet.

Në përdorim sot, kryesisht kemi dy metoda programimi: *programimin procedural* dhe *programimin e orientuar nga objektet*. Gjuhët më të hershme të programimit kanë qenë procedurale, që do të thotë se një program përbëhej nga një ose më shumë procedura. Mund ta mendosh një **procedurë** thjesht si një funksion që kryen një detyrë të caktuar, si p.sh. marrja e inputit nga përdoruesi, kryerja e llogaritjeve, leximi ose shkrimi i skedarëve, shfaqja e output-it, e kështu me radhë. Programet që ke shkruar deri tani kanë qenë nga natyra *procedurale*.

Zakonisht, procedurat operojnë mbi elementë të dhënash që janë të ndarë nga vetë procedurat. Në një program procedural, elementët e të dhënave u kalohen shpesh procedurave të ndryshme. Siç mund ta imagjinosh, fokusi i programimit procedural është krijimi i *procedurave* që operojnë mbi të dhënat e programit. Por, ndarja e të dhënave nga kodi që operon mbi to mund të çojë në probleme kur programi rritet dhe bëhet më i madh dhe më kompleks.

Për shembull, supozo se je pjesë e një ekipi programuesish që ka shkruar një program të madh për bazë të dhënash klientësh. Fillimisht, programi është projektuar që emri, adresa dhe numri i telefonit i klientit të referoheshin përmes tre variablave. Detyra jote ka qenë të projektosh disa funksione që marrin këta tre variabla si argumente dhe kryejnë operacione mbi ta. Software-i ka funksionuar me sukses për ca kohë, por ekipi yt tani është ngarkuar ta përditësojë duke shtuar disa veçori të reja. Gjatë procesit të rishikimit, programuesi më i vjetër në ekip të njofton se emri, adresa dhe numri i telefonit i klientit nuk do të ruhen më në variabla të veçantë. Në vend të kësaj, ato do të ruhen në një listë. Kjo do të thotë që ti duhet t'i modifikosh *të gjitha* funksionet që ke projektuar, në mënyrë që ato të pranojnë dhe të punojnë me një listë në vend të tre variablave. Këto modifikime të gjera jo vetëm që përbëjnë shumë punë, por gjithashtu hapin mundësinë që në kodin tënd të shfaqen gabime të reja.

Ndërsa programimi procedural është i përqendruar te krijimi i procedurave (funksioneve), programimi i orientuar nga objektet (**OOP**) është i përqendruar te krijimi i objekteve. **Objekti** është një entitet software-i që përmban si të dhëna, ashtu edhe procedura. Të dhënat e përmbajtura në një objekt njihen si **atributet e të dhënave të objektit** (*data attributes*). Atributet e të dhënave të një objekti janë thjesht variabla që referojnë të dhëna.

Procedurat që kryen një objekt njihen si **metoda**. Metodat e një objekti janë funksione që kryejnë operacione mbi atributet e të dhënave të atij objekti. Konceptualisht, objekti është një njësi *e vetë-mjaftueshme* që përbëhet nga atribute të të dhënave dhe metoda që operojnë mbi këto atribute. Kjo ilustrohet në Figurën 3.1.

OOP e zgjidh problemin e ndarjes së kodit dhe të dhënave përmes dy ideve: **inkapsulimit** dhe **fshehjes së të dhënave**. **Inkapsulimi** i referohet kombinimit të të dhënave dhe kodit në një objekt të vetëm. **Fshehja e të dhënave** i referohet aftësisë së objektit për t'i fshehur atributet e tij të të dhënave nga kodi që ndodhet jashtë objektit. Vetëm metodat e objektit mund të kenë akses të drejtpërdrejtë dhe të bëjnë ndryshime në atributet e të dhënave të objektit. Zakonisht, një objekt i fsheh të dhënat e veta, por lejon që kodi i jashtëm të ketë akses në metodat e tij. Siç tregohet në Figurën 3.2, metodat e objektit u ofrojnë deklaratave të programimit jashtë objektit një akses *të tërthortë* te atributet e të dhënave të objektit.

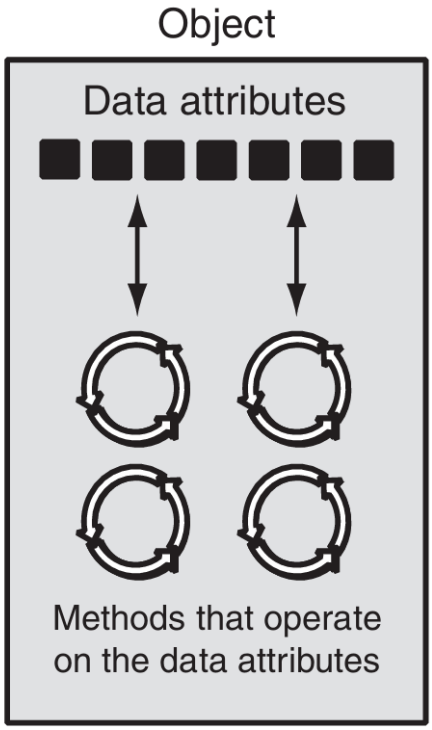

Figura 3.1: Një objekt përmban atribute të të dhënave dhe metoda

fig: oop-object

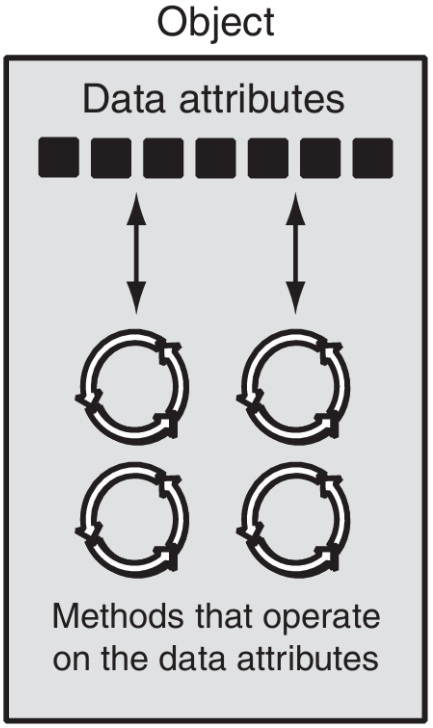

Figura 3.2: Kodi jashtë objektit ndërvepron me metodat e objektit

fig: oop-object-methods

Kur atributet e të dhënave të një objekti janë të fshehura nga kodi i jashtëm, dhe aksesimi i tyre lejohet vetëm përmes metodave të objektit, atëherë atributet e të dhënave mbrohen nga *dëmtimi aksidental*. Për më tepër, kodi jashtë objektit *nuk ka nevojë* të dijë asgjë rreth formatit ose strukturës së brendshme të të dhënave të objektit. Kodi duhet vetëm të ndërveprojë me metodat e objektit. Kur një programues ndryshon strukturën e atributeve të brendshme të të dhënave të një objekti, ai/ajo modifikon gjithashtu metodat e objektit në mënyrë që ato të mund të operojnë siç duhet mbi të dhënat. Megjithatë, mënyra se si kodi i jashtëm ndërvepron me këto metoda *nuk ndryshon*.

Ripërdorimi i objekteve

Përveç zgjidhjes së problemeve të ndarjes së kodit dhe të dhënave, përdorimi i OOP është inkurajuar edhe nga prirja drejt **ripërdorimit të objekteve**. Një objekt *nuk* është një program më vete, por përdoret nga programet që kanë nevojë për shërbimet e tij.

Për shembull, supozo që Sharan është një programiere që ka zhvilluar një grup objektesh për të vizatuar imazhe 3D. Ajo është shumë e fortë në matematikë dhe di shumë rreth grafikës kompjuterike, ndaj objektet e saj janë koduar për të kryer të gjitha operacionet matematikore 3D të nevojshme dhe për të menaxhuar harduerin video të kompjuterit. Tomi, i cili po shkruan një program për një studio arkitekturore, ka nevojë që aplikacioni i tij të shfaqë imazhe 3D të ndërtesave. Për shkak se ai punon me afat të ngushtë dhe nuk ka shumë njohuri rreth grafikës kompjuterike, ai mund t'i përdorë objektet e Sharanit për të kryer vizatimin 3D (natyrisht, kundrejt një pagese të vogël!).

Shembull i përditshëm i një objekti

Imagjino se ora jote e zgjimit në tavolinë është në të vërtetë një objekt software-i. Nëse do të ishte e tillë, ajo do të kishte atributet e mëposhtme të të dhënave:

- `current_second` (një vlerë në intervalin 0–59)
- `current_minute` (një vlerë në intervalin 0–59)
- `current_hour` (një vlerë në intervalin 1–12)

- `alarm_time` (një orë dhe minutë e vlefshme)
- `alarm_is_set` (`True` ose `False`)

Siç mund të shohësh, atributet e të dhënave janë thjesht vlera që përshkruajnë gjendjen aktuale të orës me alarm: çfarë ore është, kur është vendosur alarmi, nëse alarmi është aktiv apo jo, e kështu me radhë. Ti, përdoruesi i objektit "ora me alarm", **nuk mund t'i manipulosh direkt** këto atribute të të dhënave, sepse ato janë *private*. Për të ndryshuar vlerën e një atributi të të dhënave, duhet të përdorësh një nga metodat e objektit. Më poshtë janë disa nga metodat e objektit "ora me alarm":

- `set_time`
- `set_alarm_time`
- `set_alarm_on`
- `set_alarm_off`

Secila metodë manipulon një ose më shumë nga atributet e të dhënave. Për shembull, metoda `set_time` të lejon të vendosësh kohën e orës me alarm. Ti e aktivizon këtë metodë duke shtypur një buton në krye të orës. Duke përdorur një buton tjetër, mund të aktivizosh metodën `set_alarm_time`. Përveç kësaj, një buton tjetër të lejon të ekzekutosh metodat `set_alarm_on` dhe `set_alarm_off`. Vër re që të gjitha këto metoda mund të aktivizohen prej teje, i cili ndodhesh jashtë objektit "ora me alarm". Metodat që mund të aksesohen nga entitete të jashtme (si përdoruesi) quhen **metoda publike** (*public methods*).

Ora me alarm ka gjithashtu edhe metoda **private**, të cilat janë pjesë e funksionimit të brendshëm privat të objektit. Entitetet e jashtme (si ti, përdoruesi i orës me alarm) nuk kanë akses të drejtpërdrejtë te këto metoda private. Objekti është projektuar t'i ekzekutojë automatikisht këto metoda dhe t'i fshehë detajet prej teje. Më poshtë janë metodat private të objektit "ora me alarm":

- `increment_current_second`
- `increment_current_minute`
- `increment_current_hour`
- `sound_alarm`

Çdo sekondë, ekzekutohet metoda `increment_current_second`. Kjo ndryshon vlerën e atributit `current_second`. Nëse `current_second` është vendosur në `59` kur kjo metodë ekzekutohet, ajo është programuar të vendosë `current_second` në `0` dhe më pas të shkaktojë ekzekutimin e metodës `increment_current_minute`. Kjo metodë i shton 1 atributit `current_minute`, përveç rasteve kur ai është `59`. Në atë rast, ajo e vendos `current_minute` në `0` dhe shkakton ekzekutimin e metodës `increment_current_hour`. Metoda `increment_current_minute` krahason kohën e re me `alarm_time`. Nëse të dy kohët përputhen dhe alarmi është i ndezur, ekzekutohet metoda `sound_alarm`.

3.2 Klasat

Përkufizim

Klasë quajmë kodin që specifikon atributet e të dhënave dhe metodat për një lloj të caktuar objekti.

Tani le të diskutojmë si krijohen objektet në software. Përpara se një objekt të mund të krijohet, ai duhet të projektohet nga një programues. Programuesi vendos se cilat atribute të të dhënave dhe cilat metoda duhen, pastaj krijon një **klasë**. Një klasë është kod që specifikon atributet e të dhënave dhe metodat e një lloji të caktuar objekti. Mund ta mendosh një klasë si një **"plan"** (*blueprint*) nga i cili mund të krijohen objekte. Ajo shërben njësoj si plani i një shtëpie: vetë plani *nuk është* shtëpi, por është një përshkrim i detajuar i një shtëpie. Kur përdorim planin për të ndërtuar një shtëpi reale, mund të themi që po ndërtojmë një **instancë** të shtëpisë së përshkruar nga plani. Nëse duam, mund të ndërtojmë disa shtëpi identike nga i njëjti plan. Secila shtëpi është një instancë e veçantë e shtëpisë së përshkruar nga plani. Kjo ide ilustrohet në Figurën 3.3.
Një mënyrë tjetër për të menduar ndryshimin mes një klase dhe një objekti është të mendosh ndryshimin mes një prerëseje biskotash (*cookie cutter*) dhe vetë biskotës. Prerësja e biskotës nuk është vetë biskota, por ajo

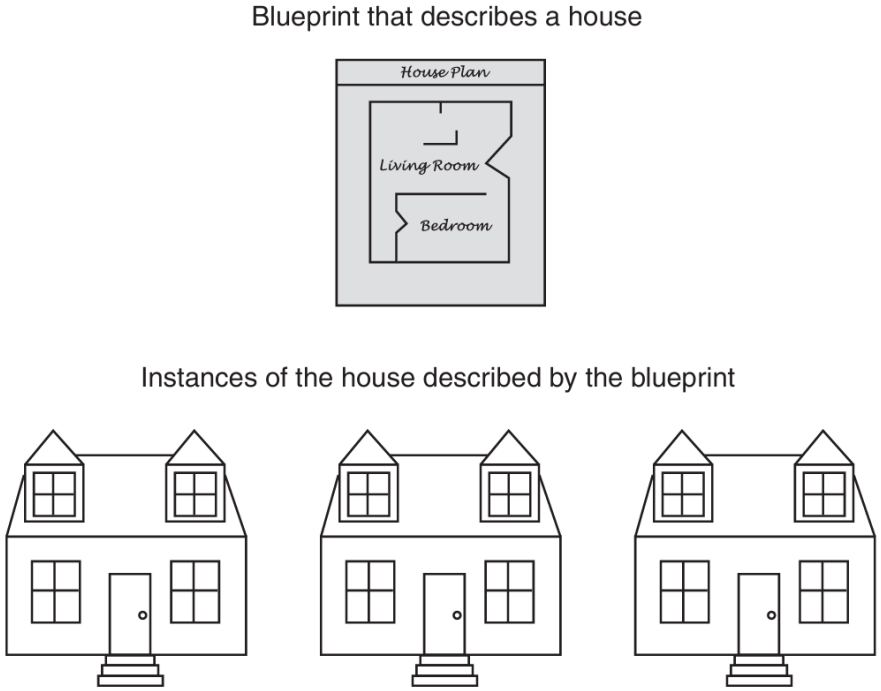

Figura 3.3: Një planimetri dhe shtëpitë e ndërtuara nga planimetria.

fig: blueprint

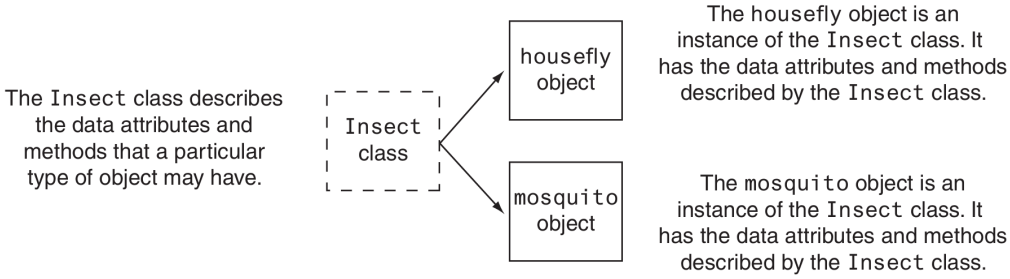

Figura 3.4: Objektet `hoursefly` and `mosquito` janë istanca të klasës `Insect`

fig: insect

"përshkruan" formën e biskotës. Me të njëjtën prerëse mund të bësh një biskotë, ose disa biskota. Mendo klasën si prerësen e biskotës, dhe objektet e krijuara nga klasa si biskotat.

Pra, një klasë është një **përshkrim i karakteristikave të një objekti**. Kur programi është në ekzekutim, ai mund të përdorë klasën për të krijuar në memorie sa më shumë objekte të atij tipi, sa ka nevojë. Çdo objekt që krijohet nga një klasë quhet **instancë e klasës**. Për shembull, Xhesika është një entomologe (dikush që studion insektet), dhe gjithashtu i pëlqen të shkruajë programe. Ajo projekton një program për të kataloguar lloje të ndryshme insektesh. Si pjesë e programit, ajo krijon një klasë të quajtur `Insect`, e cila specifikon karakteristika që janë të përbashkëta për të gjitha llojet e insekteve. Klasa `Insect` është një specifikim nga i cili mund të krijohen objekte. Më pas, ajo shkruan deklarata programimi që krijojnë një objekt të quajtur `housefly`, i cili është një **instancë** e klasës `Insect`. Objekti `housefly` është një entitet që zë vend në memorien e kompjuterit dhe ruan të dhëna për një mizë shtëpie. Ai ka atributet e të dhënave dhe metodat e specifikuara nga klasa `Insect`. Pastaj ajo shkruan deklarata programimi që krijojnë një objekt të quajtur `mosquito`. Edhe objekti `mosquito` është një instancë e klasës `Insect`. Ai ka hapësirën e vet në memorie dhe ruan të dhëna për një mushkonjë. Edhe pse objektet `housefly` dhe `mosquito` janë entitete të veçanta në memorien e kompjuterit, të dy janë krijuar nga klasa `Insect`. Kjo do të thotë që secili prej objekteve ka atributet e të dhënave dhe metodat e përshkruara nga klasa `Insect`. Kjo ilustrohet në Figurën 3.4.

Përkufizimet e klasave

Për të krijuar një klasë, shkruajmë një **përkufizim klase** (*class definition*). Një përkufizim klase është një grup deklaratash që përkufizojnë metodat dhe atributet e të dhënave të klasës.

Programi 1 — Klasa `Coin` (jo program i plotë)

Le të shohim një shembull të thjeshtë. Supozo që po shkruajmë një program që simulon hedhjen e një monedhe. Në program, duhet ta hedhim monedhën shumë herë dhe çdo herë të përcaktojmë nëse ka rënë me "kokë" (*Heads*) apo me "bisht" (*Tails*). Duke ndjekur një qasje të orientuar nga objektet, ne do të shkruajmë një klasë të quajtur `Coin` që mund të kryejë sjelljet e monedhës. Ky program tregon përkufizimin e klasës. Nuk është program i plotë; do ta plotësojmë më poshtë.

```
1  import random
2  class Coin:
3     def __init__(self):
4        self.sideup = 'Heads'
5
6     def toss(self):
7        if random.randint(0, 1) == 0:
8           self.sideup = 'Heads'
9        else:
10          self.sideup = 'Tails'
11
12    def get_sideup(self):
13       return self.sideup
```

Tani le t'i shpjegojmë këto rreshta me radhë.

- Në **rreshtin 1** importojmë modulin `random`. Kjo është e nevojshme sepse përdorim funksionin `randint` për të gjeneruar një numër të rastësishëm.
- **Rreshti 3** është fillimi i përkufizimit të klasës. Ai fillon me fjalën kyçe `class`, e ndjekur nga emri i klasës, `Coin`, dhe më pas nga dy pika `:`.

Të njëjtat rregulla që vlejnë për emrat e variablave vlejnë edhe për emrat e klasave. Megjithatë, vër re që emrin e klasës `Coin` e kemi nisur me shkronjë të madhe. Kjo *nuk* është e detyrueshme, por është një konventë shumë e përhapur mes programuesve. Kjo ndihmon që gjatë leximit të kodit të dallosh më lehtë emrat e klasave nga emrat e variablave. Klasa `Coin` ka tre metoda:

- Metoda `__init__` shfaqet në rreshtat 3–4.
- Metoda `toss` shfaqet në rreshtat 6–10.
- Metoda `get_sideup` shfaqet në rreshtat 12–13.

Përveç faktit që shfaqen brenda një klase, vër re se përkufizimet e këtyre metodave duken si çdo përkufizim tjetër funksioni në Python. Ato fillojnë me një rresht `def` (header), i ndjekur nga një bllok deklaratash të futura (*indented*). Vër re header-in e secilit përkufizim metode (rreshtat 11, 19 dhe 28) dhe do të shohësh se secila metodë ka një parametër të quajtur `self`:

- Rreshti 3: `def __init__(self):`
- Rreshti 6: `def toss(self):`
- Rreshti 12: `def get_sideup(self):`

Parametri `self` është i **detyruar** në çdo metodë të një klase. Nga diskutimi i mëparshëm rreth programimit të orientuar nga objektet, kujto që një metodë operon mbi atributet e të dhënave të një objekti specifik. Kur një metodë ekzekutohet, ajo duhet të ketë një mënyrë për të ditur mbi atributet e cilit objekt duhet të punojë. Këtu hyn në lojë parametri `self`. Kur thirret një metodë, Python bën që parametri `self` të referojë objektin specifik mbi të cilin metoda duhet të operojë.

Metoda `__init__`

Le të shohim secilën metodë. Metoda e parë, e quajtur `__init__`:

```
def __init__(self):
   self.sideup = 'Heads'
```

Shumica e klasave në Python kanë një metodë të veçantë të quajtur `__init__`, e cila ekzekutohet automatikisht kur një instancë e klasës krijohet në memorie. Metoda `__init__` njihet zakonisht si **metoda inicializuese** (*initializer*), sepse ajo inicializon atributet e të dhënave të objektit. (Emri i metodës fillon me dy karaktere _, pastaj fjala `init`, dhe mbaron me dy karaktere të tjerë _.) Menjëherë pasi një objekt është krijuar në memorie, metoda `__init__` ekzekutohet dhe parametri `self` caktohet automatikisht të referojë objektin që sapo u krijua. Brenda metodës, ekzekutohet deklarata në rreshtin 12:

```
self.sideup = 'Heads'
```

Kjo deklaratë i cakton stringun `'Heads'` atributit të të dhënave `sideup` që i përket objektit që sapo u krijua. Si rezultat i kësaj metode `__init__`, çdo objekt që krijojmë nga klasa `Coin` fillimisht do të ketë atributin `sideup` të vendosur në `'Heads'`.

Shënim

Zakonisht, `__init__` është metoda e parë që vendoset brenda përkufizimit të klasës.

Metoda `toss`

```
def toss(self):
   if random.randint(0, 1) == 0:
      self.sideup = 'Heads'
   else:
      self.sideup = 'Tails'
```

Edhe kjo metodë ka parametrin e detyrueshëm `self`. Kur thirret metoda `toss`, `self` do të referojë automatikisht objektin mbi të cilin metoda duhet të operojë. Metoda `toss` simulon hedhjen e monedhës. Kur thirret metoda, deklarata `if` në rreshtin 7 thërret funksionin `random.randint` për të marrë një integer të rastësishëm në intervalin 0 deri 1. Nëse numri është 0, deklarata në rreshtin 8 i cakton `'Heads'` atributit `self.sideup`. Përndryshe, deklarata në rreshtin 10 i cakton `'Tails'` atributit `self.sideup`.

Rreth parametrin `self`

Parametri duhet të ekzistojë në çdo metodë. Nuk je i detyruar ta quash `self`, por përdorimi i këtij emri është **fort i rekomanduar** që të ndjekësh praktikën standarde dhe kodi të jetë i lexueshëm për programues të tjerë.

Metoda `get_sideup`

```
def get_sideup(self):
   return self.sideup
```

Edhe një herë, metoda ka parametrin e detyrueshëm `self`. Kjo metodë thjesht kthen vlerën e `self.sideup`. Ne e thërrasim këtë metodë sa herë duam të dimë se cila anë e monedhës është aktualisht lart.

Përdorimi i klasës `Coin` në një program

Për të demonstruar klasën `Coin`, kemi nevojë të shkruajmë një program të plotë që e përdor atë për të krijuar një objekt. Programi 2 tregon një shembull. Përkufizimi i klasës `Coin` shfaqet në rreshtat 2–13. Programi ka një funksion `main`, që shfaqet në rreshtat 15–22.

Programi 2 — `coin_demo1.py`

```
1  import random
2  class Coin:
3     def __init__(self):
4        self.sideup = 'Heads'
5
6     def toss(self):
7        if random.randint(0, 1) == 0:
8           self.sideup = 'Heads'
9        else:
10          self.sideup = 'Tails'
11
12    def get_sideup(self):
13       return self.sideup
14
15 def main():
16    my_coin = Coin()
17
18    print('This side is up:', my_coin.get_sideup())
19    print('I am tossing the coin ...')
20    my_coin.toss()
21
22    print('This side is up:', my_coin.get_sideup())
23
24 if __name__ == '__main__':
25    main()
```

Shih më nga afër deklaratën në rreshtin 16 (në funksionin `main`):

```
my_coin = Coin()
```

Shprehja `Coin()` në anën e djathtë të operatorit `=` shkakton dy gjëra:

1. Krijohet në memorie një objekt nga klasa `Coin`.
2. Thirret metoda `__init__` e klasës `Coin`, dhe parametri `self` caktohet automatikisht të referojë objektin që sapo u krijua. Si rezultat, atributit `sideup` të atij objekti i caktohet stringu `'Heads'`.

Figura 3.5 ilustron këto hapa. Pas këtyre hapave, ekziston një objekt i klasës `Coin` me atributin `sideup` të vendosur në `'Heads'`. Pas kësaj, operatori `=` i cakton variablës `my_coin` objektin `Coin` që u krijua. Variabli `my_coin` do të referojë një objekt `Coin`, dhe atributi `sideup` i atij objekti do të ketë vlerën `'Heads'`. Deklarata tjetër që ekzekutohet është:

```
print('This side is up:', my_coin.get_sideup())
```

Kjo deklaratë printon një mesazh që tregon se cila anë e monedhës është lart. Vër re shprehjen:

```
my_coin.get_sideup()
```

Kjo shprehje përdor objektin e referuar nga `my_coin` për të thirrur metodën `get_sideup`. Kur ekzekutohet metoda, parametri `self` do të referojë objektin `my_coin`. Si rezultat, metoda kthen stringun `'Heads'`. Vër re që nuk na duhej të kalonim ndonjë argument te metoda `get_sideup`, megjithëse ajo ka parametrin `self`. Kur

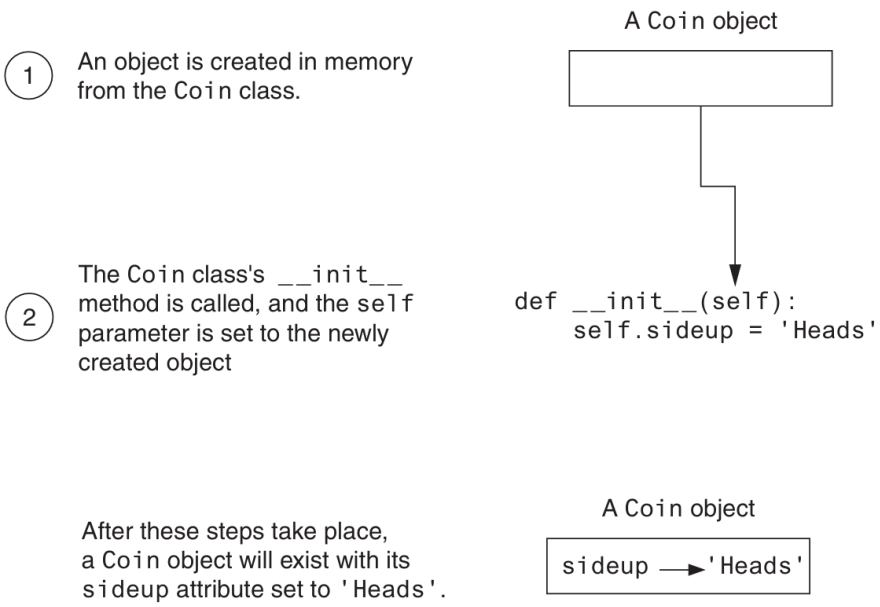

Figura 3.5: Veprimet që kryhen nga deklarata `Coint()`.

fig: obj

thirret një metodë, Python kalon automatikisht një referencë drejt objektit që po e thërret, në parametrin e parë të metodës. Si rrjedhojë, parametri `self` automatikisht do të referojë objektin mbi të cilin metoda duhet të operojë. Rreshtat 19 dhe 20 janë deklaratat e radhës që ekzekutohen:

```
1 print('I am tossing the coin ...')
2 my_coin.toss()
```

Deklarata në rreshtin 20 përdor objektin e referuar nga `my_coin` për të thirrur metodën `toss`. Kur ekzekutohet metoda, parametri `self` do të referojë objektin `my_coin`. Metoda do të gjenerojë një numër të rastësishëm dhe pastaj do ta përdorë atë numër për të ndryshuar vlerën e atributit `sideup` të objektit. Rreshti 22 ekzekutohet më pas. Kjo deklaratë thërret `my_coin.get_sideup()` për të shfaqur përsëri se cila anë e monedhës është lart.

Shembuj të mundshëm output-i nga ky program (për shkak të numrave të rastësishëm, rezultatet mund të ndryshojnë):

```
1 This side is up: Heads
2 I am tossing the coin ...
3 This side is up: Tails
```

```
1 This side is up: Heads
2 I am tossing the coin ...
3 This side is up: Heads
```

```
1 This side is up: Heads
2 I am tossing the coin ...
3 This side is up: Tails
```

Fshehja e atributeve

Më herët në këtë kapitull përmendëm që atributet e të dhënave të një objekti duhet të jenë **private**, në mënyrë që vetëm metodat e objektit të kenë akses të drejtpërdrejtë te ato. Kjo e mbron gjendjen (të dhënat) e objektit nga prishjet aksidentale. Megjithatë, te klasa `Coin` që pamë më sipër, atributi `sideup` *nuk* është privat. Ai mund të aksesohet dhe të ndryshohet direkt nga deklarata që ndodhen jashtë metodave të klasës `Coin`. Programi 3 tregon një shembull, ku ne "mashtrojmë" duke ndryshuar direkt atributin `sideup`.

Programi 3 — `coin_demo2.py`

Në këtë program përdorim të njëjtën klasë `Coin` si më parë, por në funksionin `main` bëjmë një ndryshim: pasi e hedhim monedhën me metodën `toss`, ne e detyrojmë atributin `sideup` të jetë `'Heads'`.

```
import random
class Coin:
   def __init__(self):
      self.sideup = 'Heads'

   def toss(self):
      if random.randint(0, 1) == 0:
         self.sideup = 'Heads'
      else:
         self.sideup = 'Tails'

   def get_sideup(self):
      return self.sideup

def main():
   my_coin = Coin()

   print('This side is up:', my_coin.get_sideup())
   print('I am tossing the coin ...')
   my_coin.toss()

   my_coin.sideup = 'Heads'

   print('This side is up:', my_coin.get_sideup())

if __name__ == '__main__':
   main()
```

Kur e ekzekuton këtë program, output-i mund të jetë i tillë:

```
This side is up: Heads
I am tossing the coin ...
This side is up: Heads
```

Në rreshtin:

```
my_coin.sideup = 'Heads'
```

ne po i caktojmë drejtpërdrejt stringun `'Heads'` atributit `sideup` të objektit `my_coin`. Pavarësisht se çfarë vlere ka caktuar metoda `toss`, kjo deklaratë e mbishkruan rezultatin. Si pasojë, monedha "përfundon" gjithmonë me kokë lart.

Nëse duam të simulojmë me të vërtetë hedhjen e një monedhe, nuk duam që kodi jashtë klasës të jetë në gjendje të ndryshojë rezultatin e metodës `toss`. Për ta parandaluar këtë, duhet ta bëjmë atributin `sideup` **privat**. Në Python, mund të "fshehësh" një atribut duke filluar emrin e tij me dy viza të poshtme `__`. Nëse e ndryshojmë emrin e atributit `sideup` në `__sideup`, atëherë kodi jashtë klasës `Coin` nuk do të mund ta aksesojë më drejtpërdrejt atë atribut. Programi 4 tregon një version të ri të klasës `Coin` ku atributi është bërë privat.

Programi 4 — `coin_demo3.py`

Tani atributi i të dhënave quhet `__sideup`. Vetëm metodat e klasës `Coin` mund ta ndryshojnë atë; kodi jashtë klasës përdor vetëm metodën `get_sideup` për të lexuar vlerën.

```
import random
class Coin:
   def __init__(self):
      self.__sideup = 'Heads'

   def toss(self):
      if random.randint(0, 1) == 0:
         self.__sideup = 'Heads'
      else:
         self.__sideup = 'Tails'

   def get_sideup(self):
      return self.__sideup


def main():
   my_coin = Coin()

   print('This side is up:', my_coin.get_sideup())
   print('I am going to toss the coin ten times:')
   for count in range(5):
      my_coin.toss()
      print(my_coin.get_sideup())

if __name__ == '__main__':
   main()
```

Output-i i mundshëm:

```
This side is up: Heads
I am going to toss the coin ten times:
Tails
Heads
Heads
Tails
Tails
```

Vër re që tani nuk mund të shkruajmë më diçka të tipit `my_coin.__sideup = 'Heads'` nga jashtë klasës `Coin`. Ne mund të lexojmë vlerën e atributit vetëm përmes metodës publike `get_sideup`, dhe vlera mund të ndryshohet vetëm nga metoda `toss`. Kjo është pikërisht ideja e **fshehjes së të dhënave** dhe e mbylljes së tyre brenda objektit.

Ruajtja e klasave në module

Derimë tani, përkufizimi i klasës `Coin` dhe deklaratat që e përdorin atë kanë qenë në të njëjtin skedar. Kjo mënyrë funksionon mirë për programe të vegjël që përdorin vetëm një ose dy klasa. Por sa më shumë klasa të përdorë një program, aq më shumë lind nevoja për t'i organizuar ato më mirë. Programuesit zakonisht i organizojnë klasat duke i ruajtur në **module**. Pastaj këto module mund të importohen në çdo program që ka nevojë të përdorë klasat që ato përmbajnë.

Programi 5 — `coin.py` & Programi 6 — `coin_demo4.py`

Për shembull, supozo që vendosim ta ruajmë klasën `Coin` në një modul të quajtur `coin`. Skedari `coin.py` mund të ketë përmbajtjen e mëposhtme.

```python
import random
class Coin:
   def __init__(self):
      self.__sideup = 'Heads'

   def toss(self):
      if random.randint(0, 1) == 0:
         self.__sideup = 'Heads'
      else:
         self.__sideup = 'Tails'

   def get_sideup(self):
      return self.__sideup
```

Tani, sa herë që kemi nevojë për klasën `Coin` në një program tjetër, mjafton të importojmë modulin `coin`. Programi 6 tregon një shembull.

```python
import coin
def main():
   my_coin = coin.Coin()
   print('This side is up:', my_coin.get_sideup())

   print('I am going to toss the coin ten times:')
   for count in range(10):
      my_coin.toss()
      print(my_coin.get_sideup())

if __name__ == '__main__':
   main()
```

Në rreshtin:

```python
my_coin = coin.Coin()
```

ne po përdorim një **emër të kualifikuar** (*qualified name*). Emri i klasës `Coin` paraprihet nga emri i modulit `coin`, i ndarë me një pikë. Kjo i tregon Python-it që klasa `Coin` ndodhet brenda modulit `coin`. Kjo mënyrë organizimi e bën shumë më të lehtë ripërdorimin e klasave në programe të ndryshme.

Klasa `BankAccount`

Le të shohim një shembull tjetër.

Programi 7 — `bankaccount.py`

Klasa `BankAccount`, e ruajtur në një modul të quajtur `bankaccount`, simulon një llogari bankare. Objektet e krijuara nga kjo klasë përfaqësojnë llogari bankare: mund të kenë një bilanc fillestar, mund të pranojnë depozita, të lejojnë tërheqje dhe të kthejnë bilancin aktual. Vër re që metoda `__init__` ka dy parametra: `self` dhe `bal`. Parametri `bal` do të marrë nga jashtë vlerën e bilancit fillestar të llogarisë. Në trupin e metodës, shuma `bal` i caktohet atributit privat `__balance` të objektit.

```
1  class BankAccount:
2     def __init__(self, bal):
3        self.__balance = bal
4
5     def deposit(self, amount):
6        self.__balance += amount
7
8     def withdraw(self, amount):
9        if self.__balance >= amount:
10          self.__balance -= amount
11       else:
12          print('Error: Insufficient funds')
13
14    def get_balance(self):
15       return self.__balance
```

Metoda `deposit` ndodhet në rreshtat 5–6. Ajo ka dy parametra: `self` dhe `amount`. Kur thirret metoda, shuma që duhet depozituar në llogari kalon si argument te parametri `amount`. Vlera e këtij parametri i shtohet atributit `__balance`.

Metoda `withdraw` ndodhet në rreshtat 8–12. Edhe kjo metodë ka dy parametra: `self` dhe `amount`. Kur thirret metoda, shuma që duhet tërhequr nga llogaria kalon si argument te `amount`. Struktura `if` fillon në rreshtin 9 dhe kontrollon nëse ka mjaft para në llogari për të bërë tërheqjen. Nëse po, atëherë `amount` zbritet nga `__balance` (rreshti 10). Përndryshe, shfaqet mesazhi `'Error: Insufficient funds'`.

Metoda `get_balance` ndodhet në rreshtat 14–15. Kjo metodë thjesht kthen vlerën e atributit `__balance`.

Programi 8 — `account_test.py`

Programi i mëposhtëm tregon si përdoret klasa `BankAccount`.

```
import bankaccount

def main():
   start_bal = float(input('Enter your starting balance: '))
   savings = bankaccount.BankAccount(start_bal)

   pay = float(input('How much were you paid this week? '))
   print('I will deposit that into your account.')
   savings.deposit(pay)

   print(f'Your account balance is ${savings.get_balance():,.2f}.')

   cash = float(input('How much would you like to withdraw? '))
   print('I will withdraw that from your account.')
   savings.withdraw(cash)

   print(f'Your account balance is ${savings.get_balance():,.2f}.')

if __name__ == '__main__':
   main()
```

Shembull ekzekutimi (input-i i përdoruesit është me të **trashë**):

```
Enter your starting balance: \textbf{1000.00}
How much were you paid this week? \textbf{500.00}
I will deposit that into your account.
Your account balance is $1,500.00.
How much would you like to withdraw? \textbf{1200.00}
I will withdraw that from your account.
Your account balance is $300.00.
```

Një shembull tjetër, kur përdoruesi kërkon të tërheqë më shumë para sesa ka në llogari:

```
Enter your starting balance: \textbf{1000.00}
How much were you paid this week? \textbf{500.00}
I will deposit that into your account.
Your account balance is $1,500.00.
How much would you like to withdraw? \textbf{2000.00}
I will withdraw that from your account.
Error: Insufficient funds
Your account balance is $1,500.00.
```

Në rreshtin 4 merret nga përdoruesi bilanci fillestar i llogarisë dhe i caktohet variablës `start_bal`. Rreshti 4 krijon një instancë të klasës `BankAccount` dhe ia cakton variablës `savings`:

```
savings = bankaccount.BankAccount(start_bal)
```

Vër re që variabli `start_bal` kalon brenda kllapave. Kjo bën që vlera e `start_bal` të kalohet si argument te metoda `__init__`. Brenda asaj metode, vlera kalon te parametri `bal` dhe ruhet te atributi `__balance`. Rreshti 7 merr shumën e rrogës së përdoruesit dhe ia cakton variablës `pay`. Në rreshtin 9 thirret metoda `deposit`:

```
savings.deposit(pay)
```

Kjo thirrje kalon `pay` si argument te parametri `amount` në metodën `deposit`, e cila e shton atë shumë te `__balance`. Deklarata në rreshtin 11 shfaq bilancin e llogarisë. Vër re që përdoret një **f-string** për të thirrur metodën `savings.get_balance()` dhe për ta formatuar vlerën si shumë parash:

```
print(f'Your account balance is \${savings.get_balance():,.2f}.')
```

Rreshti 13 merr shumën që përdoruesi dëshiron të tërheqë dhe ia cakton variablës `cash`. Në rreshtin 15 thirret metoda `withdraw`:

```
savings.withdraw(cash)
```

Kjo thirrje kalon `cash` si argument te parametri `amount` në metodën `withdraw`. Nëse ka mjaft para në llogari, shuma zbritet; përndryshe, shfaqet mesazhi `'Error: Insufficient funds'`. Deklarata në rreshtin 26 shfaq bilancin përfundimtar të llogarisë.

Metoda `__str__`

Shpesh na duhet të shfaqim një mesazh që tregon **gjendjen** e një objekti. Gjendja e një objekti është thjesht grupi i vlerave të atributeve të tij në një moment të caktuar. Për shembull, te klasa `BankAccount` kemi një atribut të vetëm të të dhënave: `__balance`. Në çdo moment, ky atribut mban një vlerë, e cila përfaqëson gjendjen aktuale të objektit.

Shembull i thjeshtë kodi që shfaq gjendjen e një objekti `BankAccount`:

```
account = bankaccount.BankAccount(1500.0)
print(f'The balance is \${account.get_balance():,.2f}')
```

Deklarata e parë krijon një objekt `BankAccount`, duke i kaluar vlerën `1500.0` metodës `__init__`. Pas kësaj, variabli `account` referon objektin. Rreshti i dytë shfaq një string të formatuar që tregon vlerën e atributit `__balance`. Output-i i mundshëm:

```
The balance is \$1,500.00
```

Programi 9 — `bankaccount2.py` & Programi 10 — `account_test2.py`

Shfaqja e gjendjes së një objekti është aq e zakonshme, sa shumë programues pajisin klasat e tyre me një metodë që kthen një string që përshkruan gjendjen e objektit. Në Python, kësaj metode i japim emrin e veçantë `__str__`. Programi 9 tregon klasën `BankAccount` me një metodë `__str__` të shtuar.

```
class BankAccount:
   def __init__(self, bal):
      self.__balance = bal

   def deposit(self, amount):
      self.__balance += amount

   def withdraw(self, amount):
      if self.__balance >= amount:
         self.__balance -= amount
      else:
         print('Error: Insufficient funds')

   def get_balance(self):
      return self.__balance

   def __str__(self):
      return f'The account balance is \${self.__balance:,.2f}'
```

Metodën `__str__` **nuk** e thërrasim drejtpërdrejt. Ajo thirret automatikisht kur kalojmë një objekt si argument te funksioni `print`, ose kur përdorim funksionin e brendshëm `str`. Programi 10 tregon një shembull.

```
import bankaccount2
def main():
   start_bal = float(input('Enter your starting balance: '))
   savings = bankaccount2.BankAccount(start_bal)

   pay = float(input('How much were you paid this week? '))
   print('I will deposit that into your account.')
   savings.deposit(pay)
   print(savings)

   cash = float(input('How much would you like to withdraw? '))
   print('I will withdraw that from your account.')
   savings.withdraw(cash)
   print(savings)

if __name__ == '__main__':
   main()
```

Shembull ekzekutimi (input-i me të **trashë**):

```
Enter your starting balance: \textbf{1000.00}
How much were you paid this week? \textbf{500.00}
I will deposit that into your account.
The account balance is $1,500.00
How much would you like to withdraw? \textbf{1200.00}
I will withdraw that from your account.
The account balance is $300.00
```

Në rreshtat 9 dhe 14 ne po i kalojmë direkt objektin `savings` funksionit `print`:

```
print(savings)
```

Kjo bën që Python-i të thërrasë automatikisht metodën `__str__` të klasës `BankAccount`. Stringu që kthehet nga `__str__` shfaqet më pas në ekran. Metoda `__str__` thirret gjithashtu automatikisht kur një objekt i kalon si argument funksionit të brendshëm `str`. Shembull:

```
account = bankaccount2.BankAccount(1500.0)
message = str(account)
print(message)
```

Në rreshtin e dytë, objekti `account` i kalon si argument funksionit `str`. Kjo shkakton thirrjen e metodës `__str__` të klasës `BankAccount`. Stringu i kthyer caktohet te variabla `message`, dhe rreshti i tretë e shfaq atë me `print`.

3.3 Puna me instancat

KONCEPT

Çdo **instancë** e një klase ka setin e vet të atributeve të të dhënave. Kur një metodë përdor parametrin `self` për të krijuar një atribut (p.sh. `self.__sideup`), ky atribut i përket atij objekti specifik që `self` referon. Këto quhen **atribute instance** sepse i përkasin një instance të veçantë të klasës.

Në një program mund të krijojmë shumë instanca të së *njëjtës* klasë. Çdo instancë do të ketë atribute të vetat, të pavarura nga instancat e tjera. Për shembull, Programi 11 krijon tre objekte të klasës `Coin`. Çdo objekt ka atributin e vet `__sideup`.

Programi 11 — `coin_demo5.py`

```
import coin
def main():
   coin1 = coin.Coin()
   coin2 = coin.Coin()

   print('I have three coins with these sides up:')
   print(coin1.get_sideup())
   print(coin2.get_sideup())
   print()

   print('I am tossing all three coins ...')
   print()
   coin1.toss()
   coin2.toss()

   print('Now here are the sides that are up:')
   print(coin1.get_sideup())
   print(coin2.get_sideup())
   print()

if __name__ == '__main__':
   main()
```

Shembull ekzekutimi:

```
I have three coins with these sides up:
Heads
Heads

I am tossing all three coins ...

Now here are the sides that are up:
Tails
Heads
```

Në rreshtat 3-4 krijohen tre objekte, secili instancë e klasës `Coin` :

```
coin1 = coin.Coin()
coin2 = coin.Coin()
coin3 = coin.Coin()
```

Secili prej këtyre objekteve ka atributin e vet `__sideup` (në klasën e modifikuar ku e bëmë privat). Edhe pse të tre fillojnë me të njëjtën vlerë (`'Heads'`), ato janë *atribute të ndryshme* në memorien e kompjuterit. Rreshtat 7-8 thërrasin metodën `get_sideup` për secilin objekt dhe shfaqin vlerën e tij. Më pas, rreshtat 13-14 thërrasin metodën `toss` për secilin objekt:

```
coin1.toss()
coin2.toss()
coin3.toss()
```

Pas këtyre thirrjeve, atributi `__sideup` i secilit objekt mund të ketë ndryshuar në mënyrë të pavarur. Në shembullin e output-it, dy monedha përfundojnë me `'Tails'` dhe një me `'Heads'` . Kjo tregon qartë idenë: **çdo instancë ka gjendjen (të dhënat) e vet**.

Metodat aksesuese dhe modifikuese

Siç e kemi theksuar, është praktikë e mirë që atributet e të dhënave të një klase t'i mbajmë **private** dhe të ofrojmë metoda publike për t'i lexuar dhe për t'i ndryshuar ato. Kështu, objekti vetë "kontrollon" çdo ndryshim që bëhet mbi gjendjen e tij.

- Një metodë që **kthen** vlerën e një atributi, por *nuk* e ndryshon atë, quhet **metodë aksesuese** (*accessor method*). Ajo i lejon kodit jashtë klasës të lexojë vlerën e atributit pa e ekspozuar atributin drejtpërdrejt.
- Një metodë që **ruan** një vlerë në një atribut, ose e ndryshon atë në ndonjë mënyrë tjetër, quhet **metodë modifikuese** (*mutator method*). Këto metoda vendosin *si* dhe *kur* mund të ndryshojë një atribut.

Për shembull, imagjino një klasë `CellPhone` me atributet private `__manufact`, `__model` dhe `__retail_price`. Ajo mund të ketë metodat:

- `get_manufact`, `get_model`, `get_retail_price` — **metoda aksesuese** (lexojnë të dhënat, nuk i ndryshojnë).
- `set_manufact`, `set_model`, `set_retail_price` — **metoda modifikuese** (ndryshojnë vlerat e atributeve).

Kur një pjesë e kodit jashtë klasës do të ndryshojë, p.sh., çmimin e telefonit, nuk ndërhyn drejtpërdrejt te `__retail_price`, por thërret:

```
phone.set_retail_price(299.99)
```

Brenda metodës `set_retail_price`, mund të kontrollojmë nëse vlera është e vlefshme (p.sh. nuk është negative) përpara se ta caktojmë. Kështu, **logjika e kontrollit** qëndron në një vend të vetëm, brenda klasës.

Terminologji

Metodat modifikuese quhen shpesh edhe **"setters"**, ndërsa metodat aksesuese quhen **"getters"**. Prandaj, format `get_...` dhe `set_...` janë shumë të zakonshme në kodin objektor.

Kalimi i objekteve si argumente

Kur shkruajmë programe që punojnë me objekte, shumë shpesh na duhet të shkruajmë **funksione** ose **metoda** që marrin objekte si argumente. **Shembull:** një funksion i thjeshtë që pranon një objekt `Coin` dhe shfaq gjendjen e tij:

```
def show_coin_status(coin_obj):
    print('This side of the coin is up:', coin_obj.get_sideup())
```

Më pas, mund të krijojmë një objekt `Coin` dhe ta kalojmë te funksioni:

```
my_coin = coin.Coin()
show_coin_status(my_coin)
```

Kur kalon një *objekt* si argument, ajo që kalon te parametri është **referenca** drejt objektit në memorie, jo një kopje e të gjithë objektit. Kjo do të thotë që funksioni (ose metoda) që e merr argumentin ka akses te objekti *real* dhe mund të thërrasë metodat e tij, madje edhe të ndryshojë gjendjen e tij.

Shiko funksionin e mëposhtëm `flip`, i cili pranon një objekt `Coin` dhe thërret metodën `toss`:

```
def flip(coin_obj):
    coin_obj.toss()
```

Programi 12 — `coin_argument.py`

Programi 12 tregon një shembull të plotë.

```
import coin
def main():
   my_coin = coin.Coin()
   print(my_coin.get_sideup())
   flip(my_coin)
   print(my_coin.get_sideup())

def flip(coin_obj):
   coin_obj.toss()
if __name__ == '__main__':
   main()
```

Shembuj të mundshëm output-i:

```
Heads
Tails
```

```
Heads
Heads
```

```
Heads
Tails
```

Rreshti 3 krijon një objekt `Coin` të referuar nga variabla `my_coin`. Për shkak se metoda `__init__` e klasës `Coin` e vendos atributin `__sideup` fillimisht në `'Heads'`, thirrja e parë:

```
print(my_coin.get_sideup())
```

do të shfaqë `Heads`. Në rreshtin 5 thirret funksioni `flip`, duke ia kaluar `my_coin` si argument:

```
flip(my_coin)
```

Brenda funksionit `flip`, parametri `coin_obj` referon *të njëjtin* objekt që referon `my_coin`. Thirrja:

```
coin_obj.toss()
```

hedh monedhën dhe mund të ndryshojë vlerën e atributit `__sideup`. Për këtë arsye, thirrja e dytë:

```
print(my_coin.get_sideup())
```

mund të shfaqë si `Heads`, ashtu edhe `Tails`, në varësi të rezultatit të hedhjes së monedhës.

Pra, kur kalojmë objekte si argumente, duhet të kemi parasysh që **funksioni mund të ndryshojë gjendjen e objektit**, sepse punon me të njëjtin objekt në memorie dhe jo me një kopje të tij.

3.4 Teknika për projektimin e klasave

Unified Modeling Language (UML)

Kur projekton një klasë, shpesh ndihmon shumë ta *vizatosh* më parë. Për këtë përdorim **UML** (*Unified Modeling Language*) — një gjuhë standarde për të paraqitur grafikisht sisteme të orientuara nga objektet. Diagrami UML për një klasë zakonisht paraqitet si një **kuti e ndarë në tre pjesë** (shih Figurën 3.6):

- pjesa e sipërme — **emri i klasës**;
- pjesa në mes — **atributet e të dhënave** të klasës;
- pjesa e poshtme — **metodat** e klasës.

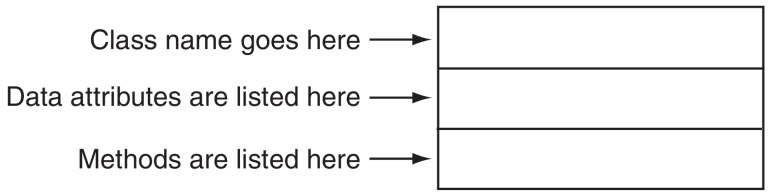

Figura 3.6: Struktura e përgjithshme e një diagrami UML për një klasë.

fig: uml-class-general

Në këtë stil, Figura 3.7 mund të paraqesë klasën `Coin`, ndërsa Figura 3.8 klasën `CellPhone` (që e kemi përmendur te metodat *get/set*). Në UML nuk tregojmë parametrin `self` te metodat, sepse kuptohet që çdo metodë instance e ka atë.

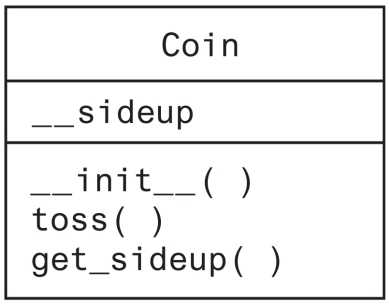

Figura 3.7: Diagram UML për klasën `Coin`.

fig: uml-coin-uml

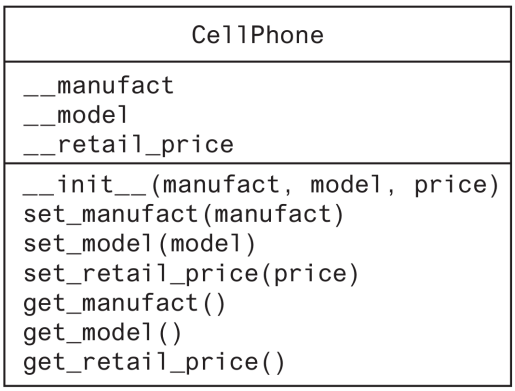

Figura 3.8: Diagram UML për klasën `CellPhone`.

fig: uml-cellphone uml

Këto diagrame janë shumë të dobishme për të pasur një *pamje të qartë* të asaj që duhet të dijë dhe të bëjë çdo klasë, përpara se të fillojmë të shkruajmë kodin në Python.

Gjetja e klasave në një problem

Kur zhvillon një aplikacion të orientuar nga objektet, një nga hapat e parë është të identifikosh **cilat klasa** të duhen. Zakonisht, qëllimi është:

- të gjejmë **llojet e objekteve reale** që shfaqen në problem;
- të krijojmë klasa që përfaqësojnë këto lloje brenda programit.

Ka shumë teknika që përdoren në praktikë. Një teknikë e thjeshtë dhe mjaft e përdorur bazohet te këta hapa:

1. shkruaj një përshkrim me fjalë të **domenit të problemit**;
2. identifiko **të gjithë emrat** (nouns, përfshirë fraza emërore) në atë përshkrim — çdo emër është një kandidat për klasë;

3. pastro listën duke lënë vetëm ato klasa që janë me të vërtetë **të nevojshme** për problemin.

Le t'i shohim këta hapa më nga afër, me një shembull konkret.

Domeni i problemit

Domeni i problemit (*problem domain*) është grupi i objekteve reale, palëve të përfshira dhe ngjarjeve kryesore që lidhen me problemin tonë.

Shembull: supozo që po shkruajmë një program për menaxherin e *"Joe's Automotive Shop"*, i cili do ta përdorë për të printuar *preventiva shërbimi* për klientët. Një ekspert (p.sh. vetë Joe) mund të shkruajë përshkrimin e mëposhtëm të domenit:

> Joe's Automotive Shop servison makina të huaja dhe është i specializuar për makina Mercedes, Porsche dhe BMW. Kur një klient sjell makinën në servis, menaxheri merr emrin, adresën dhe numrin e telefonit të klientit. Pastaj menaxheri përcakton markën, modelin dhe vitin e prodhimit të makinës dhe i jep klientit një preventiv shërbimi (*service quote*). Preventivi tregon koston e parashikuar të pjesëve, koston e parashikuar të punës, taksën e shitjes dhe totalin e parashikuar të çmimit.

Në përgjithësi, përshkrimi i domenit duhet të përmendë:

- **objekte fizike**, p.sh. automjete, makina, produkte;
- **role personash**, p.sh. menaxher, punonjës, klient, mësues, student;
- **rezultatet e ngjarjeve të biznesit**, p.sh. porosi, fatura, preventiv shërbimi;
- **objekte regjistrimi**, p.sh. historikë klientësh, listë pagash, etj.

Identifikimi i emrave

Hapi tjetër: gjejmë të gjithë **emrat dhe frazat emërore** në përshkrim. (Nëse ka përemra, mund t'i fusim edhe ata.) Nga përshkrimi i mëparshëm, mund të nxjerrim këtë listë emrash (pa duplikime):

- `address`
- `BMW`
- `car`
- `cars`
- `customer`
- `estimated labor charges`
- `estimated parts charges`
- `foreign cars`
- `Joe's Automotive Shop`
- `make`
- `manager`
- `Mercedes`
- `model`
- `name`
- `Porsche`
- `sales tax`
- `service quote`

- `shop`
- `telephone number`
- `total estimated charges`
- `year`

Të gjithë këta *mund* të jenë klasa — për momentin janë vetëm **kandidatë**.

Pastrimi i listës së emrave

Tani fillojmë dhe e **pastrojmë** listën, për të lënë vetëm klasat që na duhen për zgjidhjen.

1. Disa emra në fakt nënkuptojnë të njëjtën gjë.
Në shembullin tonë:

- `car`, `cars` dhe `foreign cars` i referohen të gjitha konceptit *makinë*;
- `Joe's Automotive Shop` dhe `shop` i referohen të dy kompanisë së Joe.

Për thjeshtësi, mbajmë vetëm `car` dhe `shop`, dhe heqim `cars`, `foreign cars` dhe `Joe's Automotive Shop` nga lista.

2. Disa emra përfaqësojnë gjëra me të cilat nuk kemi pse merremi.
Qëllimi i aplikacionit tonë është *të printojë preventiva shërbimi*. Nuk kemi nevojë për informacione globale për dyqanin, dhe as për menaxherin.

- heqim `shop` nga lista;
- heqim `manager` nga lista.

3. Disa emra përfaqësojnë objekte konkrete, jo klasa.
Emrat `Mercedes`, `Porsche` dhe `BMW` nuk janë klasa më vete — ato janë **shembuj të veçantë** të klasës `Car`. Pra:

- i heqim `Mercedes`, `Porsche`, `BMW` nga lista e klasave të mundshme.

4. Disa emra përfaqësojnë vetëm vlera të thjeshta.
Një klasë ka **atribute** dhe **metoda**. Nëse një emër përfaqëson diçka që nuk ka ndonjë grup vlerash të lidhura dhe as veprime të dukshme, zakonisht është thjesht një *vlerë* që mund ta ruajmë në një variabël. Me këtë logjikë, këta emra *nuk* kanë nevojë të jenë klasa:

- `address`, `estimated labor charges`, `estimated parts charges`,
- `make`, `model`, `name`, `sales tax`,
- `telephone number`, `total estimated charges`, `year`.

Të gjitha këto mund të ruhen si **string** ose **numra** brenda atributeve të klasave të tjera. Pas të gjitha këtyre filtrimeve, na mbeten vetëm:

`car`, `customer`, `service quote`

Kjo do të thotë që në aplikacionin tonë do të na duhen klasa që përfaqësojnë:

- një **makinë** (`Car`);
- një **klient** (`Customer`);
- një **preventiv shërbimi** (`ServiceQuote`).

Përcaktimi i përgjegjësive të një klase

Pasi kemi gjetur klasat, hapi tjetër është të përcaktojmë **për çfarë janë përgjegjëse**. Për çdo klasë pyesim:

- *Çfarë duhet të dijë kjo klasë?* ⇒ këto janë **atributet e të dhënave**.
- *Çfarë duhet të bëjë kjo klasë?* ⇒ këto janë **metodat**.

Shpesh këto informacione gjenden tashmë në përshkrimin e domenit të problemit, por disa herë duhet të mendojmë pak më tej.

Klasa `Customer`

Në kontekstin e problemit tonë, çfarë duhet të dijë një `Customer`?

- emrin e klientit;
- adresën e klientit;
- numrin e telefonit të klientit.

Këto janë **atributet** e klasës. Çfarë duhet të bëjë klasa `Customer`?

- të inicializojë një objekt të ri `Customer`;
- të vendosë (set) dhe të kthejë (get) emrin;
- të vendosë dhe të kthejë adresën;
- të vendosë dhe të kthejë numrin e telefonit.

Pra, do të kemi një metodë `__init__`, dhe pastaj **getters/setters** për secilin atribut. Figura 3.9 mund të paraqesë diagramin UML për këtë klasë.

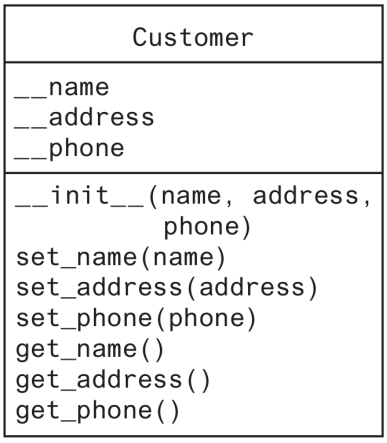

Figura 3.9: Diagram UML për klasën `Customer`.

fig: uml-customer

Programi 13 — `customer.py`

```
class Customer:
   def __init__(self, name, address, phone):
      self.__name = name
      self.__address = address
      self.__phone = phone

   def set_name(self, name):
      self.__name = name

   def set_address(self, address):
      self.__address = address

```

```
    def set_phone(self, phone):
        self.__phone = phone

    def get_name(self):
        return self.__name

    def get_address(self):
        return self.__address

    def get_phone(self):
        return self.__phone
```

Klasa `Car`

Çfarë duhet të dijë një objekt i klasës `Car`?

- markën e makinës (`make`);
- modelin e makinës (`model`);
- vitin e prodhimit (`year`).

Këto janë atributet kryesore të një makine në këtë problem. Çfarë duhet të bëjë klasa `Car`?

- të inicializojë një objekt `Car`;
- të vendosë dhe të kthejë markën;
- të vendosë dhe të kthejë modelin;
- të vendosë dhe të kthejë vitin.

Edhe këtu kemi një `__init__` dhe një grup **set/get** për secilin atribut. Diagrami UML paraqitet në Figurën 3.10.

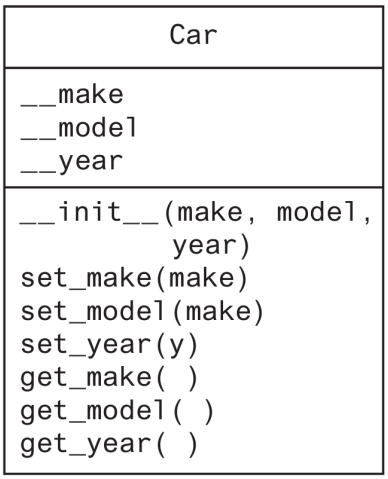

Figura 3.10: Diagram UML për klasën `Car`.

fig: uml-car

Programi 14 — `car.py`

```
class Car:
    def __init__(self, make, model, year):
        self.__make = make
        self.__model = model
        self.__year = year

    def set_make(self, make):
        self.__make = make
```

```
9
10      def set_model(self, model):
11         self.__model = model
12
13      def set_year(self, year):
14         self.__year = year
15
16      def get_make(self):
17         return self.__make
18
19      def get_model(self):
20         return self.__model
21
22      def get_year(self):
23         return self.__year
```

Klasa `ServiceQuote`

Tani shohim klasën `ServiceQuote`, e cila do të përfaqësojë një *preventiv shërbimi*. Çfarë duhet të dijë kjo klasë?

- `estimated parts charges` — kostoja e parashikuar e pjesëve;
- `estimated labor charges` — kostoja e parashikuar e punës;
- `sales tax` — taksa e shitjes;
- `total estimated charges` — totali i parashikuar.

Në implementim, ne do të ruajmë si atribute vetëm vlerat bazë (pjesët dhe puna), ndërsa taksën dhe totalin do t'i *llogarisim* me metoda. Çfarë duhet të bëjë klasa `ServiceQuote`?

- të inicializojë një objekt `ServiceQuote` me koston e pjesëve dhe të punës;
- të lejojë ndryshimin e kostos së pjesëve dhe të punës (**mutatorët**);
- të kthejë vlerat e kostos së pjesëve dhe punës (**accessorët**);
- të llogarisë dhe të kthejë **taksën e shitjes**;
- të llogarisë dhe të kthejë **totalin e parashikuar**.

Diagrami UML i kësaj klase paraqitet në Figurën 3.11.

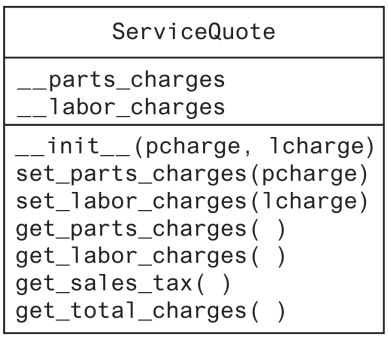

Figura 3.11: Diagram UML për klasën `ServiceQuote`.

fig: uml-servicequ

Programi 15 — `servicequote.py`

```
1  TAX_RATE = 0.05
2
3  class ServiceQuote:
4     def __init__(self, pcharge, lcharge):
```

```
5        self.__parts_charges = pcharge
6        self.__labor_charges = lcharge
7
8     def set_parts_charges(self, pcharge):
9        self.__parts_charges = pcharge
10
11    def set_labor_charges(self, lcharge):
12       self.__labor_charges = lcharge
13
14    def get_parts_charges(self):
15       return self.__parts_charges
16
17    def get_labor_charges(self):
18       return self.__labor_charges
19
20    def get_sales_tax(self):
21       return self.__parts_charges * TAX_RATE
22
23    def get_total_charges(self):
24       return (self.__parts_charges +
25             self.__labor_charges +
26             self.__parts_charges * TAX_RATE)
```

Këtu konstanta `TAX_RATE` përfaqëson normën e taksës (p.sh. 5%). Metoda `get_sales_tax` e llogarit taksën si `parts_charges * TAX_RATE`, ndërsa `get_total_charges` kthen shumën e pjesëve, punës dhe taksës. Procesi që ndoqëm (gjetja e klasave, përcaktimi i atributeve dhe metodave) është vetëm **fillimi**. Në praktikë, dizenjimi i një aplikacioni objektor është *iterativ*: sa më shumë e kupton problemin, aq më mirë mund t'i riorganizosh klasat dhe t'u përmirësosh përgjegjësitë.

KAPITULLI 4

dhe Polimorfizmi

4.1 Hyrje në trashëgiminë

KONCEPT

Trashëgimia (*inheritance*) lejon që një klasë e re të *zgjerojë* një klasë ekzistuese. Klasa e re **trashëgon** atributet dhe metodat e klasës që zgjeron.

Përgjithësimi dhe specializimi

Në botën reale, hasim shpesh objekte që janë versione të **specializuara** të objekteve të tjera më **të përgjithshme**. P.sh. termi **"insekt"** përshkruan një lloj të përgjithshëm krijese me disa karakteristika të përbashkëta. Si karkalecat, ashtu edhe bletët *bumblebee* janë insekte, pra:

- i **kanë** të gjitha karakteristikat e përgjithshme të një *insekti*;
- **plus** disa karakteristika të veçanta të tyret.

Për shembull:

- karkaleci ka aftësinë e tij karakteristike për të **kërcyer**;
- bleta *bumblebee* ka **thumbin** e saj.

Pra, karkalecat dhe bletët bumblebee janë versione të **specializuara** të konceptit të përgjithshëm **"insekt"**. Kjo ide ilustrohet në Figurën 4.1.

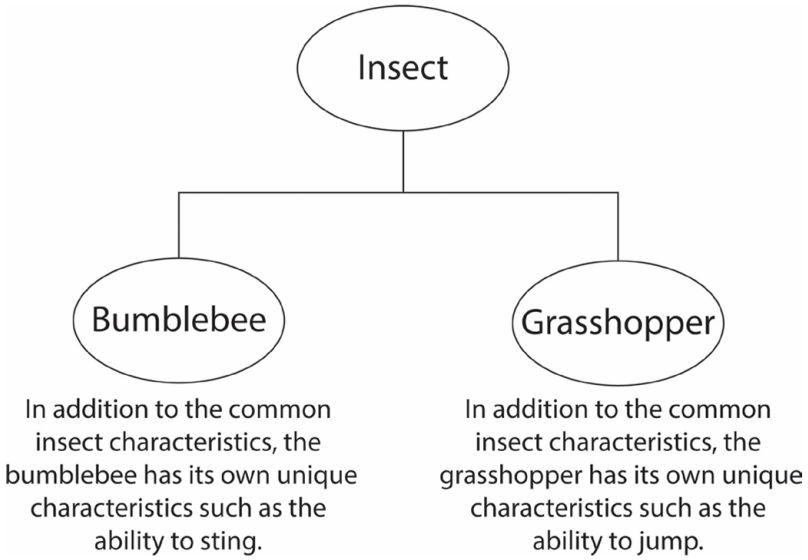

Figura 4.1: Bletët dhe karkalecat janë versione të specializuara të një insekti.

fig: inherit-insect

Në termat e programimit të orientuar nga objektet, do të thoshim:

- **Insect** është një klasë *e përgjithshme*;
- **Grasshopper** dhe **Bumblebee** janë klasa *më të specializuara* që "e zgjerojnë" idenë e një insekti.

Trashëgimia është pikërisht mekanizmi me të cilin e kthejmë këtë ide në kod.

Trashëgimia dhe marrëdhënia "është një" (*is a*)

Kur një objekt është një version i *specializuar* i një objekti tjetër, themi që ka një marrëdhënie të tipit **"është një"** (*is a*) mes tyre. Shembuj:

- një karkalec **është një** insekt;
- një *poodle* **është një** qen;
- një makinë (*car*) **është një** automjet (*vehicle*);
- një lule **është një** bimë;
- një drejtkëndësh **është një** formë (*shape*);
- një futbollist **është një** sportist.

Kur ekziston një marrëdhënie *"është një"* objekti i specializuar i ka **të gjitha** karakteristikat e objektit të përgjithshëm dhe **plus** disa karakteristika shtesë që e bëjnë "të veçantë". Në programimin e orientuar nga objektet, **trashëgimia** përdoret pikërisht për të krijuar këtë marrëdhënie *"është një"* midis klasave. Kjo na lejon:

- të **zgjerojmë** funksionalitetin e një klase ekzistuese,
- duke krijuar një klasë tjetër që është një version më i specializuar i saj.

Trashëgimia përfshin dy lloje klasash:

- **superklasa** (*superclass*) — klasa *e përgjithshme*;
- **nënklasa** (*subclass*) — klasa *e specializuar*.

Mund të mendosh nënklasën si një version të **zgjeruar** të superklasës:

- nënklasa **trashëgon** atributet dhe metodat e superklasës, pa qenë nevoja t'i rishkruajmë;
- nënklasa mund të ketë **atribute të reja** dhe **metoda shtesë**, që e bëjnë atë më të specializuar.

Terminologji

Përveç termave **superklasë** dhe **nënklasë**, shpesh përdoren edhe:

- *base class* për superklasën;
- *derived class* për nënklasën.

Të dyja palët e termave janë të sakta. Në këtë libër do të përdorim kryesisht **superklasë** dhe **nënklasë**.

Të kuptuarit e trashëgimisë së klasave

Njerëzit që merren seriozisht me programim të orientuar nga objektet mezi presin të flasin për **trashëgiminë e klasave**, **nënklasa** e kështu me radhë — gjëra që për një fillestar mund të mos kenë asnjë kuptim. Megjithatë, ajo për të cilën flasin ata si koncept në Python është diçka që ti e sheh në jetën e përditshme gjatë gjithë kohës.

Për shembull, nëse e konsiderojmë *ADN-në e qenit* si një lloj fabrike ose klase në Python, mund t'i grumbullojmë të gjithë qentë bashkë si anëtarë të një **klase kafshësh që e quajmë** `Dogs`. Edhe pse çdo qen është unik, të gjithë qentë *janë prapë qen* sepse janë anëtarë të klasës që ne e quajmë **Dogs**. Këtë mund ta ilustrojmë me një figurë si ajo në Figurën 4.2.

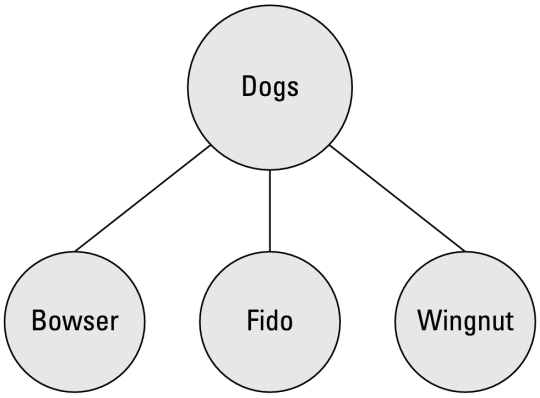

Figura 4.2: Qentë si objekte të klasës `Dog`.

fig: inherit-dogs

Pra, çdo qen është unik por ajo që i bën qentë *të ngjashëm* me njëri-tjetrin janë **karakteristikat që ata i trashëgojnë nga klasa** `Dogs`. Me fjalë të tjera, ka diçka të përbashkët "prej qenit" që vjen nga një "model i përgjithshëm" që të gjithë e ndajnë.

Idetë e **klasës** dhe **trashëgimisë së klasave** që ofron Python (dhe gjuhë të tjera të orientuara nga objektet) *nuk kanë rënë nga qielli* thjesht që të ta bëjnë më të vështirë dhe më të bezdisshëm mësimin e kësaj lënde. Shumë nga informacioni për botën që na rrethon mund të ruhet, kategorizohet dhe kuptohet më së miri pikërisht duke përdorur **klasa**, **nënklasa**, **nënnënklasa** e kështu me radhë, deri te individi i veçantë. Për shembull, mund të kesh vënë re që në planet lëvizin edhe krijesa të tjera "të ngjashme me qenin". Mund të përmendim:

- ujqit,
- kojotat,
- çakejtë.

Ata i ngjajnë qenve sepse të gjithë **e trashëgojnë "qenësinë" e tyre** nga një klasë më e përgjithshme të cilën mund ta quajmë `Canines`. Kjo paraqitet në një figurë si Figura 4.3.

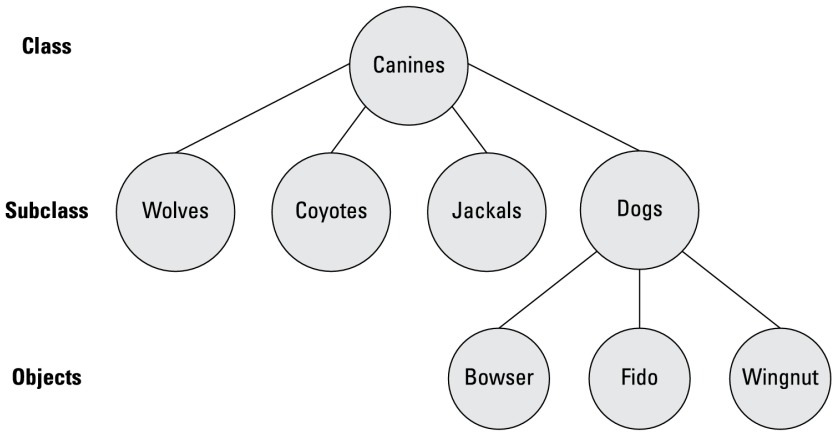

Figura 4.3: Shumë lloje të ndryshme kafshësh janë të ngjashme me qentë.

fig: inherit-canines

Duke përdorur analogjinë e qenit, nuk kemi pse të ndalemi te **kaninët**. Mund të ngjitemi edhe më lart në "hierarki":

- mbi **kaninët** mund të vendosim **gjitarët** (*mammals*), sepse të gjithë kaninët janë gjitarë;
- mbi **gjitarët** mund të vendosim **kafshët** (*animals*), sepse të gjithë gjitarët janë kafshë;
- mbi **kafshët** mund të vendosim **qeniet e gjalla** (*living things*), sepse të gjitha kafshët janë qenie të gjalla.

Pra, në thelb, gjithçka që e bën një qen "të jetë qen" vjen nga fakti që ai **trashëgon karakteristika** nga shumë "klasa" më të përgjithshme që vijnë para tij: `LivingThing` → `Animal` → `Mammal` → `Canine` → `Dog`

Natyrisht, koncepti nuk vlen vetëm për qentë. Bota ka edhe shumë **mace** të ndryshme. Ka mace të vogla të këndshme, p.sh. "Bootsy", me të cilën do ishe i lumtur të ndaje shtëpunë, dhe ka edhe plot felinë të tjerë, si: *luanë*, *tigra* dhe *jaguarë*. me të cilët *ndoshta nuk do të doje* ti mbaje në shtëpi! Edhe këto janë pjesë e një familjeje më të gjerë (*felines*) dhe ndjekin të njëjtin model klasash dhe nënklasash. Nëse bën një kërkim në internet për *"living things hierarchy"* dhe hap seksionin *Images*, do të shohësh sa shumë mënyra ka për të klasifikuar të gjitha qeniet e gjalla, dhe si **trashëgimia "zbret" nga e përgjithshmja tek e veçanta**. Kjo është pikërisht ajo ide që ne sjellim në Python me klasat dhe trashëgiminë.

Edhe analogjia me makinat mund të ndjekë të njëjtën logjikë. Në krye mund të kemi: **mjetet e transportit** (*transportation vehicles*), nën të cilat mund të vendosim:

- varka (*boats*),
- aeroplanë (*planes*),
- automjete (*automobiles*).

Nën **automjetet** mund të kemi:

- makina të zakonshme (*cars*),
- kamionë (*trucks*),
- furgonë (*vans*),

e pastaj shkojmë poshtë deri te një **makinë specifike** (p.sh. "BMWja jote e vitit 2015 me targa XXX"). Pra, **klasat dhe nënklasat nuk janë asgjë e re**: ne i përdorim këto struktura mendore gjatë gjithë kohës. E reja është thjesht ideja për t'i përfaqësuar këto gjëra përpara "makinerive pa mend" që i quajmë kompjuterë.

Trashëgimia nga këndvështrimi i kodit

Nga këndvështrimi i kodimit në Python, forma më e thjeshtë për të bërë **trashëgimi** është të krijosh **nënklasa** (*subclasses*) që trashëgojnë nga një superklasë. **Superklasa** përkufizon gjërat që vlejnë për *të gjitha instancat* e asaj klase:

- atributet e përbashkëta (p.sh. `make`, `model`, `price`),
- metodat e përbashkëta (p.sh. `get_price()`, `set_model()`).

Secila **nënklasë** përkufizon vetëm:

- gjërat që kanë kuptim *vetëm për atë nënklasë* (p.sh. `doors` për `Car`, `drive_type` për `Truck`),
- pa "prishur" ose zëvendësuar atë që vjen nga superklasa, përveç rasteve kur ne **qëllimisht** duam të rishkruajmë një metodë (këtë e quajmë *overriding*, dhe do ta shohim më vonë).

Pra, nga ana konceptuale superklasa është "*modeli i përgjithshëm*" dhe nënklasat janë "*versionet e specializuara*" që trashëgojnë gjithçka të përbashkët dhe shtojnë vetëm atë që u duhet. Më pas do të shohim pikërisht *si* ta bëjmë këtë në kod, si të shkruajmë nënklasa në Python dhe si të përfitojmë plotësisht nga trashëgimia për të organizuar më mirë programet tona.

4.2 Shembull praktikë: `Automobile`

Supozoni se po zhvillojmë një program që një autosallon (*car dealership*) do ta përdorë për të menaxhuar inventarin e makinave të përdorura. Në inventar ka tre lloje automjetesh:

- makina të zakonshme (*cars*),
- furgonë/pickup trucks (*pickup trucks*),
- automjete *SUV* (*sport utility vehicles*).

Pavarësisht llojit, autosalloni mban këto të dhëna për *çdo* automjet: marka (*make*), modeli/viti i prodhimit (*year model*), kilometrazhi (*mileage*) dhe çmimi (*price*). Pastaj, çdo lloj automjeti ka edhe të dhëna *të specializuara*:

1. Për **makinat** (`Car`): numri i dyerve (*number of doors*, p.sh. 2 ose 4).
2. Për **pickup trucks** (`Truck`): tipi i lëvizjes (*drive type*), p.sh. `'2WD'` ose `'4WD'` .
3. Për **SUV-të** (`SUV`): kapaciteti i pasagjerëve (*passenger capacity*).

Një qasje e drejtpërdrejtë, por e **dobët**, do të ishte të shkruanim tre klasa të ndara:

- një klasë `Car` me atributet: markë, model, kilometrazh, çmim, numri i dyerve;
- një klasë `Truck` me atributet: markë, model, kilometrazh, çmim, tipi i lëvizjes;
- një klasë `SUV` me atributet: markë, model, kilometrazh, çmim, kapaciteti i pasagjerëve.

Problemi është të treja klasat do të kishin shumë **atribute të përbashkëta** (make, model, mileage, price). Si pasojë:

- do të kishim shumë **kod të kopjuar**;
- nëse më vonë vendosim të shtojmë një atribut të ri të përbashkët (p.sh. ngjyra), duhet të **modifikojmë të tre klasat**.

Qasje më e mirë di të ishte që të shkruajmë një superklasë `Automobile` që ruan **të gjitha të dhënat e përgjithshme** për një automjet dhe pastaj shkruajmë nënklasa për secilin lloj specifik: `Car` , `Truck` , `SUV` . Kështu, të dhënat e përbashkëta ruhen vetëm një herë (në `Automobile`), dhe çdo nënklasë shton vetëm atë që i duhet veçanërisht. Le të fillojmë duke shkruar superklasën `Automobile` në një modul të quajtur `vehicles.py` .

Programi 1 — `Automobile` (`vehicles.py`)

```
class Automobile:
   def __init__(self, make, model, mileage, price):
      self.__make = make
      self.__model = model
      self.__mileage = mileage
      self.__price = price

   def set_make(self, make):
      self.__make = make

   def set_model(self, model):
      self.__model = model

   def set_mileage(self, mileage):
      self.__mileage = mileage

   def set_price(self, price):
      self.__price = price

   def get_make(self):
      return self.__make

   def get_model(self):
      return self.__model

   def get_mileage(self):
      return self.__mileage

   def get_price(self):
      return self.__price
```

Metoda `__init__` pranon si argumente: `make` është marka e automjetit, `model` është modeli/viti, `mileage` është kilometrazhi dhe `price` është çmimi. Këto vlera përdoren për të inicializuar atributet private:

`__make`, `__model`, `__mileage` dhe `__price`

Kujto nga kapitulli i mëparshëm: kur emri i një atributi fillon me `__`, atributi bëhet **i fshehur** (*hidden*) nga kodi jashtë klasës. Kjo ndodh sepse Python bën *name mangling* të emrit (`__make` shndërrohet në një emër të brendshëm). Metodat `set_make`, `set_model`, `set_mileage`, `set_price` janë **metoda modifikuese** (*mutators*): ato *ndryshojnë* vlerat e atributeve. Metodat `get_make`, `get_model`, `get_mileage`, `get_price` janë **metoda aksesuese** (*accessors*): ato *kthejnë* vlerat, por nuk i ndryshojnë. `Automobile` është një klasë *e plotë*: mund të krijojmë objekte prej saj dhe t'i përdorim pa problem. P.sh. mund të shkruanim një program që importon modulin `vehicles` dhe krijon instanca të klasës `Automobile`. Por kjo klasë përmban vetëm të dhënat **e përgjithshme** për një automjet. Nuk përmban asnjërën nga të dhënat specifike që autosalloni do të mbajë për:

- makinat (`Car`),
- pickup trucks (`Truck`),
- SUV-të (`SUV`).

Për këto lloje konkrete automjetesh, do të shkruajmë **nënklasa** që trashëgojnë nga `Automobile`. Tani do të shkruajmë klasën `Car`, e cila përfaqëson një makinë të zakonshme. Kjo klasë do të jetë nënklasë e `Automobile`.

Programi 2 — Klasa `Car` (pjesë e `vehicles.py`)

```
class Car(Automobile):
   def __init__(self, make, model, mileage, price, doors):
      Automobile.__init__(self, make, model, mileage, price)
      self.__doors = doors

   def set_doors(self, doors):
      self.__doors = doors

   def get_doors(self):
      return self.__doors
```

Shiko me kujdes header-in e klasës në rreshtin:

```
class Car(Automobile):
```

Kjo tregon që po përkufizojmë një klasë `Car` që **trashëgon** nga klasa `Automobile`. Pra:

- `Car` është **nënklasa**;
- `Automobile` është **superklasa**.

Në gjuhë natyrore mund ta shprehim marrëdhënien si: `Car` **është një** `Automobile`. Do të thotë se çdo objekt `Car` është automjet, por me disa të dhëna shtesë (numrin e dyerve). Header-i i metodës `__init__` është:

```
def __init__(self, make, model, mileage, price, doors):
```

Përveç parametrave standardë: `make`, `model`, `mileage`, `price` që janë të njëjtë me `Automobile`, klasa `Car` merr edhe: `doors` — numri i dyerve. Disa nga këta atribute krijohen nga superklasa `Automobile`, prandaj brenda `__init__` thërrasim metodën `__init__` të superklasës:

```
Automobile.__init__(self, make, model, mileage, price)
```

Kjo deklaratë:

- thërret metodën `__init__` të klasës `Automobile`;
- i kalon `self` (objektin aktual) dhe vlerat `make`, `model`, `mileage`, `price`;
- si rezultat, inicializohen atributet `__make`, `__model`, `__mileage`, `__price`.

Pastaj, në rreshtin:

```
self.__doors = doors
```

krijohet atributi privat `__doors` dhe inicializohet me vlerën e parametrin `doors`. Metoda `set_doors` është **mutatori** për `__doors`, ndërsa `get_doors` është **aksesuesi** për të njëjtin atribut. Përpara se të vazhdojmë me nënklasat e tjera, le të demonstrojmë klasën `Car`.

Programi 3 — `car_demo.py`

```
import vehicles

def main():
    used_car = vehicles.Car('Audi', 2007, 12500, 21500.0, 4)

    print('Make:', used_car.get_make())
    print('Model:', used_car.get_model())
    print('Mileage:', used_car.get_mileage())
    print('Price:', used_car.get_price())
    print('Number of doors:', used_car.get_doors())

if __name__ == '__main__':
    main()
```

Shembull output-i:

```
Make: Audi
Model: 2007
Mileage: 12500
Price: 21500.0
Number of doors: 4
```

Në rreshtin:

```
import vehicles
```

importohet moduli `vehicles`, i cili përmban përkufizimet e klasave `Automobile` dhe `Car` (dhe më vonë do të shtojmë edhe klasat `Truck` dhe `SUV`). Rreshti:

```
used_car = vehicles.Car('Audi', 2007, 12500, 21500.0, 4)
```

krijon një instancë të klasës `Car`. Parametrat:

- `'Audi'` — marka,
- `2007` — modeli/viti,
- `12500` — kilometrazhi,
- `21500.0` — çmimi,

- `4` — numri i dyerve,

i kalohen metodës `__init__` të klasës `Car`, e cila:

- thërret `Automobile.__init__` për të inicializuar atributet e përgjithshme;
- inicializon atributin `__doors`.

Deklaratat:

```
used_car.get_make()
used_car.get_model()
used_car.get_mileage()
used_car.get_price()
```

thërrasin metodat `get_make`, `get_model`, `get_mileage`, `get_price`. Vër re se këto metoda *nuk* janë përkufizuar në klasën `Car`; ato janë trashëguar nga superklasa `Automobile`. Kjo është fuqia e trashëgimisë: **nënklasa përdor drejtpërdrejt metodat e superklasës**. Deklarata:

```
used_car.get_doors()
```

thërret metodën `get_doors`, e cila është specifike për klasën `Car`. Tani do të shohim klasën `Truck`, e cila përfaqëson një pickup truck. Edhe kjo klasë do të trashëgojë nga `Automobile`.

Programi 4 — Klasa `Truck` (pjesë e `vehicles.py`)

```
class Truck(Automobile):
   def __init__(self, make, model, mileage, price, drive_type):
      Automobile.__init__(self, make, model, mileage, price)
      self.__drive_type = drive_type

   def set_drive_type(self, drive_type):
      self.__drive_type = drive_type

   def get_drive_type(self):
      return self.__drive_type
```

Metoda `__init__` merr:

- `make`, `model`, `mileage`, `price` — si më parë;
- `drive_type` — p.sh. `'2WD'` ose `'4WD'`.

Ashtu si te klasa `Car`, thërrasim `Automobile.__init__` për të inicializuar atributet e përgjithshme. Pastaj krijohet atributi privat `__drive_type`, i cili ruan tipin e lëvizjes. Metoda `set_drive_type` është mutatori për `__drive_type`, ndërsa `get_drive_type` është aksesuesi. Së fundi, do të krijojmë klasën `SUV`, e cila përfaqëson një *sport utility vehicle*.

Programi 5 — Klasa SUV (pjesë e vehicles.py)

```
class SUV(Automobile):
   def __init__(self, make, model, mileage, price, pass_cap):
      Automobile.__init__(self, make, model, mileage, price)
      self.__pass_cap = pass_cap

   def set_pass_cap(self, pass_cap):
      self.__pass_cap = pass_cap

   def get_pass_cap(self):
      return self.__pass_cap
```

Metoda `__init__` merr:

- `make`, `model`, `mileage`, `price` — të njëjtat si më parë;
- `pass_cap` — kapacitetin e pasagjerëve (p.sh. 5).

Sërish, thërritet `Automobile.__init__` për atributet e përgjithshme, dhe pastaj inicializohet atributi privat `__pass_cap`. Metodat `set_pass_cap` dhe `get_pass_cap` janë respektivisht mutatori dhe aksesuesi i këtij atributi. Tani le të shohim një program që i përdor të gjitha këto klasa: krijon një objekt `Car`, një objekt `Truck` dhe një objekt `SUV` dhe shfaq të dhënat e tyre.

Programi 6 — car_truck_suv_demo.py

```
import vehicles

def main():
   car = vehicles.Car('BMW', 2001, 70000, 15000.0, 4)
   truck = vehicles.Truck('Toyota', 2002, 40000, 12000.0, '4WD')
   suv = vehicles.SUV('Volvo', 2000, 30000, 18500.0, 5)

   print('USED CAR INVENTORY')
   print('===================')

   print('The following car is in inventory:')
   print('Make:', car.get_make())
   print('Model:', car.get_model())
   print('Mileage:', car.get_mileage())
   print('Price:', car.get_price())
   print('Number of doors:', car.get_doors())
   print()

   print('The following pickup truck is in inventory.')
   print('Make:', truck.get_make())
   print('Model:', truck.get_model())
   print('Mileage:', truck.get_mileage())
   print('Price:', truck.get_price())
   print('Drive type:', truck.get_drive_type())
   print()

   print('The following SUV is in inventory.')
   print('Make:', suv.get_make())
   print('Model:', suv.get_model())
   print('Mileage:', suv.get_mileage())
   print('Price:', suv.get_price())
   print('Passenger Capacity:', suv.get_pass_cap())

if __name__ == '__main__':
   main()
```

Shembull output-i:

```
USED CAR INVENTORY
==================
The following car is in inventory:
Make: BMW
Model: 2001
Mileage: 70000
Price: 15000.0
Number of doors: 4

The following pickup truck is in inventory.
Make: Toyota
Model: 2002
Mileage: 40000
Price: 12000.0
Drive type: 4WD

The following SUV is in inventory.
Make: Volvo
Model: 2000
Mileage: 30000
Price: 18500.0
Passenger Capacity: 5
```

Në `main` krijojnë tre objekte:

- `car` — një `Car` me markë `'BMW'`, vit `2001`, `70000` km, çmim `15000.0`, 4 dyer;
- `truck` — një `Truck` me markë `'Toyota'`, vit `2002`, `40000` km, çmim `12000.0`, lëvizje `'4WD'`;
- `suv` — një `SUV` me markë `'Volvo'`, vit `2000`, `30000` km, çmim `18500.0`, kapacitet 5 pasagjerë.

Për secilin objekt:

- metodat `get_make`, `get_model`, `get_mileage`, `get_price` vijnë nga superklasa `Automobile`;
- metodat `get_doors`, `get_drive_type`, `get_pass_cap` janë specifike për nënklasat përkatëse.

Ky shembull tregon qartë idenë: `Car`, `Truck`, `SUV` **janë** `Automobile`. Secila nënklasë:

- **trashëgon** kodin e përbashkët nga `Automobile`;
- **shton** vetëm atë që e bën të veçantë.

4.3 Trashëgimia në diagramet UML

Në **diagramet UML** (*Unified Modeling Language*) trashëgimia paraqitet me një **vijë** që lidh nënklasën me superklasën, me një **shigjetë trekëndore bosh** që tregon nga *superklasa*. Më saktë:

- secila **klasë** paraqitet si një drejtkëndësh (zakonisht i ndarë në seksione: emri i klasës, atributet, metodat);
- ndërmjet dy klasave që kanë marrëdhënie trashëgimie, vizatohet një **vijë** nga nënklasa në superklasë;
- skaji pranë **superklasës** ka një **shigjetë trekëndore bosh** (*open arrowhead*).

Për shembull, marrëdhënia mes `Automobile`, `Car`, `Truck` dhe `SUV` mund të paraqitet si në figurën më poshtë: `Car`, `Truck` dhe `SUV` janë nënklasa që *"janë"* `Automobile`.
Diagrami lexohet kështu:

- `Car` *është një* `Automobile`;
- `Truck` *është një* `Automobile`;
- `SUV` *është një* `Automobile`.

Në shumicën e projekteve, diagramet UML përdoren për të **planifikuar** klasat përpara se të shkruajmë kodin — ose për të **dokumentuar** kodin ekzistues në mënyrë më vizuale.

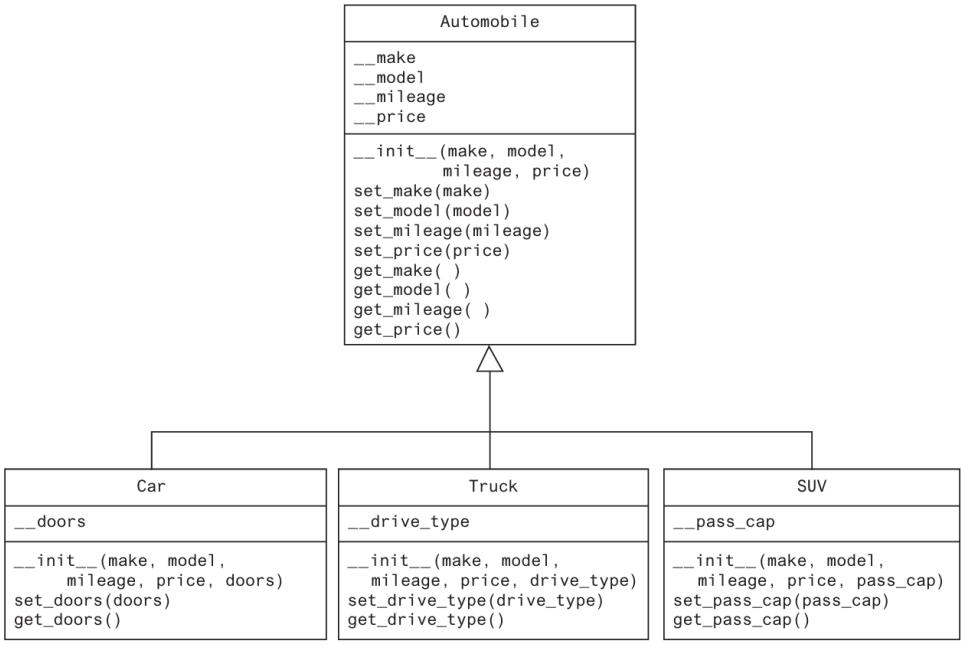

Figura 4.4: Trashëgimia mes klasave `Automobile`, `Car`, `Truck` dhe `SUV` në UML.

fig: uml-vehicles

4.4 Polimorfizmi

KONCEPT

Polimorfizmi (*polymorphism*) lejon që nënklasat të kenë **metoda me të njëjtin emër** si superklasa, por me sjellje të ndryshme. Pastaj programi zgjedh **versionin e duhur të metodës** në varësi të **llojit të objektit** që e thërret metodën.

Fjala *polimorfizëm* do të thotë "të kesh shumë forma". Në programimin e orientuar nga objektet, një **interface i njëjtë** (p.sh. emri i një metode) mund të ketë **zbatim të ndryshëm** në klasa të ndryshme. Polimorfizmi ka dy "përbërës" kryesorë:

1. **Rishkrimi i metodave** (*method overriding*) — definon një metodë në superklasë dhe pastaj *të njëjtin emër metode* në nënklasë. Metoda e nënklasës "mbulon" (*overrides*) versionin e superklasës.

2. **Thirrja e versionit të duhur** — kur thërret një metodë përmes një objekti, Python vendos automatikisht nëse duhet të përdorë versionin e superklasës apo të nënklasës, në varësi të **llojit real** të objektit.

Në fakt, ti e ke parë tashmë një shembull: çdo nënklasë që kemi përdorur deri tani ka pasur metodën `__init__` që **rishkruan** metodën `__init__` të superklasës. Kur krijojmë një objekt të nënklasës, është `__init__` **i nënklasës** ai që ekzekutohet. Le të shohim një shembull të thjeshtë për të kuptuar polimorfizmin. Do të ndërtojmë një modul `animals.py`, ku kemi një superklasë `Mammal` që përfaqëson një *gjitar të përgjithshëm* dhe dy nënklasa `Dog` dhe `Cat` që specializojnë sjelljen.

Programi 7 — Klasa `Mammal` (`animals.py`)

```
class Mammal:
   def __init__(self, species):
      self.__species = species

   def show_species(self):
      print('I am a', self.__species)

   def make_sound(self):
      print('Grrrrr')
```

Kjo klasë ka tre metoda:

- `__init__` — ruan specien në atributin privat `__species`;
- `show_species()` — shfaq tekstin “ `I am a...` ”;
- `make_sound()` — bën një tingull të përgjithshëm.

Tani le të krijojmë nënklasat `Dog` dhe `Cat`:

```
class Dog(Mammal):
   def __init__(self):
      Mammal.__init__(self, 'Dog')

   def make_sound(self):
      print('Woof! Woof!')

class Cat(Mammal):
   def __init__(self):
      Mammal.__init__(self, 'Cat')

   def make_sound(self):
      print('Meow')
```

Vër re dy pika të rëndësishme:

- `Dog` dhe `Cat` **trashëgojnë** metodat `show_species` dhe `make_sound` nga `Mammal`.
- Në të dyja klasat, metoda `make_sound` **rishkruhet** (*overrides*) për të dhënë tingullin tipik të qenit/maces.

Shembull përdorimi:

```
import animals

mammal = animals.Mammal('regular mammal')
mammal.show_species()
mammal.make_sound()

dog = animals.Dog()
dog.show_species()
dog.make_sound()

cat = animals.Cat()
cat.show_species()
cat.make_sound()
```

Shembull output-i

```
I am a regular mammal
Grrrrr
I am a Dog
Woof! Woof!
I am a Cat
Meow
```

Kur thirret `make_sound()` mbi një objekt `Dog`, Python ekzekuton versionin **e klasës** `Dog`, jo atë të `Mammal`. Kjo është polimorfizëm në veprim.

Funksione që punojnë me shumë lloje objektesh

Forca e polimorfizmit vihet re kur shkruan funksione që nuk "dallojnë" se çfarë nënklase po përdorin, mjafton që objekti të ketë *metodat e duhura*. Për shembull, funksioni `show_mammal_info` mund të pranojë një objekt `Mammal`, një objekt `Dog` ose një objekt `Cat` dhe do të thërrasë gjithmonë versionin e duhur të metodave, sipas llojit të objektit.

Programi 9 — `polymorphism_demo.py`

```python
import animals

def main():
    mammal = animals.Mammal('regular animal')
    dog = animals.Dog()
    cat = animals.Cat()

    show_mammal_info(mammal)
    show_mammal_info(dog)
    show_mammal_info(cat)

def show_mammal_info(creature):
    creature.show_species()
    creature.make_sound()

if __name__ == '__main__':
    main()
```

Shembull output-i:

```
I am a regular animal
Grrrrr
I am a Dog
Woof! Woof!
I am a Cat
Meow
```

Vëzhgim: Funksioni `show_mammal_info` nuk ka asnjë `if` ose `elif` për të parë nëse objekti është `Dog` apo `Cat`. Ai thjesht thërret metodat. Python zgjedh automatikisht versionin e duhur në bazë të llojit të objektit — kjo është **polimorfizëm**.

Programi 10 — `wrong_type.py`

Çfarë ndodh nëse dikush thërret funksionin `show_mammal_info` me një objekt që **nuk** është `Mammal`, p.sh. me një `string`?

```
def main():
   show_mammal_info('I am a string')

def show_mammal_info(creature):
   creature.show_species()
   creature.make_sound()

if __name__ == '__main__':
   main()
```

Kur interpreti të provojë të ekzekutojë `creature.show_species()` do të ndodhë një `AttributeError`, sepse objektet `str` nuk kanë metodën `show_species`. Për të shmangur këtë, përdorim funksionin e brendshëm `isinstance`. `isinstance(obj, ClassName)` kthen `True` nëse `obj` është:

- instancë e `ClassName`, **ose**
- instancë e një **nënklase** të `ClassName`.

Në të kundërt kthen `False`.

Programi 11 — `polymorphism_demo2.py`

Le ta përmirësojmë shembullin tonë duke përdorur `isinstance` që të sigurohemi se argumenti është vërtet një `Mammal` (ose nënklasë e tij):

```
import animals

def main():
   mammal = animals.Mammal('regular animal')
   dog = animals.Dog()
   cat = animals.Cat()

   show_mammal_info(mammal)
   show_mammal_info(dog)
   show_mammal_info(cat)

   show_mammal_info('I am a string')

def show_mammal_info(creature):
   if isinstance(creature, animals.Mammal):
      creature.show_species()
      creature.make_sound()
   else:
      print('That is not a Mammal!')

if __name__ == '__main__':
   main()
```

Shembull output-i:

```
I am a regular animal
Grrrrr
I am a Dog
Woof! Woof!
I am a Cat
Meow
That is not a Mammal!
```

Tani:

- kur i kalojmë `Mammal`, `Dog` ose `Cat`, funksioni ekzekuton metodat `show_species` dhe `make_sound`;
- kur i kalojmë një objekt që **nuk** është `Mammal` (as nënklasë e tij), funksioni shfaq mesazhin `'That is not a Mammal!'`.

Ky kombinim i **trashëgimisë**, **polimorfizmit** dhe **kontrollit të tipit** të lejon të ndërtosh programe fleksibël, ku i njëjti funksion punon natyrshëm me objekte të ndryshme, për sa kohë ato "*janë një*" instancë e klasës së duhur.

4.5 Përdorimi i `super()` dhe zgjerimi i metodave të superklasës

Deri tani, kur kemi dashur të thërrasim konstruktorin e superklasës (`Automobile`) nga nënklasa (`Car`, `Truck`, `SUV`), kemi përdorur formën:

```
Automobile.__init__(self, make, model, mileage, price)
```

Kjo formë është e saktë dhe funksionon pa problem, por Python na ofron një mënyrë më të qartë dhe më fleksibël për të bërë të njëjtën gjë: funksionin `super()`. Ideja është e thjeshtë: `super()` është "një referencë" tek superklasa, e cila përdoret për të thirrur metodat që vijnë nga ajo. Në vend të:

```
class Car(Automobile):
    def __init__(self, make, model, mileage, price, doors):
        Automobile.__init__(self, make, model, mileage, price)
        self.__doors = doors
```

mund të shkruajmë:

Programi 12 — `vehicles2.py`

```
class Car(Automobile):
    def __init__(self, make, model, mileage, price, doors):
        super().__init__(make, model, mileage, price)
        self.__doors = doors
```

Në këtë formë, `super().__init__(...)` ka të njëjtin efekt: thërret konstruktorin e superklasës `Automobile` dhe inicializon atributet e përgjithshme. Por kodi është më i pastër, më i lehtë për t'u lexuar dhe më i lehtë për t'u mirëmbajtur. Nëse nesër vendosim ta ndryshojmë emrin e superklasës, `super()` vazhdon të funksionojë pa pasur nevojë të ndërrojmë çdo thirrje `Automobile.__init__` në kod.

Pse vlen `super()`

`super()` nuk është "magji e re", por një mënyrë më e mirë për të bërë atë që tashmë bënim:

- kodi bëhet më i qartë: ne po i themi Python-it "përdor versionin e metodës nga superklasa";
- shmangim përsëritjen e emrit të klasës në çdo thirrje;
- funksionon më mirë në hierarki më komplekse (p.sh. trashëgimi me më shumë se një nivel).

Zgjerim dhe jo zëvendësim i metodave të prindit

Derisa tani, shembujt tanë kanë rishkruar metoda për t'i *zëvendësuar* sjelljet e superklasës (p.sh. `make_sound()` te `Dog` dhe `Cat`). Por shpesh ne duam diçka më të nuancuar: duam të *mbajmë* atë që bën superklasa **dhe** të

shtojmë diçka sipër. Mendo, për shembull, që klasa `Automobile` ka një metodë që shfaq informacionin bazë për automjetin:

Programi 13 — `vehicles3.py`

```
class Automobile:
   def __init__(self, make, model, mileage, price):
      self.__make = make
      self.__model = model
      self.__mileage = mileage
      self.__price = price

   def show_info(self):
      print('Make:', self.__make)
      print('Model:', self.__model)
      print('Mileage:', self.__mileage)
      print('Price:', self.__price)
```

Tani duam që `Car` të ketë të njëjtën metodë `show_info`, por me një rresht shtesë që tregon numrin e dyerve. Në vend që ta kopjojmë të gjithë kodin e `show_info` nga `Automobile`, bën më shumë sens ta *zgjerojmë* metodën ekzistuese:

Programi 13 — `vehicles3.py`

```
class Car(Automobile):
   def __init__(self, make, model, mileage, price, doors):
      super().__init__(make, model, mileage, price)
      self.__doors = doors

   def show_info(self):
      super().show_info()
      print('Number of doors:', self.__doors)
```

Në këtë mënyrë, `Car.show_info` nuk e hedh poshtë metodën e superklasës, por e **përdor** atë si bazë dhe shton pjesën e vet. Nëse nesër e ndryshojmë `show_info` tek `Automobile` (p.sh. shtojmë një fushë të re si ngjyra), `Car.show_info` përfiton automatikisht nga ky ndryshim, sepse thërret `super().show_info()`.

Ky model "*thirr prindin, pastaj bëj diçka ekstra*" është shumë i zakonshëm në programe të mëdha me trashëgimi:

- superklasa përkufizon sjelljen *standard* për të gjithë,
- nënklasat e zgjerojnë këtë sjellje me detajet e tyre specifike.

E njëjta ide nuk vlen vetëm për `__init__` apo për metoda që shfaqin të dhëna. Mund të përdorësh `super()` në çdo metodë të rishkruar ku dëshiron të ruash logjikën bazë të superklasës, por edhe të shtosh rregulla ose hapa të tjerë. Për shembull, mund të kesh një metodë `calculate_price()` në superklasë dhe nënklasa të ndryshme që shtojnë taksa, zbritje ose tarifa sipas llojit të automjetit, gjithmonë duke nisur nga llogaritja standarde e prindit.

Nëse e sheh trashëgiminë si një piramidë idesh, `super()` është ura që të lejon të ndërtosh një "kat të ri" pa prishur katet poshtë: ti i përdor metodat e superklasës si bazë dhe i zgjeron pa pasur nevojë të kopjosh apo dublosh kodin.

4.6 Atributet e klasës dhe të instancës

Deri tani kemi punuar me *atributet e instancës* pa i quajtur me këtë emër: çdo herë që në `__init__` kemi shkruar diçka si `self.__make = make`, kemi krijuar atribute që i përkasin *objektit konkret* (*instancës*). Por në Python mund të kemi edhe *atribute të klasës*, që i përkasin vetë klasës si "lloj", jo një objekti të veçantë.

Atributet e instancës vs atributet e klasës

Atributet e instancës:

- krijohen zakonisht në `__init__` me `self`;
- çdo objekt ka **kopjen e vet** të këtyre vlerave.

Atributet e klasës:

- përkufizohen brenda klasës, por *jashtë* metodave;
- janë të **përbashkëta** për të gjitha instancat e asaj klase.

Programi 14 — `dogs.py`

Le të shohim një shembull të thjeshtë ku i përdorim të dyja llojet e atributeve:

```
class Dog:
   # Atribut i klasës: i përbashkët për të gjithë qentë
   species = 'Canis familiaris'

   def __init__(self, name, age):
      # Atribute të instancës: ndryshojnë për çdo qen
      self.name = name
      self.age = age

def main():
   d1 = Dog('Luna', 3)
   d2 = Dog('Rex', 5)

   print(d1.name, d1.age, d1.species)
   print(d2.name, d2.age, d2.species)

   # Mund të lexojmë atributin e klasës edhe nga vetë klasa
   print('Specia e të gjithë qenve është:', Dog.species)

if __name__ == '__main__':
   main()
```

Shembull output-i:

```
Luna 3 Canis familiaris
Rex 5 Canis familiaris
Specia e të gjithë qenve është: Canis familiaris
```

Çfarë po ndodh këtu?

- `name` dhe `age` janë **atribute të instancës**: Luna dhe Rex kanë emra dhe mosha të ndryshme.
- `species` është **atribut i klasës**: *të gjithë* qenve u takon e njëjta specie, prandaj ka kuptim të ruhet një herë në klasë.

Vër re që ne e lexojmë `species` si `d1.species` dhe `d2.species`, por në të vërtetë vlera ruhet në `Dog.species`. Python së pari kërkon atributin te objekti; nëse nuk e gjen, e kërkon te klasa.

Kur ka kuptim të përdorim atribute të klasës?

Atributet e klasës janë të dobishme kur ke informacion që:

- nuk i përket një objekti të veçantë,
- por përshkruan **vetë llojin** e objekteve.

Programi 14 — `dogs2.py`

Për shembull, mund të duam të numërojmë sa objekte janë krijuar nga një klasë:

```
class Automobile:
   # Atribut i klasës: numri total i automjeteve të krijuara
   count = 0

   def __init__(self, make, model):
      self.__make = make
      self.__model = model

      # Sa herë krijohet një objekt i ri, rrisim numërimin
      Automobile.count += 1


def main():
   a1 = Automobile('Audi', 2007)
   a2 = Automobile('BMW', 2015)
   a3 = Automobile('Toyota', 2020)

   print('Janë krijuar', Automobile.count, 'automjete.')

if __name__ == '__main__':
   main()
```

Këtu `count` nuk është një vlerë "personale" e një makine të vetme, por një informacion për *të gjithë klasën* `Automobile`. Prandaj është natyrale ta ruajmë si atribut klase.

Programi 15 — `dogs3.py`

Atributet e klasë mund të jenë edhe *private*.

```
class Automobile:
   # Atribut i klasës: numri total i automjeteve të krijuara
   __count = 0

   @classmethod
   def count(cls):
      return cls.__count

   def __init__(self, make, model):
      self.__make = make
      self.__model = model

      # Sa herë krijohet një objekt i ri, rrisim numërimin
      Automobile.__count += 1

def main():
   a1 = Automobile('Audi', 2007)
   a2 = Automobile('BMW', 2015)
   a3 = Automobile('Toyota', 2020)

   print('Janë krijuar', Automobile.count(), 'automjete.')

if __name__ == '__main__':
   main()
```

Në kontekstin e trashëgimisë, një atribut i klasës i superklasës mund të **përdoret** ose edhe të **mbivendoset** nga nënklasat. P.sh. të gjitha nënklasat e `Mammal` mund të trashëgojnë një atribut klase `kingdom = 'Animalia'`, ndërsa një nënklasë mund të shtojë atribute të tjera të klasës që i duhen asaj në mënyrë specifike.

Metodat e instancës dhe metodat e klasës

Në pamje të parë, *të gjitha* metodat brenda klasave tona kanë qenë të një lloji: ato që marrin `self` si parametrin e parë. Këto janë **metoda të instancës**. Por Python njeh edhe **metoda të klasës** (`@classmethod`), të cilat punojnë me vetë klasën si objekt.

Dy llojet kryesore të metodave

Metodat e instancës:

- kanë parametrin e parë `self`;
- përdoren për të punuar me *një objekt të veçantë*.

Metodat e klasës:

- kanë parametrin e parë `cls`;
- shënohen me dekoratorin `@classmethod`;
- punojnë me *klasën si një e tërë* (p.sh. lexojnë/modifikojnë atribute të klasës, krijojnë instanca).

Metodat e instancës janë ato që kemi përdorur gjithë kohën:

```
class Car(Automobile):
   def __init__(self, make, model, mileage, price, doors):
      super().__init__(make, model, mileage, price)
      self.__doors = doors

   def get_doors(self):
      return self.__doors

   def show_info(self):
      print('Make:', self.get_make())
      print('Model:', self.get_model())
      print('Doors:', self.__doors)
```

Këto metoda lexojnë ose ndryshojnë gjendjen e *objektit konkret* (`car1`, `car2`, ...). Kur thërret:

```
car1.show_info()
```

Python i kalon automatikisht `car1` si argumentin `self`. Tani le të shtojmë një metodë të klasës te `Automobile`, që punon me atributin e klasës `count` nga shembulli i mëparshëm. Ne duam një mënyrë të thjeshtë për të pyetur: "Sa automjete janë krijuar deri tani?" pa pasur nevojë të dimë një objekt të veçantë.

Programi 16 — `automobile.py`

```
class Automobile:
   __count = 0

   def __init__(self, make, model):
      self.__make = make
      self.__model = model
      Automobile.__count += 1

   @classmethod
   def how_many(cls):
      print('Deri tani janë krijuar', cls.__count, 'automjete.')
```

Vër re dy gjëra:

- Metoda `how_many` është shënuar me `@classmethod`.
- Parametri i parë i saj është `cls`, jo `self`. Ky parametrë përfaqëson *klasën* `Automobile` (ose nënklasën, nëse metoda thirret nga një nënklasë).

Kjo metodë mund të thirret në dy mënyra të natyrshme:

```
def main():
   Automobile.how_many() # thirrje përmes klasës

   a1 = Automobile('Audi', 2007)
   a2 = Automobile('BMW', 2015)

   Automobile.how_many() # tani count është 2
   a2.how_many() # funksionon

if __name__ == '__main__':
   main()
```

Edhe kur e thërret `a2.how_many()`, Python përsëri i kalon *klasën* si argumentin `cls`. Pra, metodat e klasës janë menduar për të punuar me:

- atribute të klasës (`cls.count`),
- logjikë që ka kuptim për llojin e objektit, jo për një instancë të veçantë.

Programi 17 — `automobile2.py`

Një përdorim shumë i zakonshëm i metodave të klasës është ndërtimi i **konstruktorëve alternativë**. Për shembull, mund të duam të krijojmë një automjet nga një `string` e formatuar në një mënyrë të veçantë:

```
class Automobile:
   def __init__(self, make, model):
      self.__make = make
      self.__model = model

   @classmethod
   def from_string(cls, text):
      __make, __model_str = text.split('-')
      model = int(__model_str)
      return cls(__make, model)

   def __str__(self):
      return f"{self.__make} {self.__model}"

def main():
   auto1 = Automobile.from_string('Audi-2007')
   print(auto1)

if __name__ == '__main__':
   main()
```

Këtu `from_string`:

- është metodë klase sepse nuk punon me një objekt ekzistues, por **krijon një të ri**;
- përdor `cls` në vend të `Automobile`, kështu që nëse një nënklasë e trashëgon këtë metodë, ajo do të krijojë instanca të nënklasës (jo gjithmonë `Automobile`).

Në kombinim me trashëgiminë, kjo është shumë e fuqishme: e njëjta metodë e klasës mund të funksionojë *polimorfikisht* për superklasën dhe për të gjitha nënklasat e saj, pa qenë nevoja ta kopjojmë kodin në çdo klasë.

4.7 Marrëdhëniet “is-a” dhe “has-a”

Deri tani kemi folur kryesisht për marrëdhënien **“është një”** (*is-a*): `Car` *është një* `Automobile`, `Dog` *është një* `Mammal`, etj. Por në modelimin e programeve reale hasim shpesh edhe një lloj tjetër marrëdhënieje: **“ka një”** (*has-a*). Kjo lidhet me konceptin e **kompozitës** (*composition*).

“is-a” (është një):

- përdoret kur një klasë është një **lloj më i specializuar** i një klase tjetër;
- në kod përfaqësohet me **trashëgimi** (`class Car(Automobile): ...`).

“has-a” (ka një):

- përdoret kur një objekt **përbëhet** nga objekte të tjera;
- në kod përfaqësohet me **kompozitë**: një objekt ruan të tjerë si atribute (`self.engine`, `self.address`, ...).

Programi 18 — `vehicles4.py`

Le të kombinojmë të dyja idetë në një shembull të thjeshtë. Do të kemi:

- `Automobile` — superklasë e përgjithshme;
- `Car` — nënklasë: **Car “është një” Automobile**;
- `Engine` — klasë më vete që përfaqëson motorin;
- `Car` **“ka një” Engine** si atribut.

```
class Engine:
   def __init__(self, horsepower):
      self.__horsepower = horsepower

   def start(self):
      print('Engine starting...')

   def get_horsepower(self):
      return self.__horsepower
class Automobile:
   def __init__(self, make, model):
      self.__make = make
      self.__model = model

   def show_basic_info(self):
      print('Make:', self.__make)
      print('Model:', self.__model)

class Car(Automobile): # Car IS-A Automobile
   def __init__(self, make, model, engine):
      super().__init__(make, model)
      # Car HAS-A Engine
      self.__engine = engine

   def start(self):
      print('Starting car...')
      self.__engine.start()

   def show_info(self):
      self.show_basic_info()
      print('Horsepower:', self.__engine.get_horsepower())
```

```python
def main():
    engine = Engine(150)
    my_car = Car('Toyota', 2020, engine)

    my_car.show_info()
    my_car.start()

if __name__ == '__main__':
    main()
```

Vër re:

- `Car` **trashëgon** nga `Automobile` (marrëdhënie **is-a**);
- `Car` ka një atribut privat `__engine` që mban një objekt `Engine` (marrëdhënie **has-a**).

Kur të përdorim trashëgiminë dhe kur kompozitën?

Përdor trashëgiminë (`is-a`) kur fjalia: `SubClass` **është një** `SuperClass` ka kuptim në gjuhë natyrore. P.sh. " `Car` është një `Automobile` ".

Përdor kompozitën (`has-a`) kur fjalia: `ClassA` **ka një** `ClassB` përshkruan më mirë marrëdhënien. P.sh. " `Car` ka një `Engine` ".

Rregull praktik: nëse nuk mund ta thuash me gojë me "*është një*", por më shumë me "*ka një*", **mos** përdor trashëgimi — përdor kompozitë.

Në projekte reale, shpesh kompozita është më fleksibël dhe më e lehtë për t'u mirëmbajtur se trashëgimia. Trashëgiminë ia vlen ta përdorësh kur ndodhemi përballë një hierarkie të qartë *nga e përgjithshmja tek e veçanta* (si shembujt me `Automobile` , `Car` , `SUV` , `Truck`).

4.8 Duck typing dhe `isinstance`

Në shembujt me `Mammal` , `Dog` dhe `Cat` pamë që mund të përdorim `isinstance` për të kontrolluar llojin e një objekti përpara se ta përdorim në një funksion. Por Python ka edhe një filozofi tjetër shumë tipike: **duck typing**.

Parimi i *duck typing*

Shprehja klasike (e përkthyer lirisht) është:

"Nëse ecën si rosë dhe bën zhurmë si rosë, atëherë është rosë."

Në Python kjo do të thotë:

- nuk më intereson shumë se *çfarë lloji* është objekti;
- mjafton që objekti të ketë **metodat** që pres (`make_sound` , `show_species` , . . .);
- nëse i ka ato metoda, unë e përdor — nëse jo, programi jep gabim.

Programi 19 — `duck.py`

Le të shohim një shembull ku **nuk** kemi trashëgimi fare, por gjithsesi polimorfizmi funksionon, sepse po përdorim duck typing.

```python
class Dog:
   def make_sound(self):
      print('Woof! Woof!')

class Cat:
   def make_sound(self):
      print('Meow')

class Robot:
   def make_sound(self):
      print('Beep boop!')

def play_sound(creature):
   # Duck typing: supozojmë që ka metodën make_sound
   creature.make_sound()

def main():
   d = Dog()
   c = Cat()
   r = Robot()

   play_sound(d) # Woof! Woof!
   play_sound(c) # Meow
   play_sound(r) # Beep boop!

if __name__ == '__main__':
   main()
```

Vër re:

- `Dog`, `Cat` dhe `Robot` **nuk** trashëgojnë nga një superklasë e përbashkët;
- funksioni `play_sound` nuk përdor `isinstance`;
- i vetmi "kërkim" është: objekti të ketë metodën `make_sound`.

Ky është **duck typing**: për sa kohë objekti sillet si "rosë" (ka metodat që duam), ne e trajtojmë si të tillë.

Kur mungon metoda

Çfarë ndodh nëse dikush thërret `play_sound` me një objekt që nuk ka metodën `make_sound`? P.sh.:

```python
play_sound('I am a string')
```

Python do të hedhë një `AttributeError` kur të provojë të ekzekutojë `creature.make_sound()`. Në filozofinë e duck typing, kjo është krejtësisht e pranueshme: *ti* si programues ia ke bërë të qartë që funksioni punon me objekte që kanë `make_sound`, dhe nëse dikush i kalon një lloj tjetër, është gabimi i tij.

Megjithatë, ndonjëherë duam të jemi më tolerantë dhe të japim një mesazh më të qartë për përdoruesin, ose të shmangim rrëzimin e programit. Atëherë rikthehemi te `isinstance`.

Krahasimi: duck typing vs `isinstance`

Le të marrim shembullin me gjitarët që pamë më parë dhe ta shohim në dy versione: një me duck typing dhe një me `isinstance`.

```python
def show_creature_info(creature): # Supozojmë që creature ka show_species() dhe make_sound()
   creature.show_species()
   creature.make_sound()
```

Ky funksion do të punojë për çdo objekt që ka këto dy metoda, pavarësisht nëse është `Mammal`, `Dog`, `Cat` apo një klasë krejt tjetër që i implementon këto metoda.

```
import animals

def show_mammal_info(creature):
    if isinstance(creature, animals.Mammal):
        creature.show_species()
        creature.make_sound()
    else:
        print('That is not a Mammal!')
```

Ky funksion është më i "rreptë":

- pranon vetëm objekte që janë `Mammal` ose nënklasa të tij;
- refuzon objekte të tjera, edhe nëse ato *rastësisht* kanë metodat `show_species` dhe `make_sound`.

Duck typing apo `isinstance`?

Prefero duck typing kur:

- je në një kontekst tipik Python-ik (kod i brendshëm, jo "library publike");
- nuk ka rëndësi lloji i saktë i objektit, për sa kohë ai ka metodat që duhen;
- do fleksibilitet maksimal (edhe klasa të tjera të papara mund të punojnë me funksionin tënd).

Përdor `isinstance` kur:

- ke nevojë të **garantosh** që objekti është pjesë e një hierarkie të caktuar (p.sh. duhet të jetë `Mammal` sepse më vonë do ta ruash në një strukturë të posaçme);
- po shkruan funksione apo libra kode që do t'i përdorin njerëz të tjerë dhe dëshiron **mesazhe gabimi më të qarta**;
- logjika e programit *ndryshon* sipas llojit (`if isinstance(x, Car): ... elif isinstance(x, Truck): ...`).

Në Python të dyja qasjet janë të vlefshme. Si programues, ti zgjedh se cilën të përdorësh në varësi të saktësisë që kërkon dhe fleksibilitetit që dëshiron t'i japësh programeve të tua.

Appendices

SHTESA A

Zgjidhje të Ushtrimeve të Zgjedhura

Këtu mund të vendosësh zgjidhje për disa ushtrime kyçe nga kapitujt e librit.